AI VIET NAM
@aivietnam.edu.vn

Data Ingestion and Processing using PySpark

Dương Nguyễn Thuận – Senior TA

Anh Dương - STA

Year 2026 Quiz-Feedback Code-data Proofreaders: Phạm Tiến - QA/QC: Nguyễn Thông

Objective

❖ ETL Pipeline





- Clean
- Join
- Aggregate

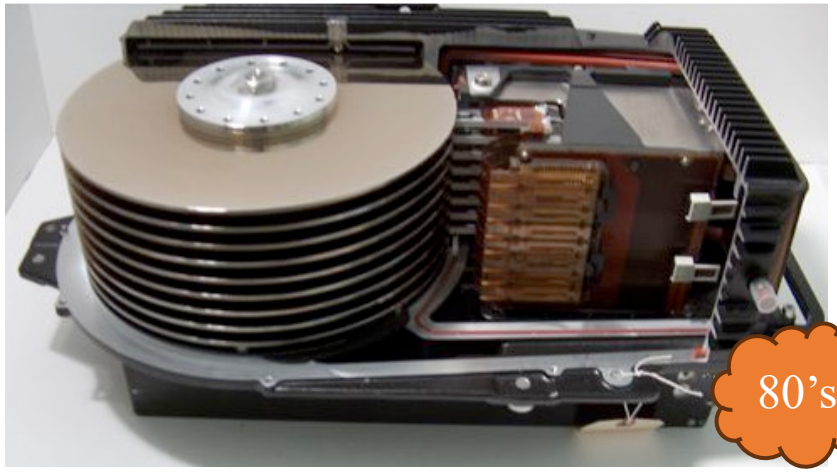
Outline

- Data Ingestion
- Apache Spark
- PySpark
- Questions

Introduction

Introduction

❖ The evolution of hard drives



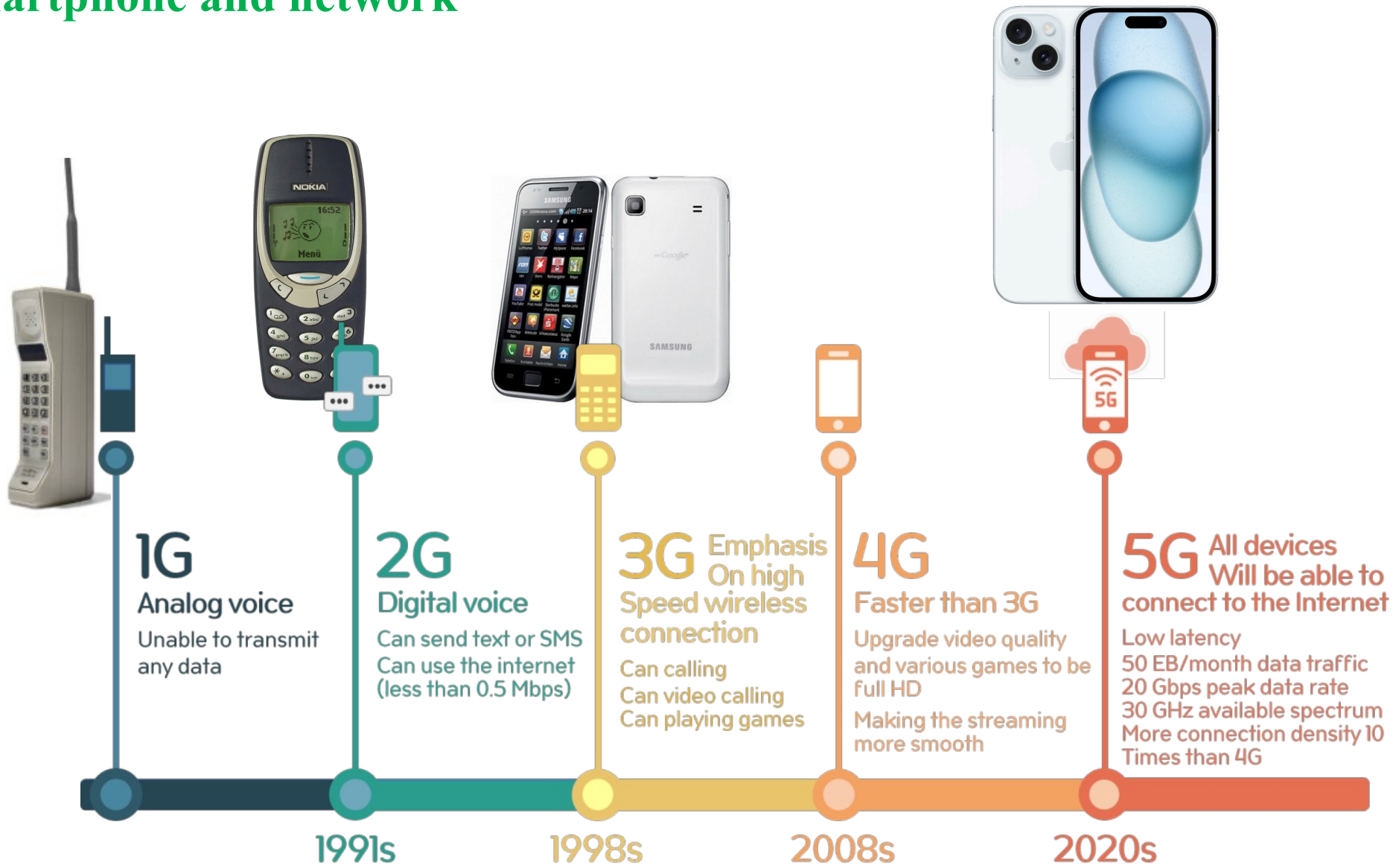






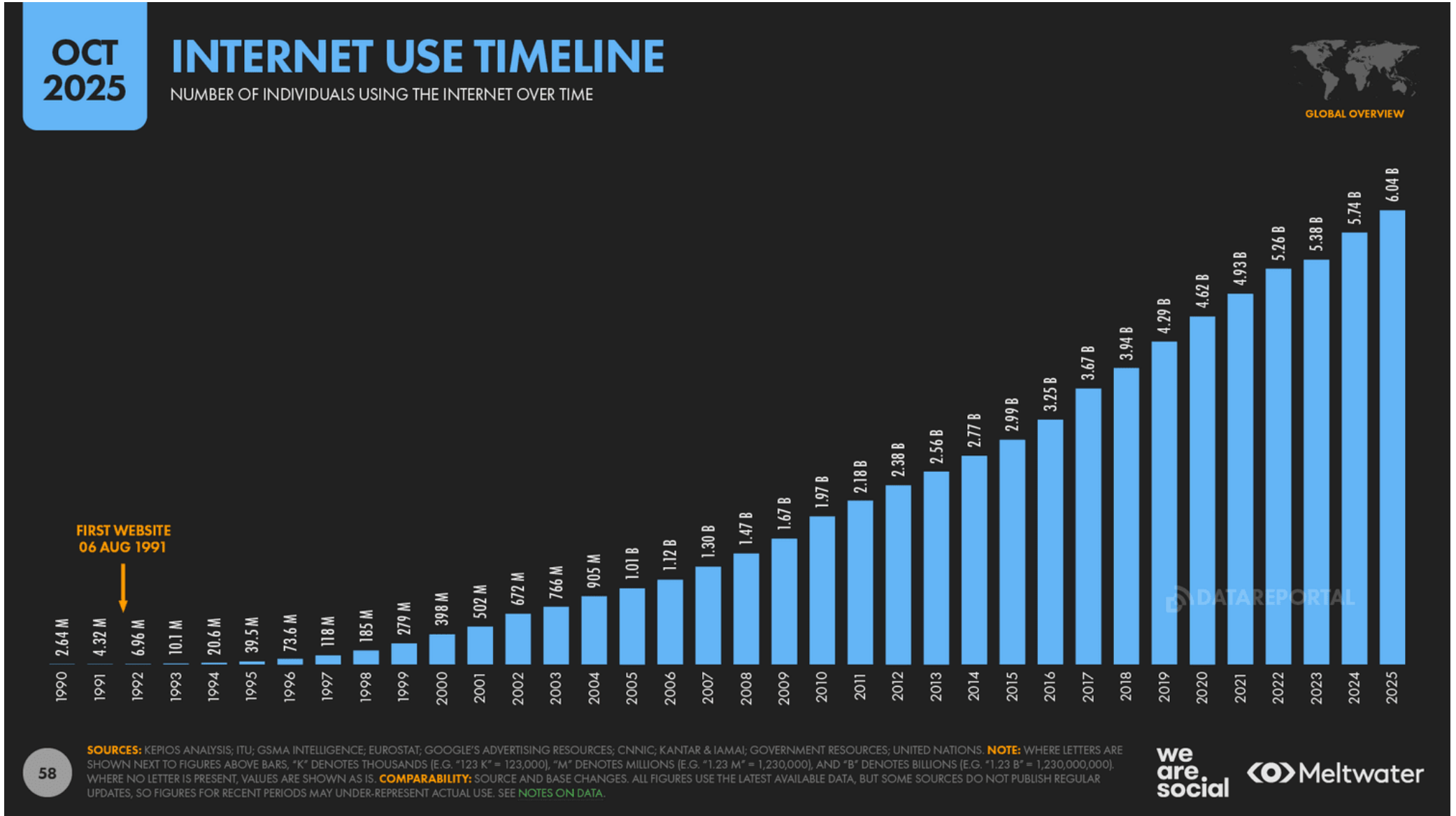
Introduction

❖ Smartphone and network



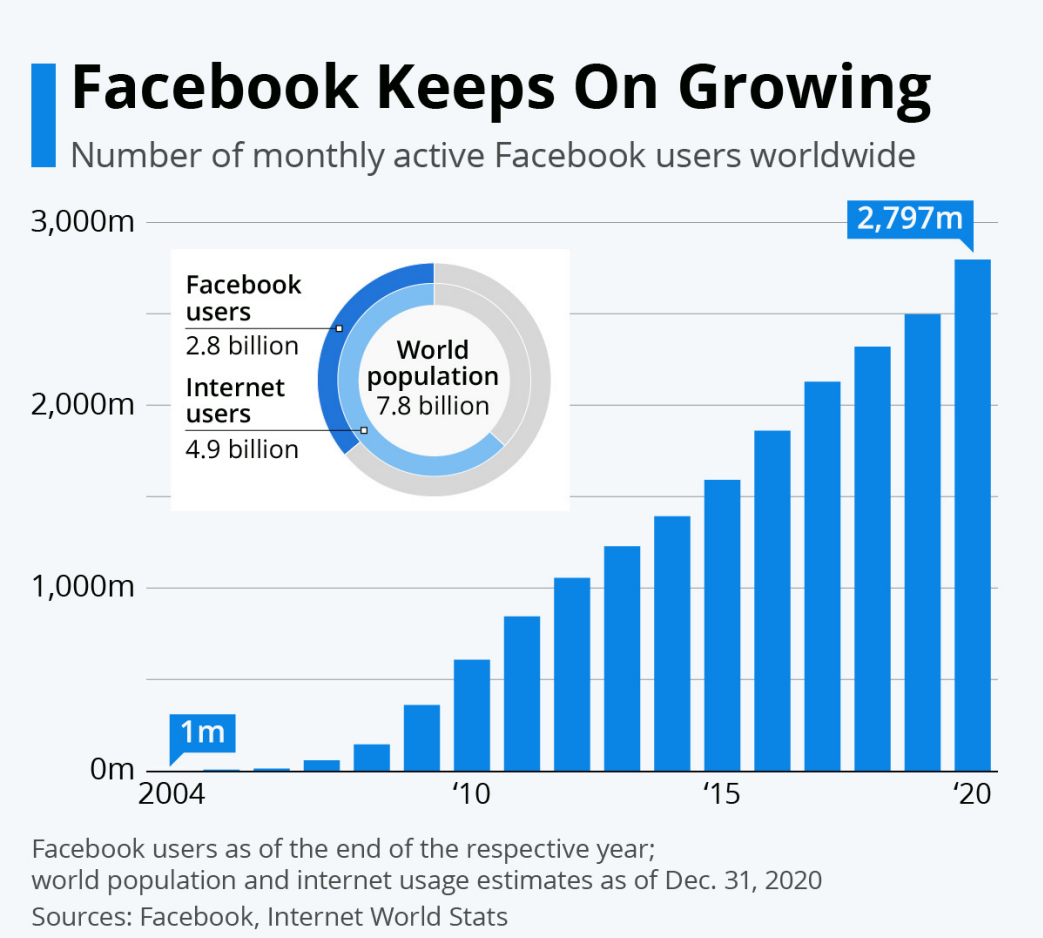
Introduction

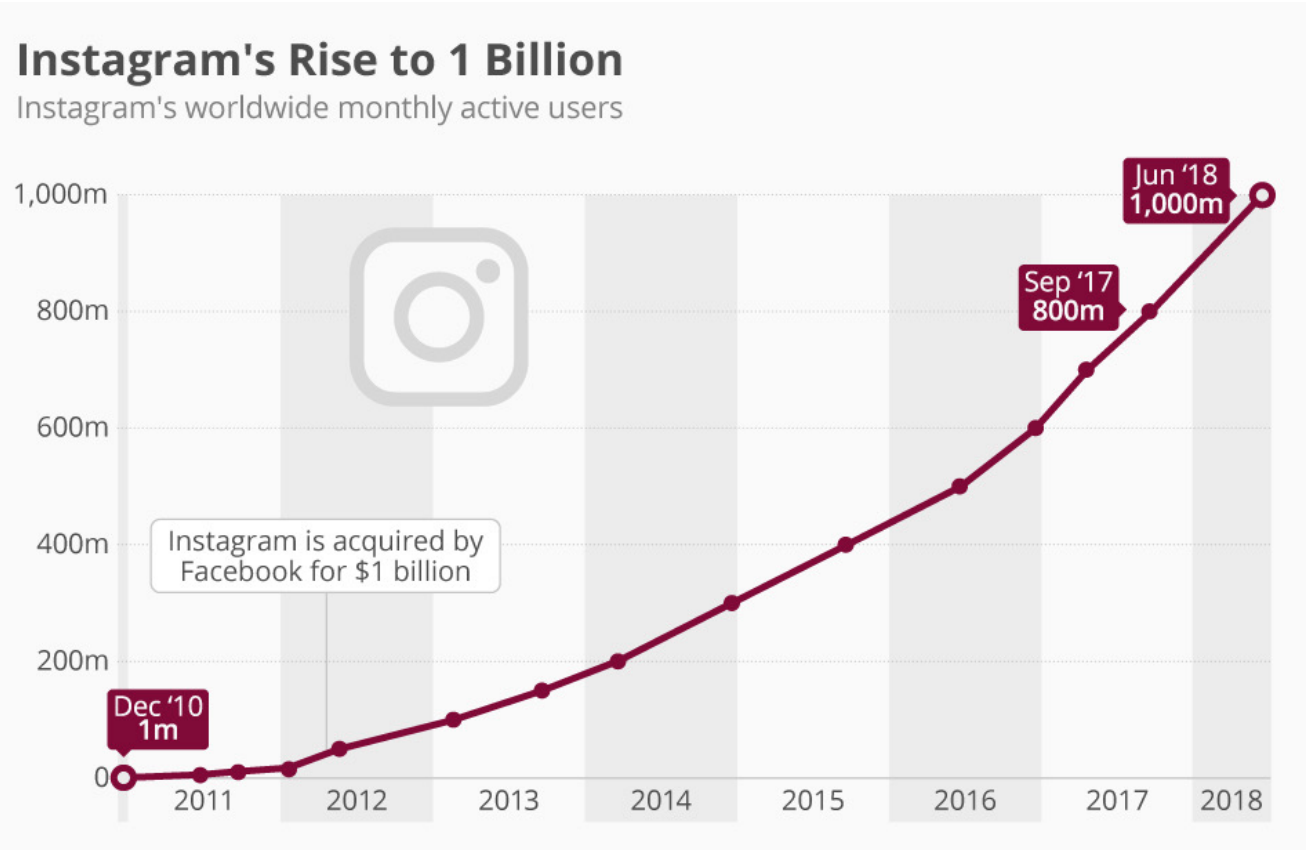
❖ The Internet



Introduction

❖ Social media

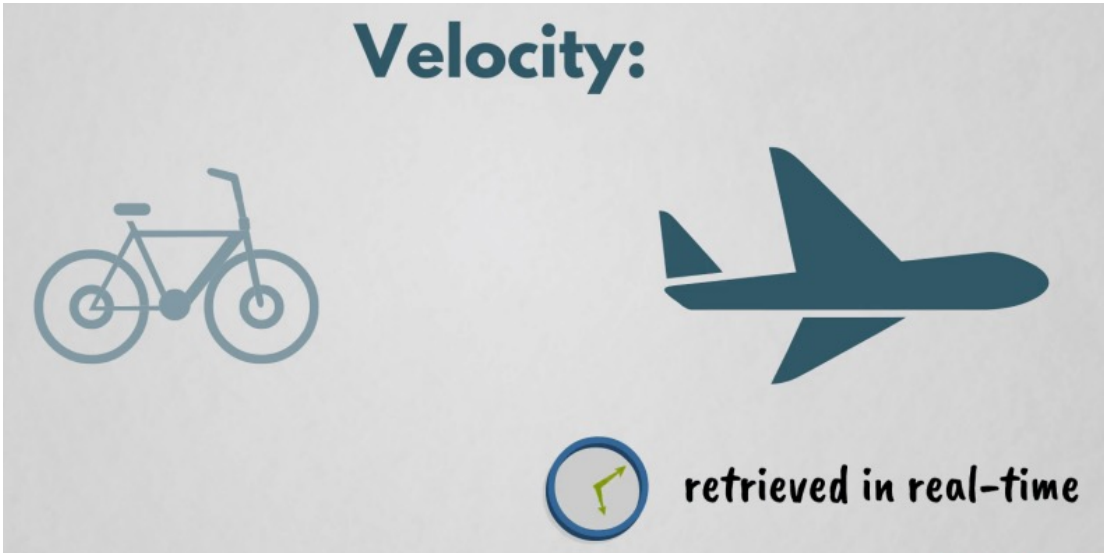


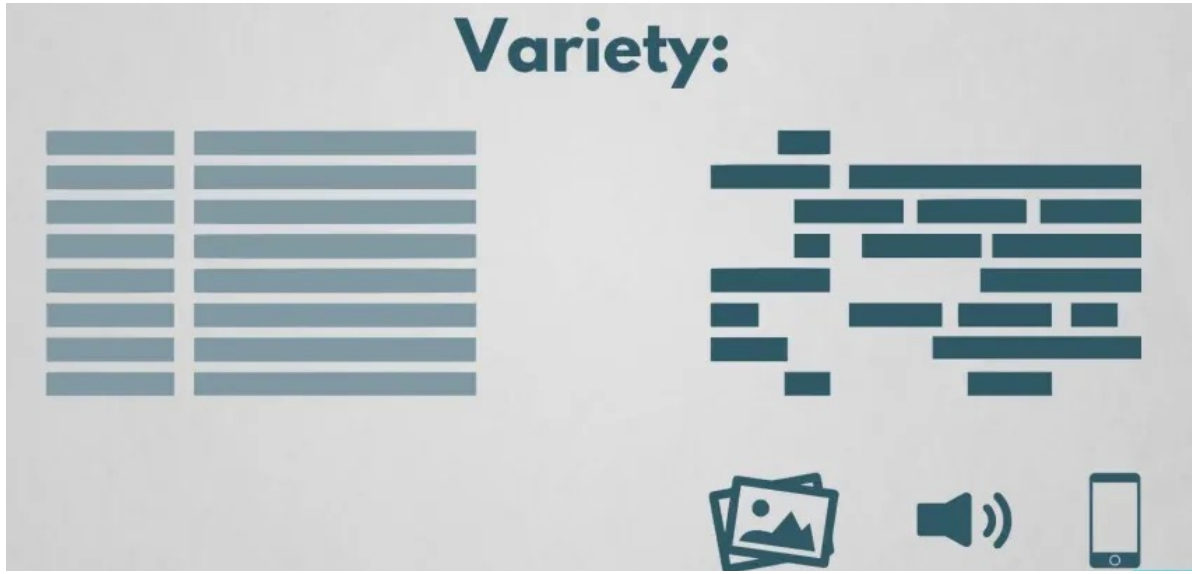


Introduction

❖ Challenges

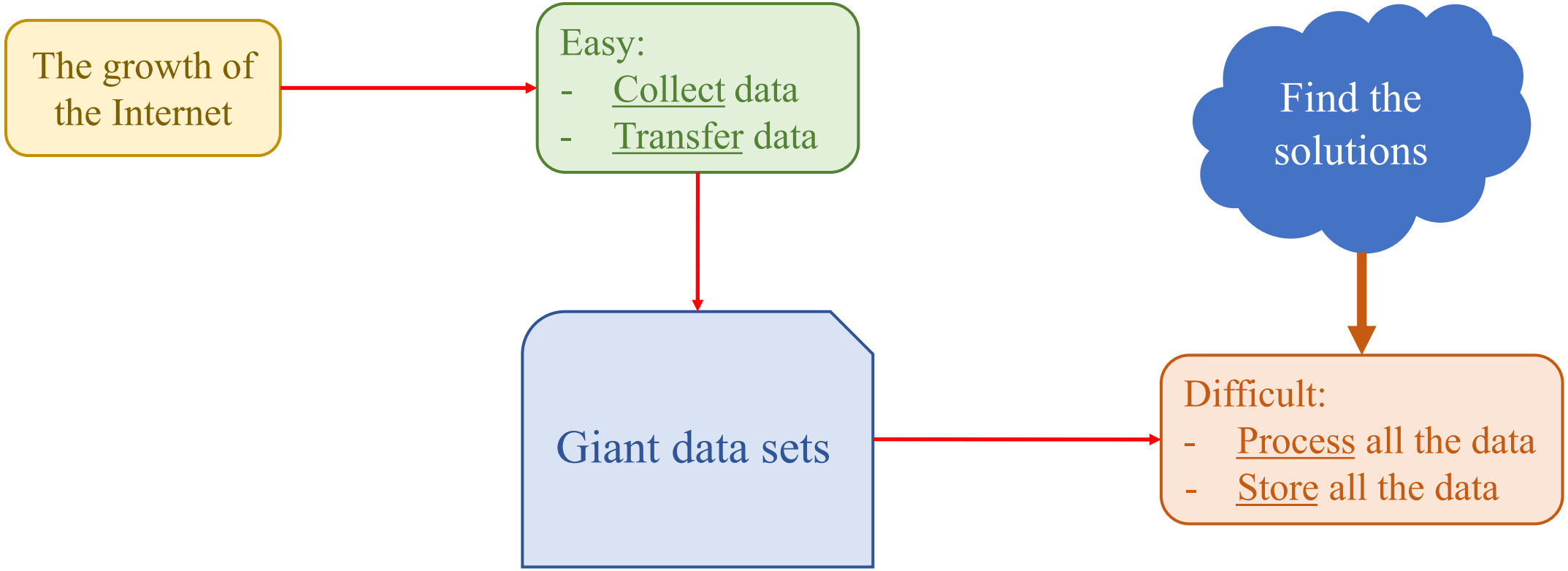






Introduction

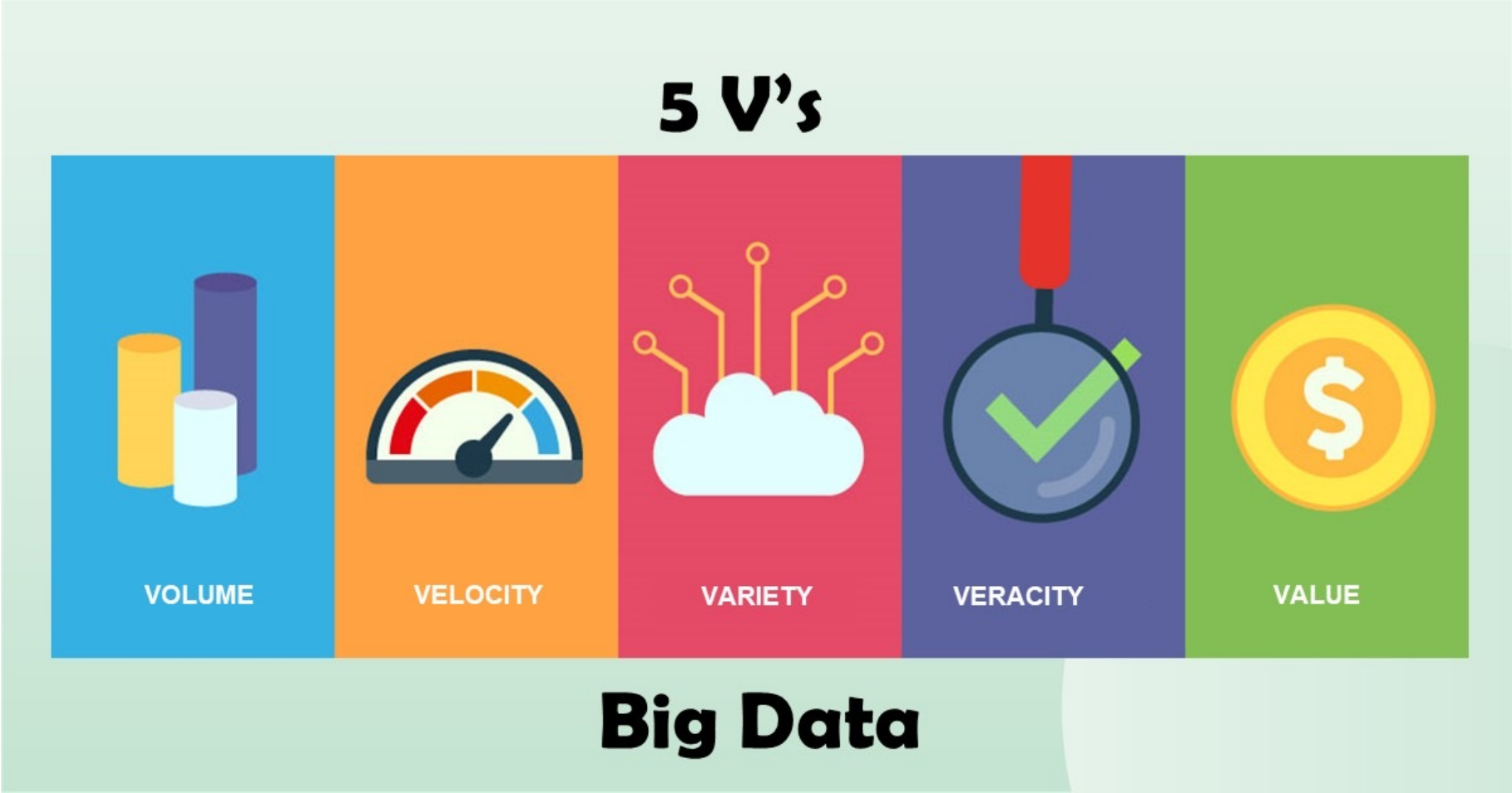
❖ Challenges



Introduction

❖ Big Data

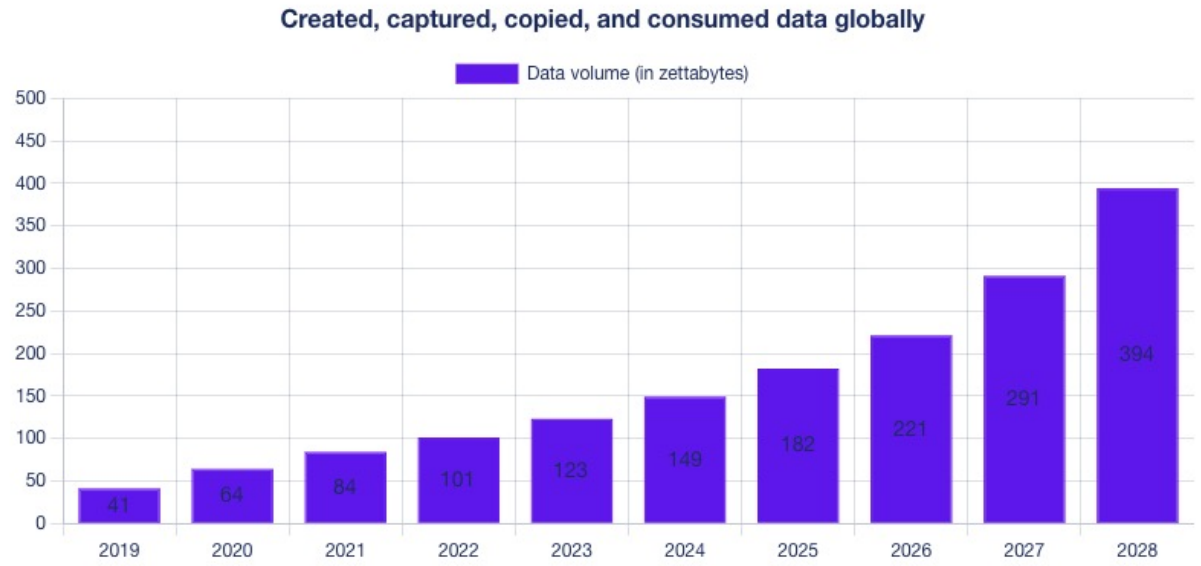
- Big data refers to extremely large and complex datasets that are too large or complex to be processed by traditional data processing applications.



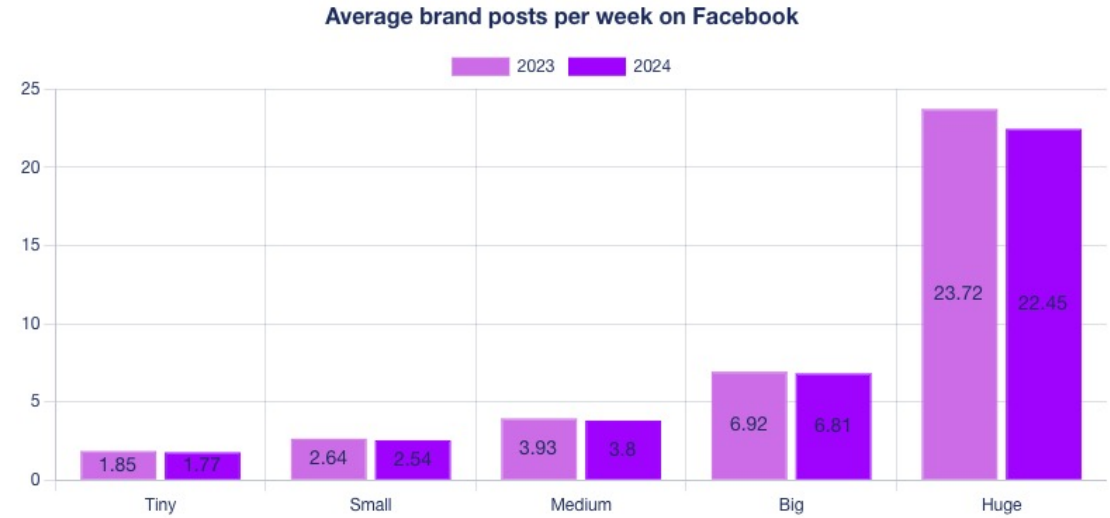
Introduction

❖ Volume

1. **402.74 million** terabytes of data is generated on the internet per day.
2. The amount of data on the internet will reach **182 zettabytes** in 2025. This figure includes all forms of data, including created, captured, copied and consumed information across the world.
3. By 2028 this total is projected to grow to 394 zettabytes.

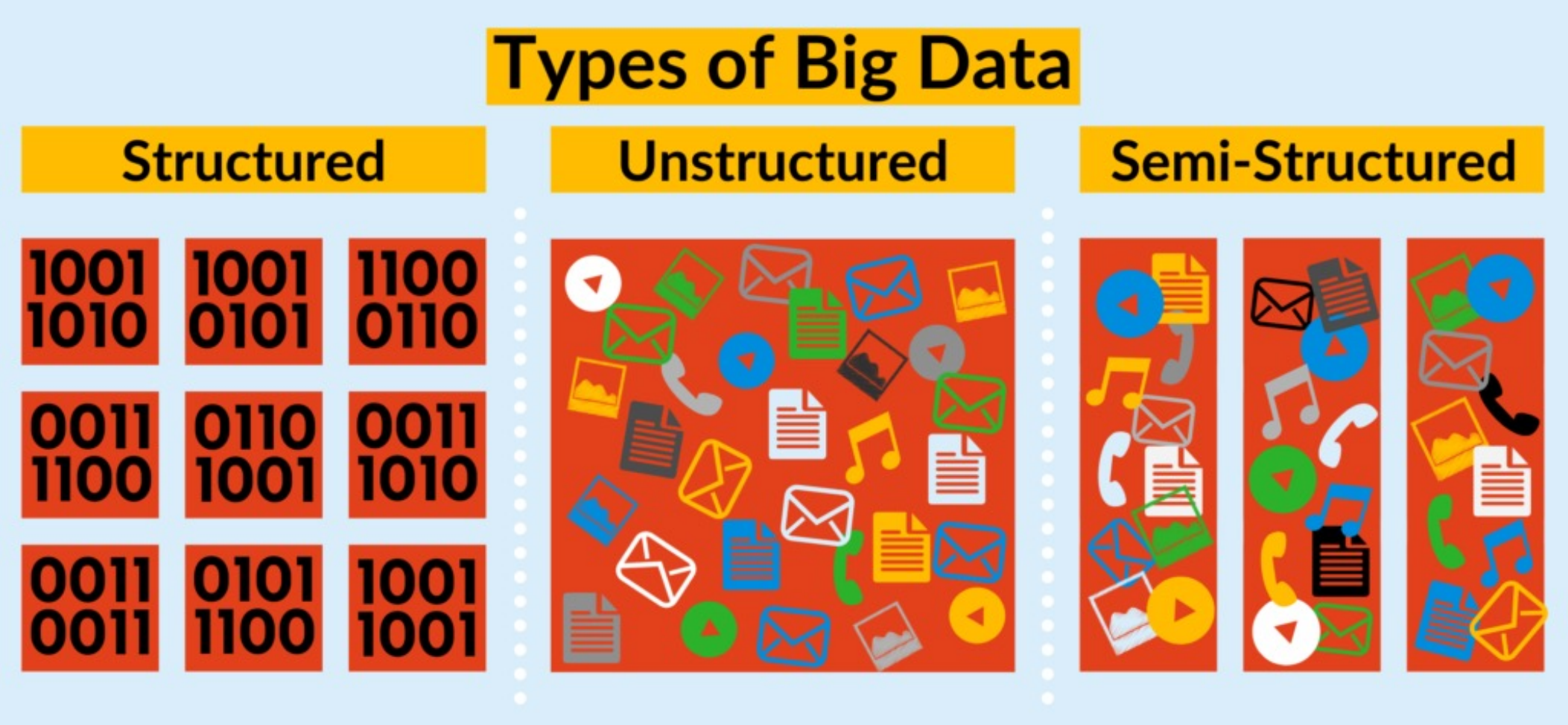


1. Facebook generates **5 petabytes** of data per day.
2. Facebook scaled its data warehouse to **300 petabytes** in 2014.
3. Facebook accounts with over 50,000 followers post **23.72 times per week** on average.



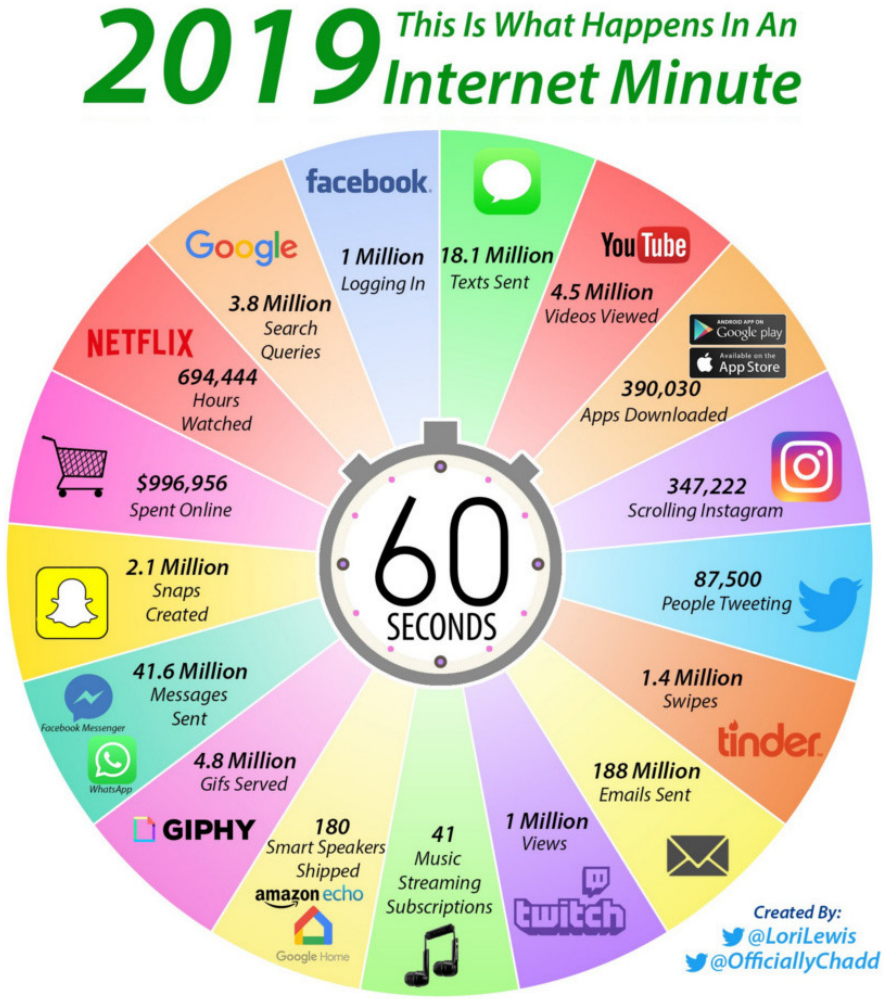
Introduction

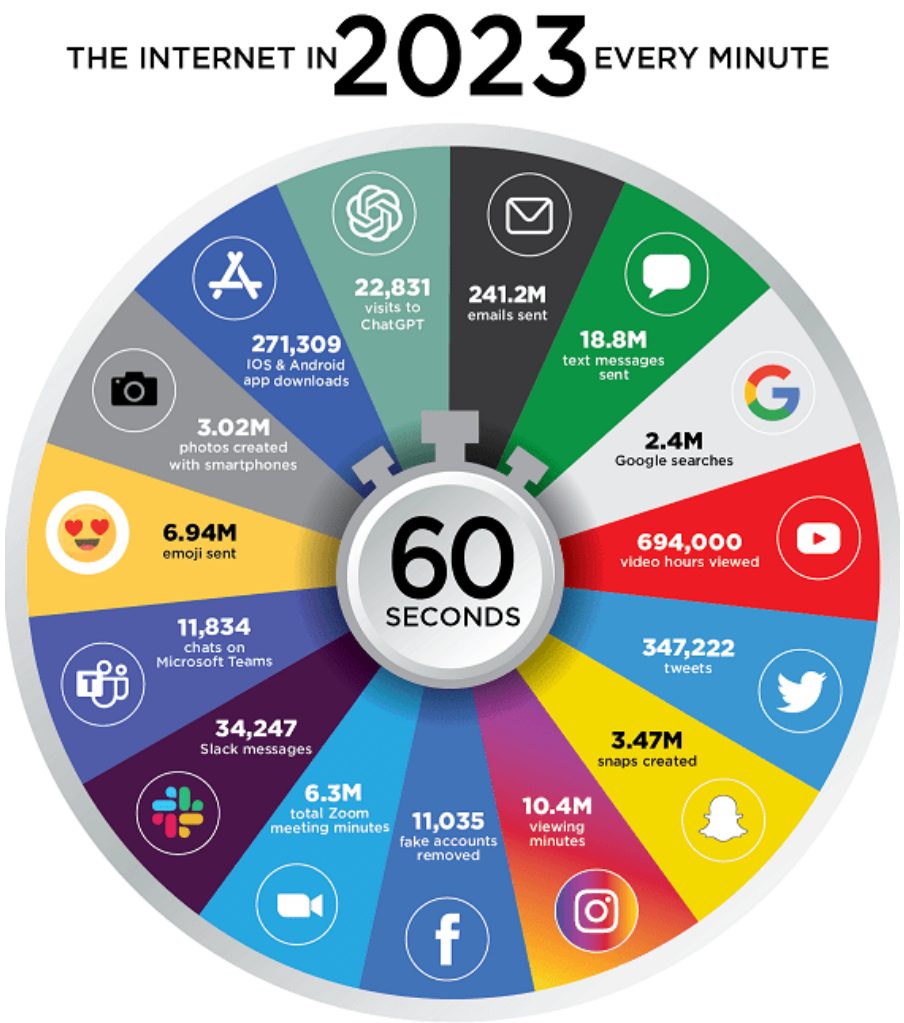
❖ Variaty



Introduction

❖ Velocity





Introduction

❖ Veracity

Data quality and validity are essential to effective Big Data projects.
If the data is not accurate or reliable than the expected benefits of the Big Data initiative will be lost.





Introduction

❖ Veracity

Imagen's training data was drawn from several pre-existing datasets of image and English alt-text pairs. A subset of this data was filtered to removed noise and undesirable content, such as pornographic imagery and toxic language. However, a recent audit of one of our data sources, LAION-400M [61], uncovered a wide range of inappropriate content including pornographic imagery, racist slurs, and harmful social stereotypes [4]. This finding informs our assessment that Imagen is not suitable for public use at this time and also demonstrates the value of rigorous dataset audits and comprehensive dataset documentation (e.g. [23, 45]) in informing consequent decisions about the model's appropriate and safe use. Imagen also relies on text encoders trained on uncurated web-scale data, and thus inherits the social biases and limitations of large language models [5, 3, 50].

Our primary aim with Imagen is to advance research on generative methods, using text-to-image synthesis as a test bed. While end-user applications of generative methods remain largely out of scope, we recognize the potential downstream applications of this research are varied and may impact society in complex ways. On the one hand, generative models have a great potential to complement, extend, and augment human creativity [30]. Text-to-image generation models, in particular, have the potential to extend image-editing capabilities and lead to the development of new tools for creative practitioners. On the other hand, generative methods can be leveraged for malicious purposes, including harassment and misinformation spread [20], and raise many concerns regarding social and cultural exclusion and bias [67, 62, 68]. These considerations inform our decision to not to release code or a public demo. In future work we will explore a framework for responsible externalization that balances the value of external auditing with the risks of unrestricted open-access.

Reference: https://arxiv.org/pdf/2205.11487

Introduction

❖ Value

Improve Customer Experiences

- Recommendation system
- Customer Segmentation

Cutting Costs

Fraud Detection and Prevention
- Anomaly Detection
- Risk Assessment

Pricing Optimization
- Dynamic Pricing
- Price Sensitivity Analysis

Solving Problems

Identifying Patterns and Trends:
- Predicting Future Outcomes
- Uncovering Hidden Relationships

Data-Driven Decision Making - Informed Choices

Insceasing Revenue

Personalized Marketing
- Targeted Campaigns
- Personalized Recommendations

Product Optimization
- Product Development
- Assortment Planning

Introduction

❖ Big Data Frameworks





ApacheFlink





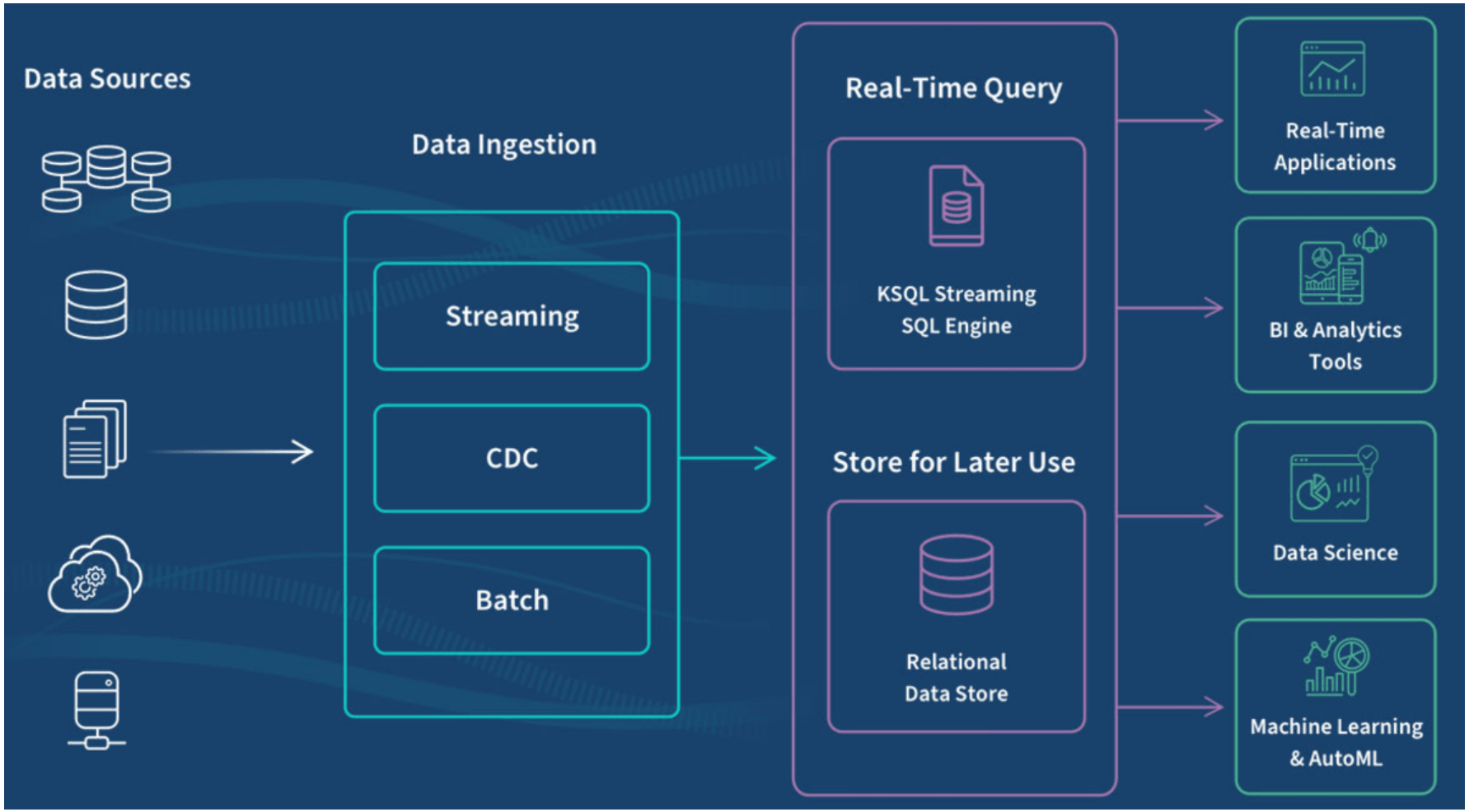


Data Ingestion

Data Ingestion

❖ Getting Started

➢ **Data Ingestion** is the process of collecting and loading data from various sources—such as databases, files, APIs, sensors, and streaming platforms—into centralized systems like **Data Lakes, Data Warehouses, or Data Lakehouses**.

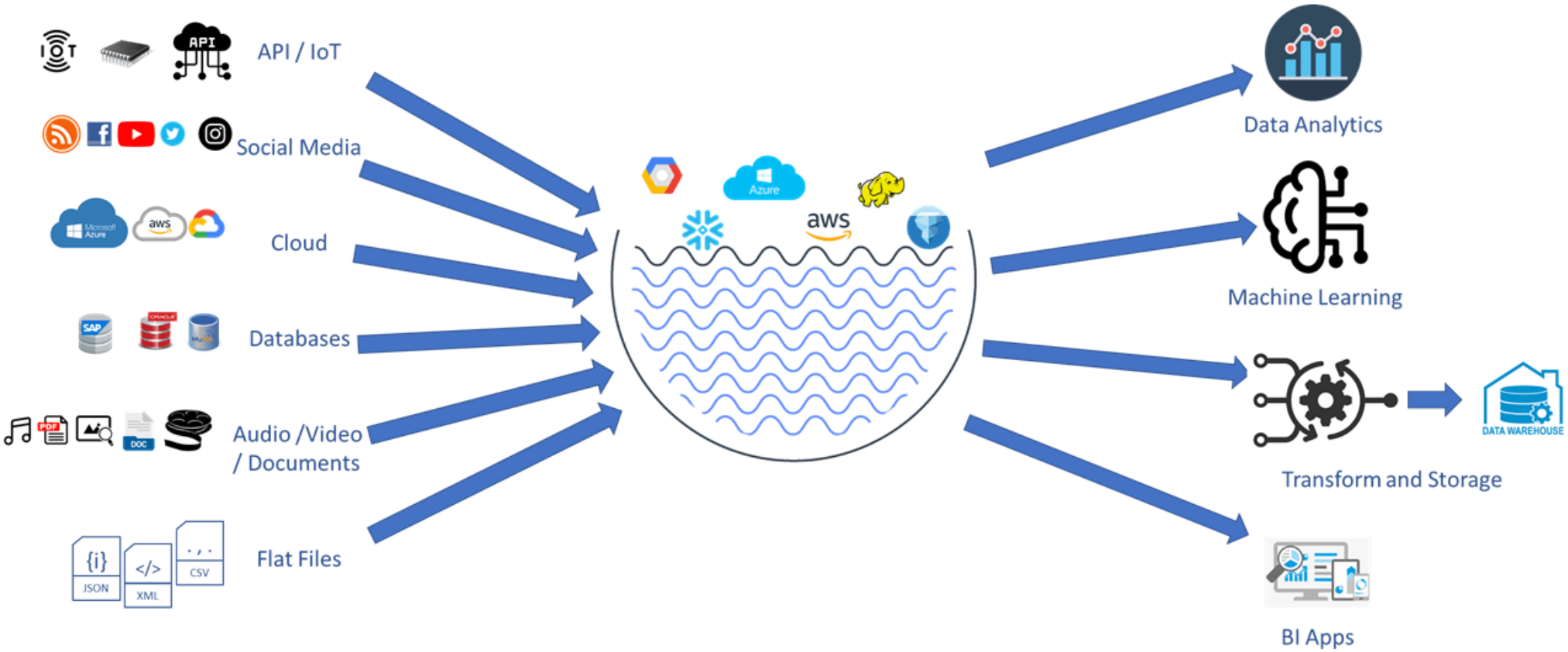


It is a foundational stage of the data pipeline that directly impacts downstream processing and analytics.

Data Ingestion

❖ Data Lake

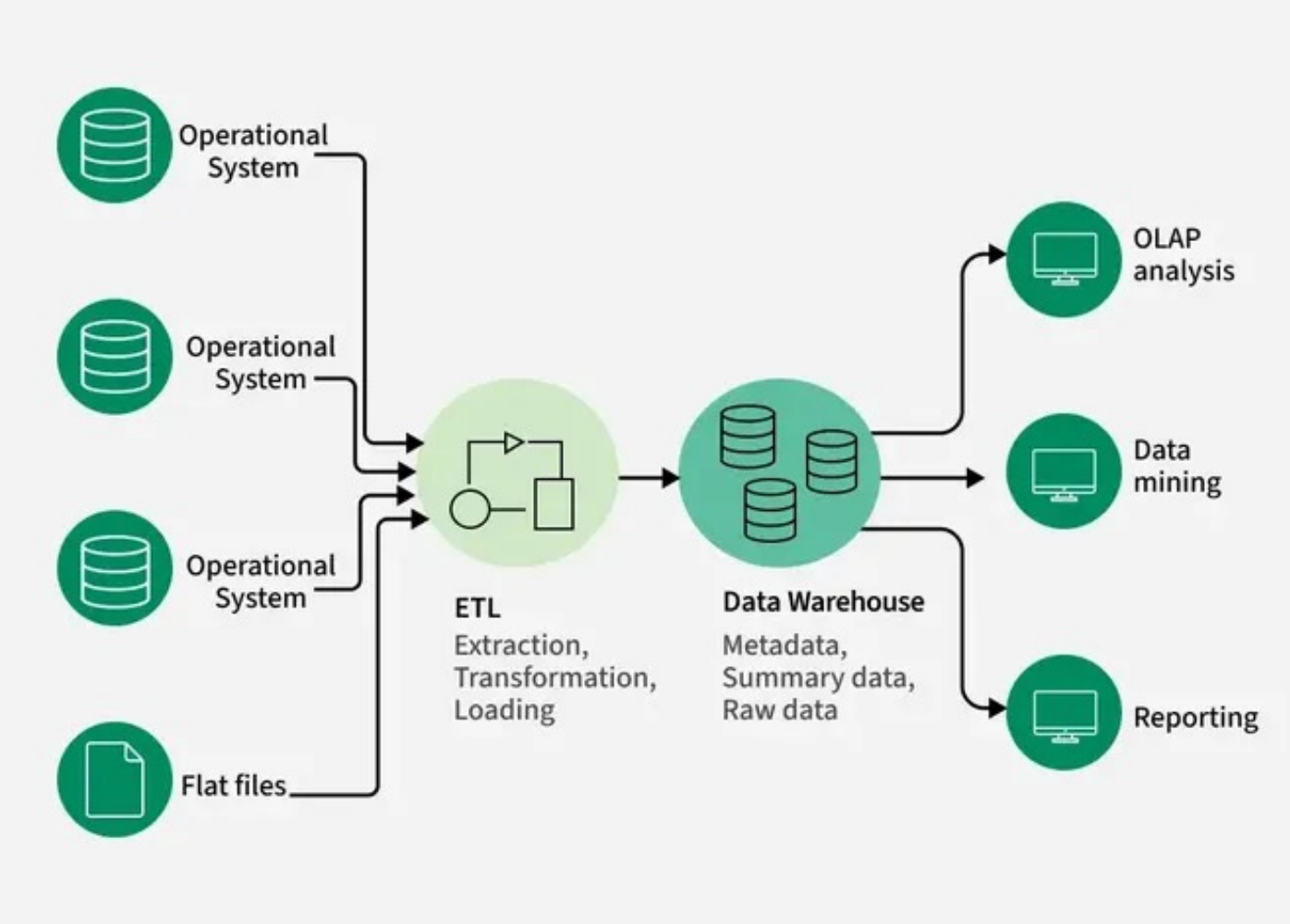
➢ Data Lake is a centralized repository for storing large volumes of raw data in its native format for downstream processing, analytics, and machine learning.



Data Ingestion

❖ Data Warehouse

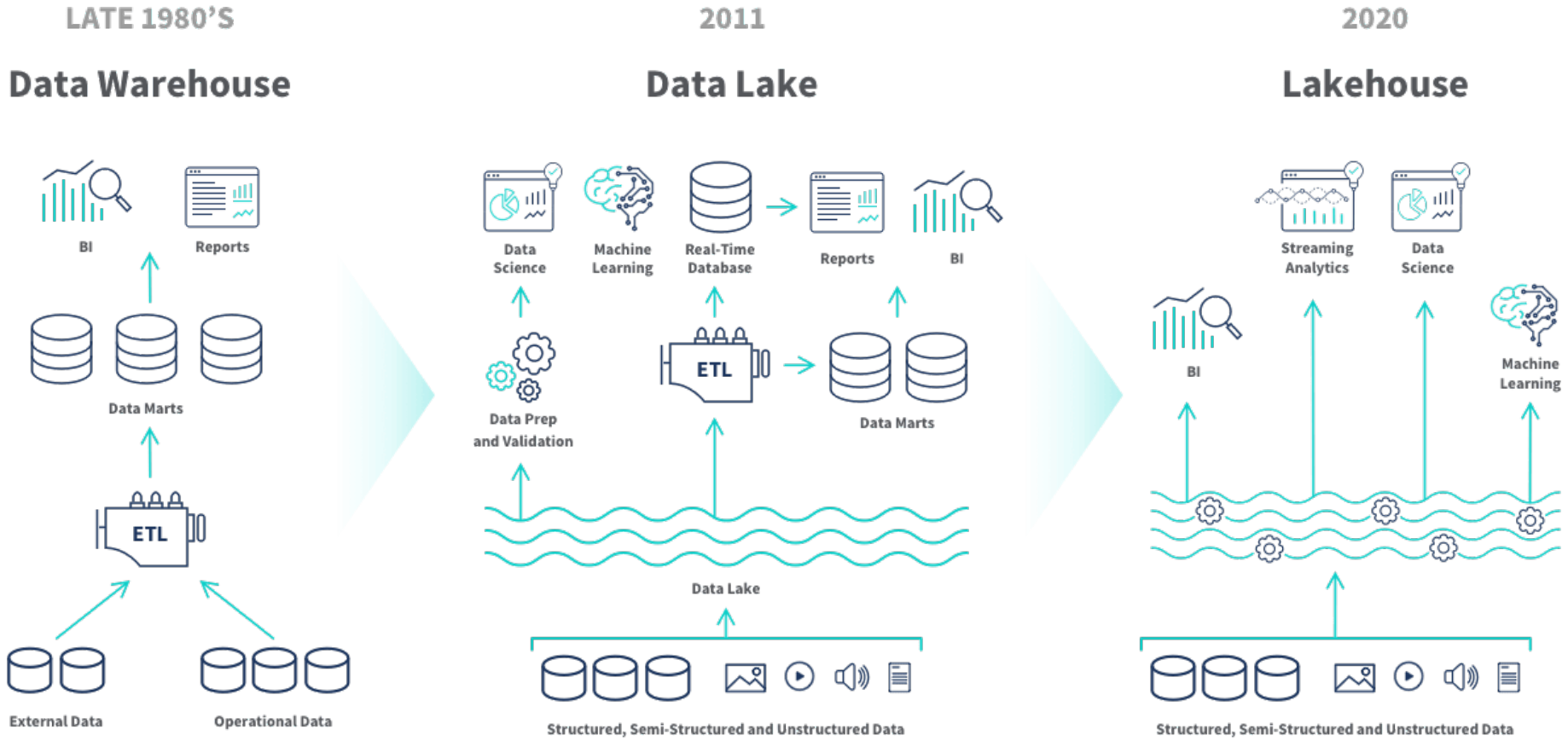
Data Warehouse is a centralized system for storing and analyzing structured, processed data from multiple sources. It is optimized for **SQL queries, reporting, business intelligence, and analytical workloads**.



Data Ingestion

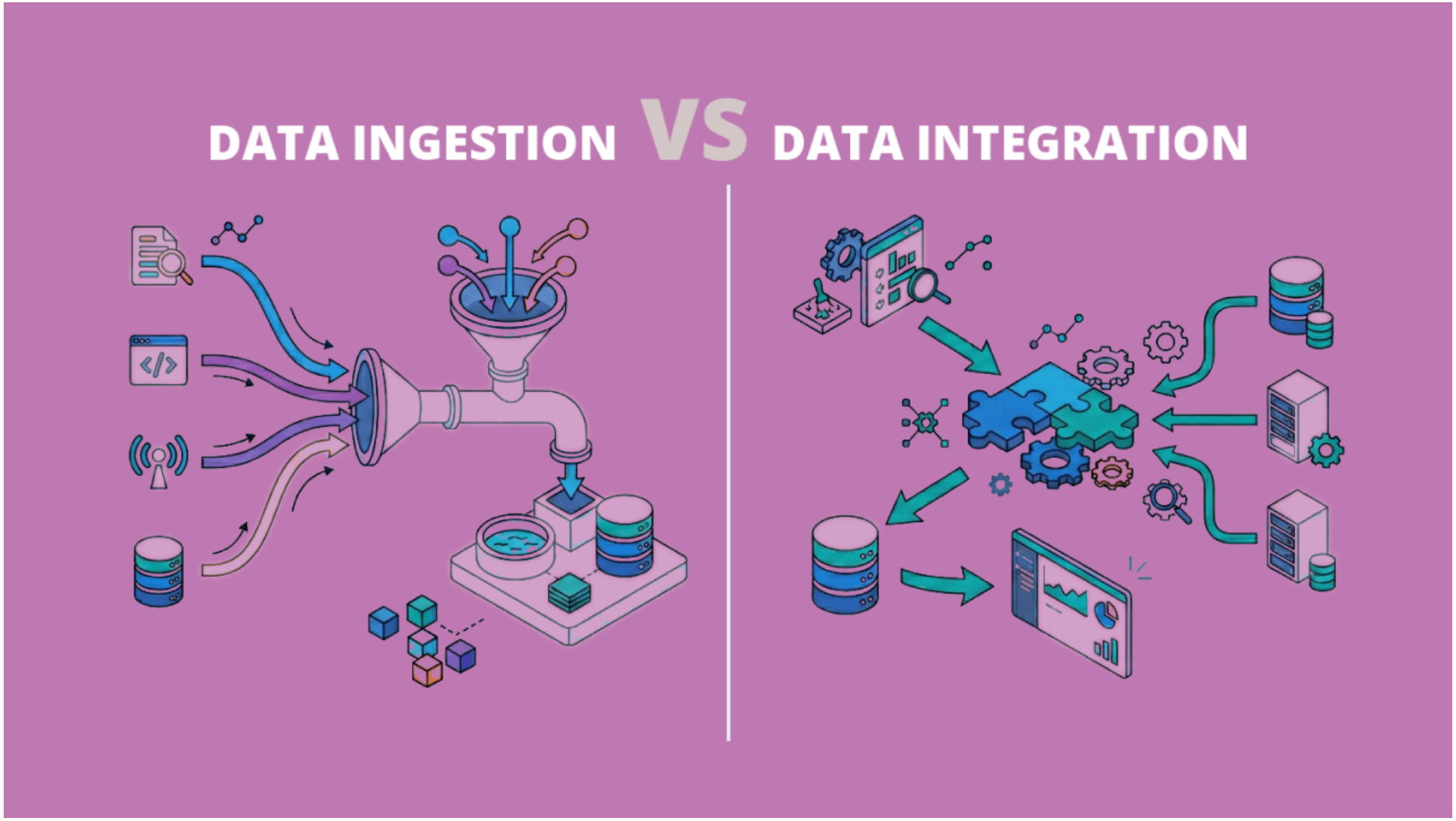
❖ Data Lakehouse

➢ **Data Lakehouse** is a data architecture that combines the flexibility and scalability of a **Data Lake** with the structured data management and analytical capabilities of a **Data Warehouse**. It supports both **data engineering and analytics/ML workloads** on a unified platform.



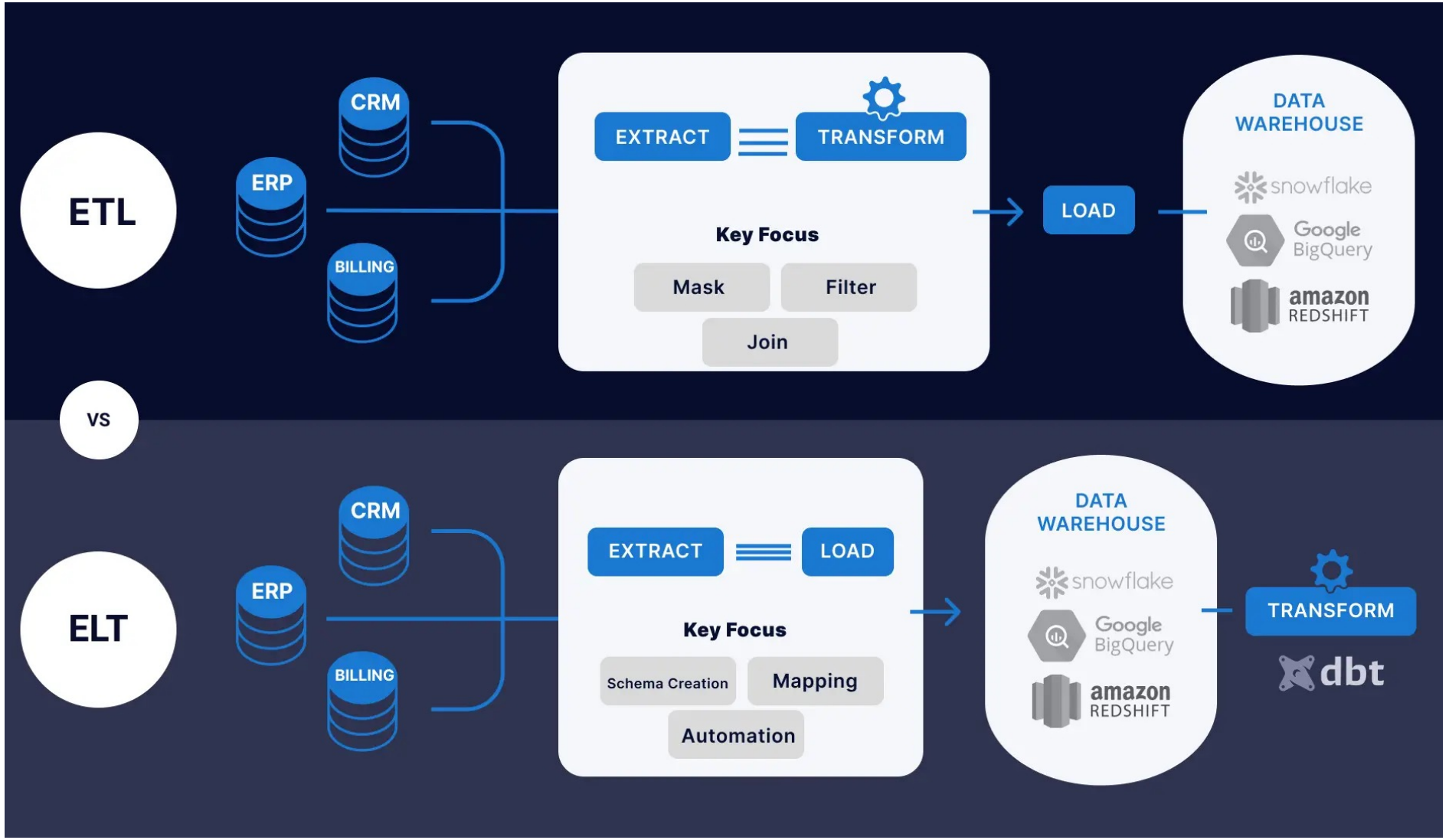
Data Ingestion

❖ Data Ingestion vs Data Integration



Data Ingestion

❖ ETL vs ELT



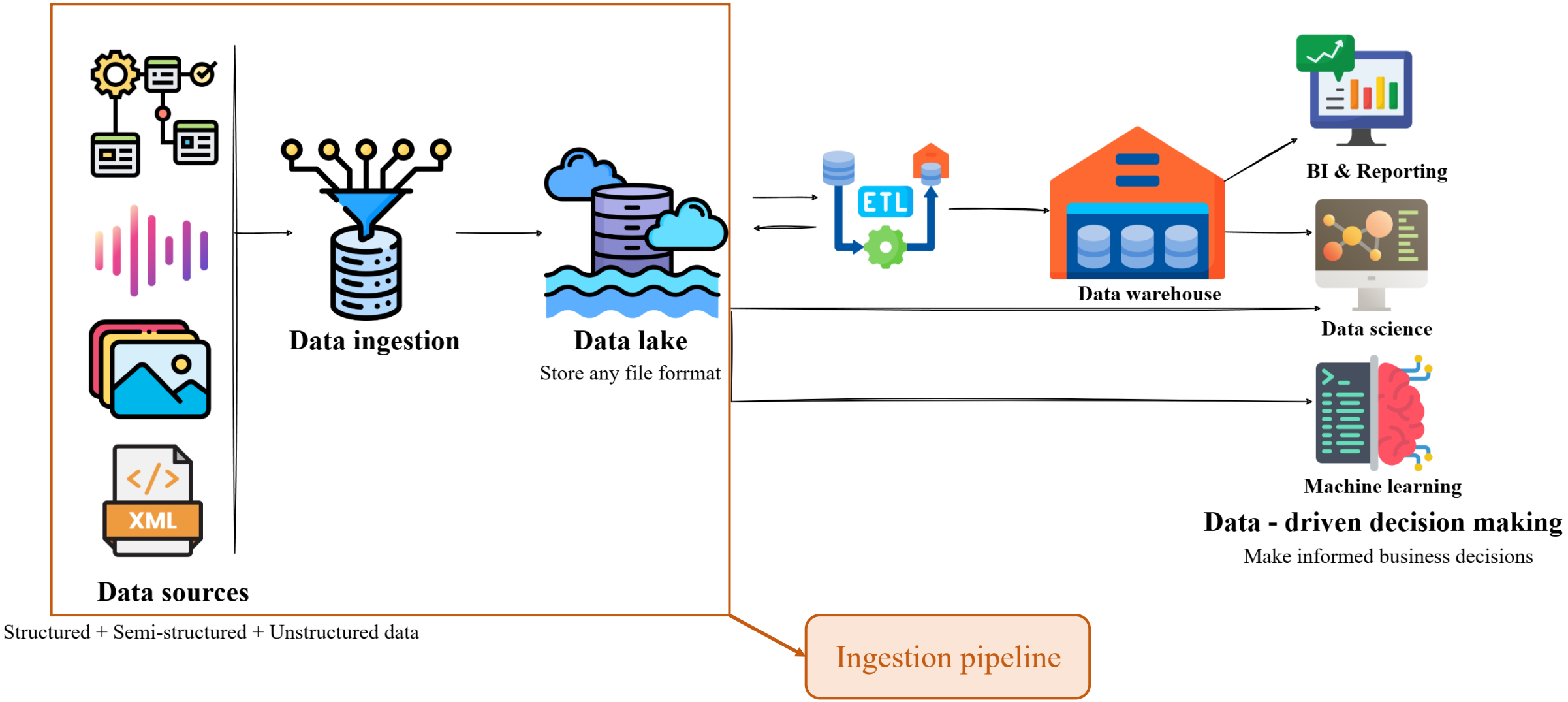
Data Ingestion

❖ ETL vs ELT

	ETL	ELT
Flow	Source → **Transform** → Destination	Source → **Load** → **Transform**
Transform	Before entering the target system	After entering the target system
Destination	Data Warehouse (DWH)	Data Lake/Lakehouse/DWH
Used for	processed before storing	Keep raw
Compute	Need ETL engine	Leverage the target system's compute resources.
Raw data	No keep full	Keep
Flexibility	Low	High

Data Ingestion

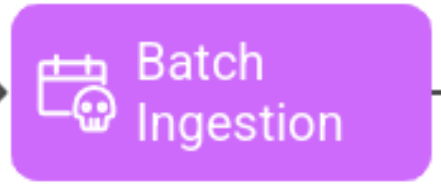
❖ Data pipeline

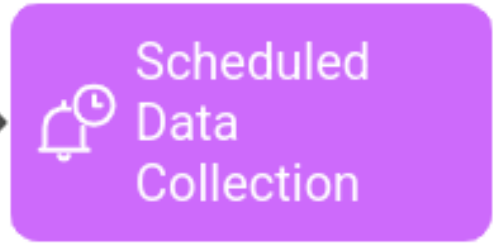


Data Ingestion

❖ Methods

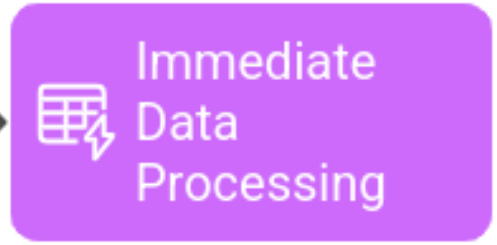




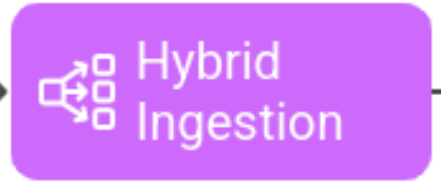


Data is collected and processed in periodic batches; suitable for revenue reporting and daily data





Data is collected as soon as it is generated; suitable for fraud detection, stock market analysis, and IoT

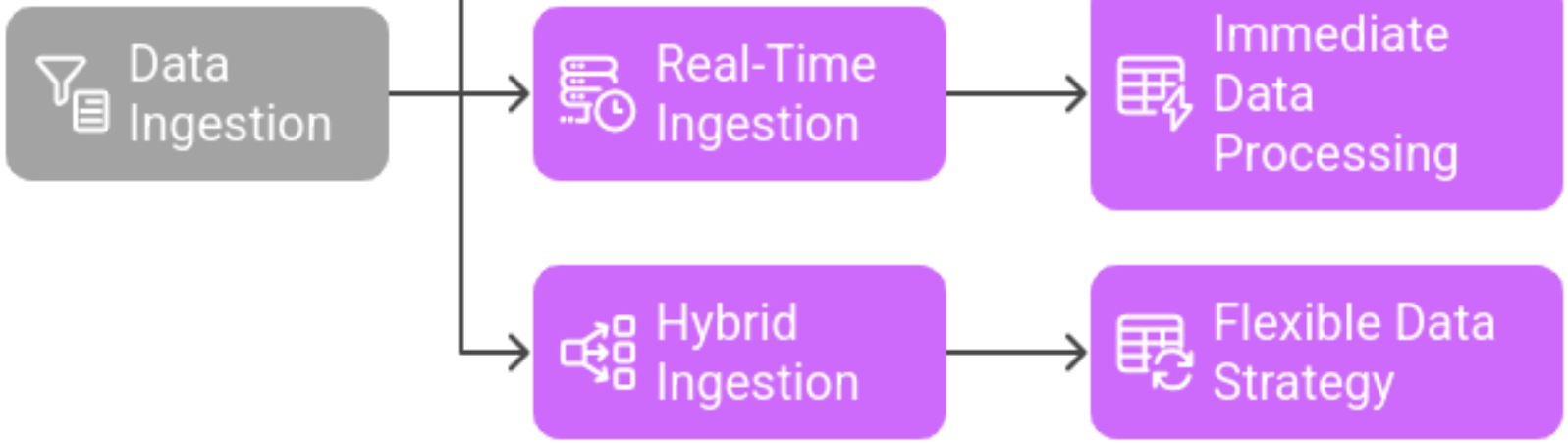




Combines Batch and Real-Time methods based on requirements; for example, real-time fraud detection alongside end-of-day batch reporting.

Data Ingestion

❖ Methods



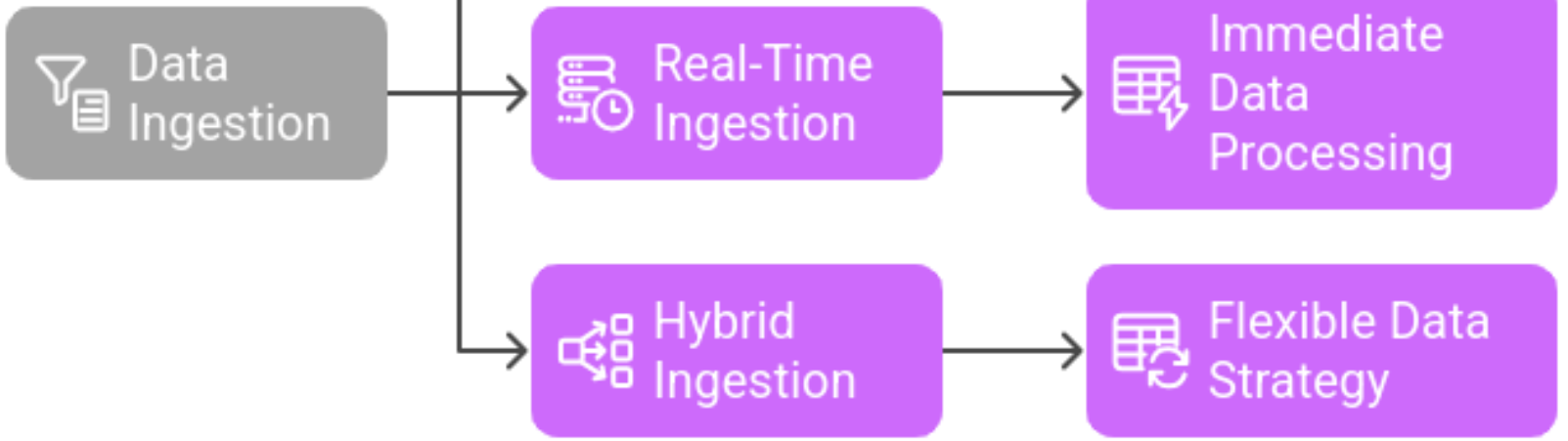


Data is collected as soon as it is generated; suitable for fraud detection, stock market analysis, and IoT

Combines Batch and Real-Time methods based on requirements; for example, real-time fraud detection alongside end-of-day batch reporting.

Data Ingestion

❖ Methods





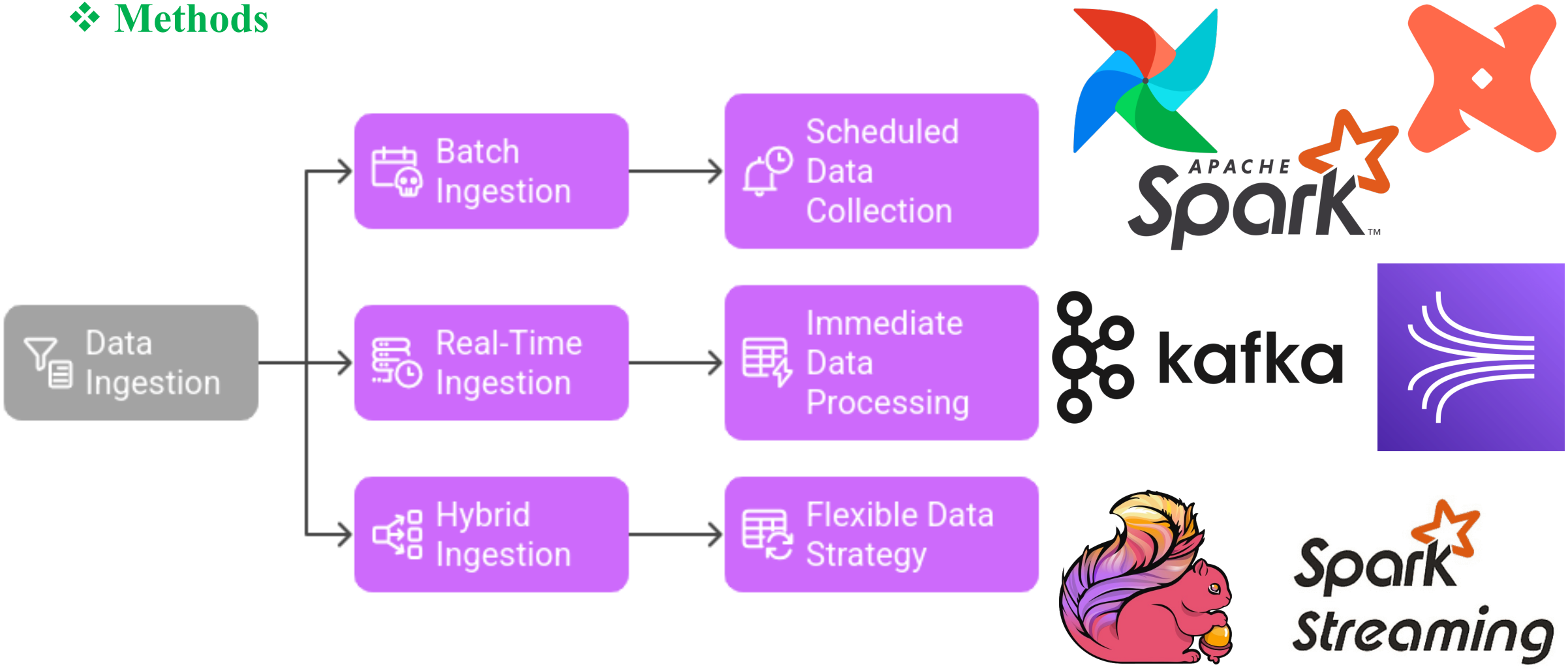




Combines Batch and Real-Time methods based on requirements; for example, real-time fraud detection alongside end-of-day batch reporting.

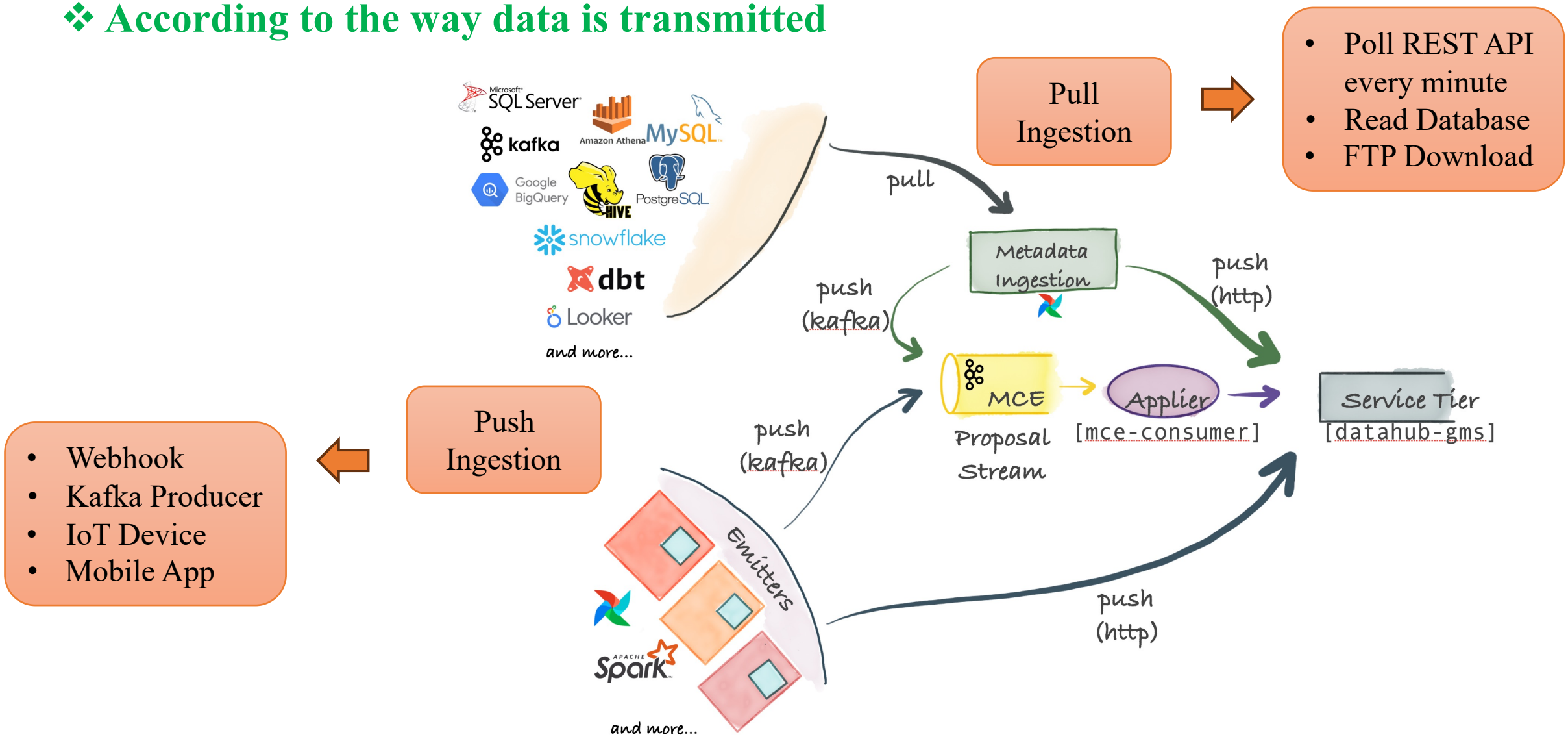
Data Ingestion

❖ Methods



Data Ingestion

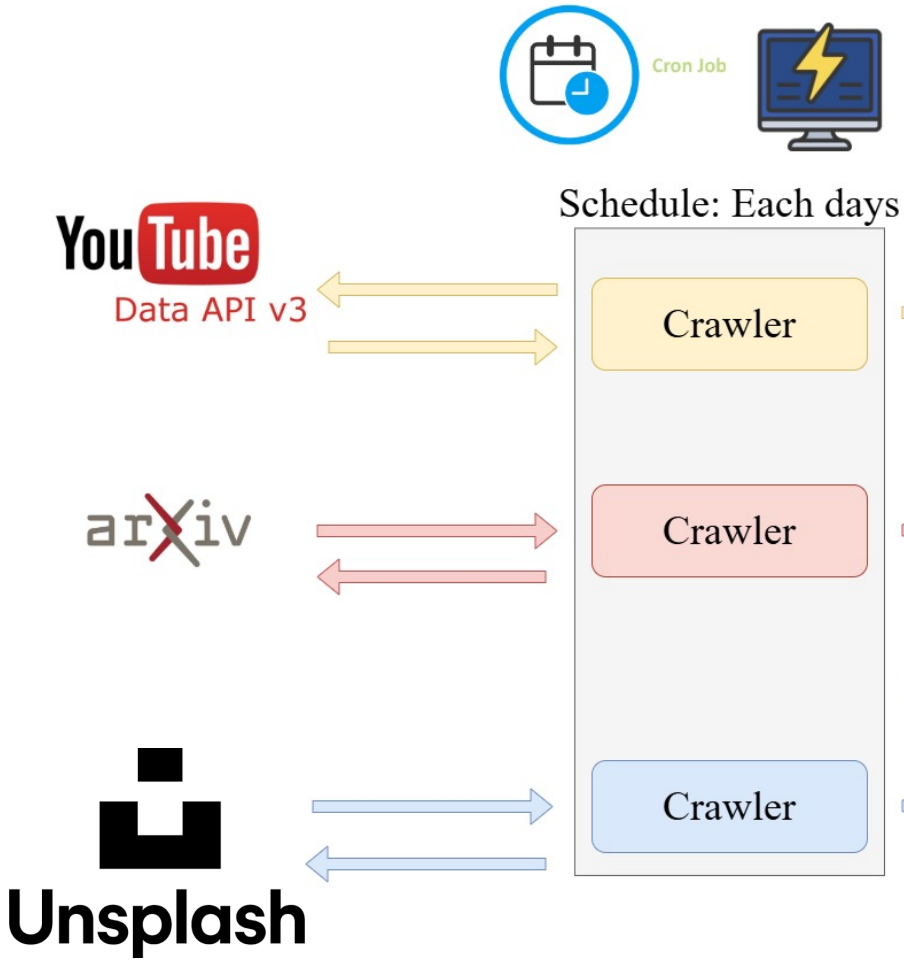
❖ According to the way data is transmitted



https://docs.datahub.com/docs/architecture/metadata-ingestion

Data Ingestion

❖ Ingestion Pipeline




```
## Outputs (what you get after a successful run)

After the crawler finishes, the main outputs live under:

- `crawl_yt_video_audio/downloads/YYYY-MM-DD/`

Expected folders/files:

- **Video files**: `video/`
  - e.g. `video/<id>.mp4`
- **Audio files**: `audio/`
  - e.g. `audio/<id>.m4a`
- **Subtitles**: `subs/`
  - e.g. `subs/<id>.<lang>.vtt` (or `.srt` depending on availability)
- **Per-video metadata**: `info/`
  - e.g. `info/<id>.info.json`
- **Deduplication state** (global, across days):
  - `downloads/downloaded_ids.json`
```

```
## Output

A new CSV file is created daily in `data/` folder with format: `papers_YYYY-MM-DD.csv`

CSV columns:
- `arxiv_id`: Paper ID on arXiv
- `title`: Title
- `authors`: List of authors
- `abstract`: Abstract
- `categories`: Paper categories
- `published`: Publication date
- `updated`: Last update date
- `pdf_url`: PDF download link
- `primary_category`: Primary queried category

## Logs

Logs are saved in `logs/` folder with format: `crawl_YYYY-MM-DD.log`
```

```
All outputs are written under `crawl_unsplash_image/downloads/`:

- **Images** (partitioned by crawl date):
  - `downloads/YYYY-MM-DD/<photo_id>.<ext>` (usually `.jpg`)
- **Dedup state**:
  - `downloads/downloaded_ids.json`
  - Contains a minimal record per downloaded `photo_id` (title, file path, url, downloaded_at)
- **Metadata index**:
  - `downloads/metadata.json`
  - Contains a richer JSON record per `photo_id` (dimensions, user info, urls, links, etc.)
```

https://github.com/ThuanNaN/aio-data-ingestion

```python
import requests
import pandas as pd
from datetime import datetime

# 1. Thu thập dữ liệu (Data Ingestion) từ API giả lập
def fetch_sales_data(api_url):
    print(f"Đang kéo dữ liệu từ {api_url}...")
    response = requests.get(api_url)

    if response.status_code == 200:
        return response.json() # Trả về list các dictionary (JSON)
    else:
        raise Exception("Không thể lấy dữ liệu!")

# Giả sử đây là API của công ty
API_URL = "https://fakestoreapi.com/carts"
raw_data = fetch_sales_data(API_URL)

# 2. Đưa vào Pandas DataFrame để xử lý nhẹ
df = pd.DataFrame(raw_data)

# 3. Tính toán đơn giản: Đếm số lượng sản phẩm
print(f"Tổng số giao dịch lấy về: {len(df)}")

# 4. Lưu trữ (Load) xuống Local Storage để phân tích sau
today = datetime.now().strftime("%Y-%m-%d")
output_path = f"sales_data_{today}.csv"

df.to_csv(output_path, index=False)
print(f"Đã lưu dữ liệu thành công tại: {output_path}")
```

Problems



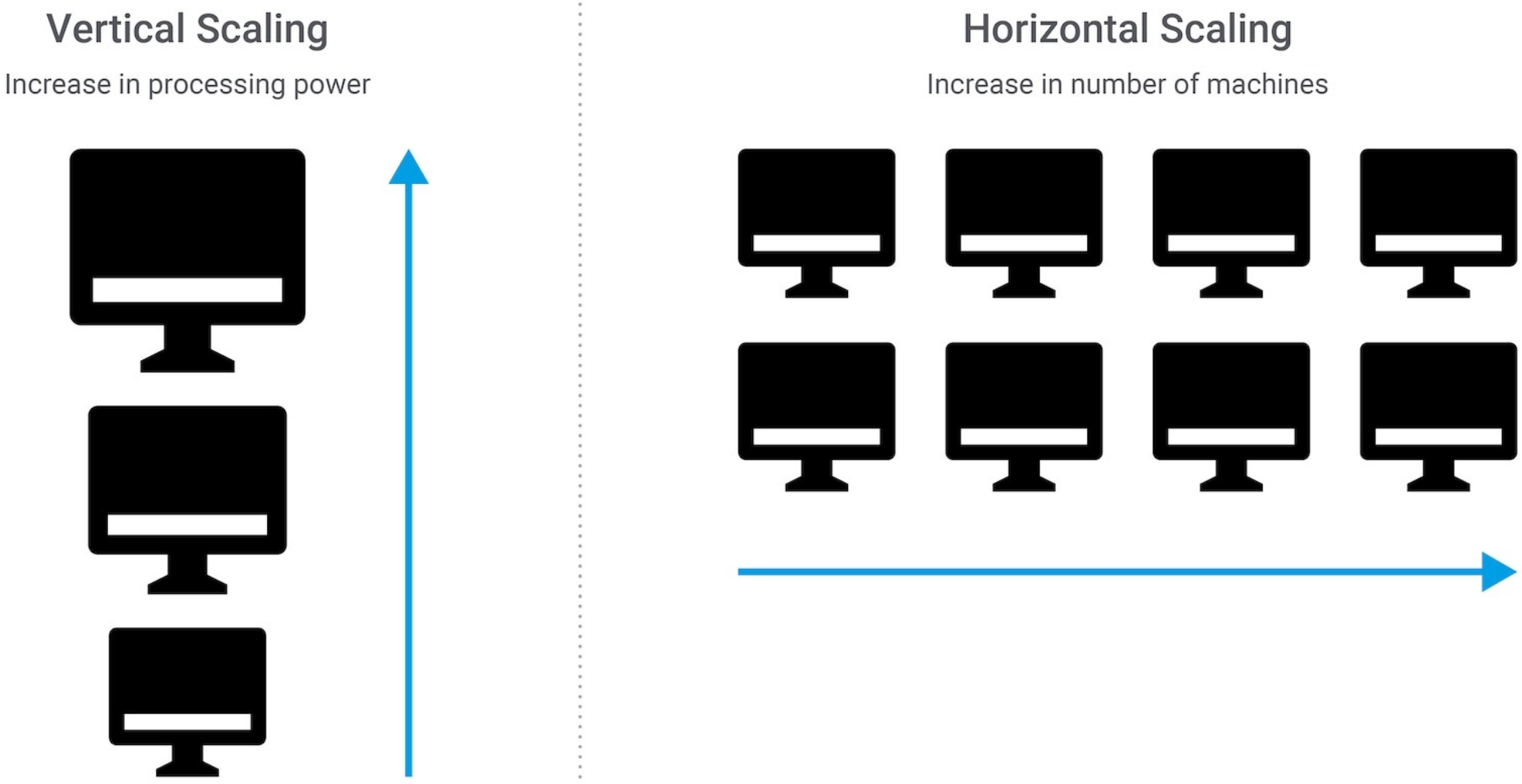
```python
import pandas as pd

# Cố gắng đọc file 50GB
df = pd.read_csv("historical_sales_50GB.csv")
```



Problems

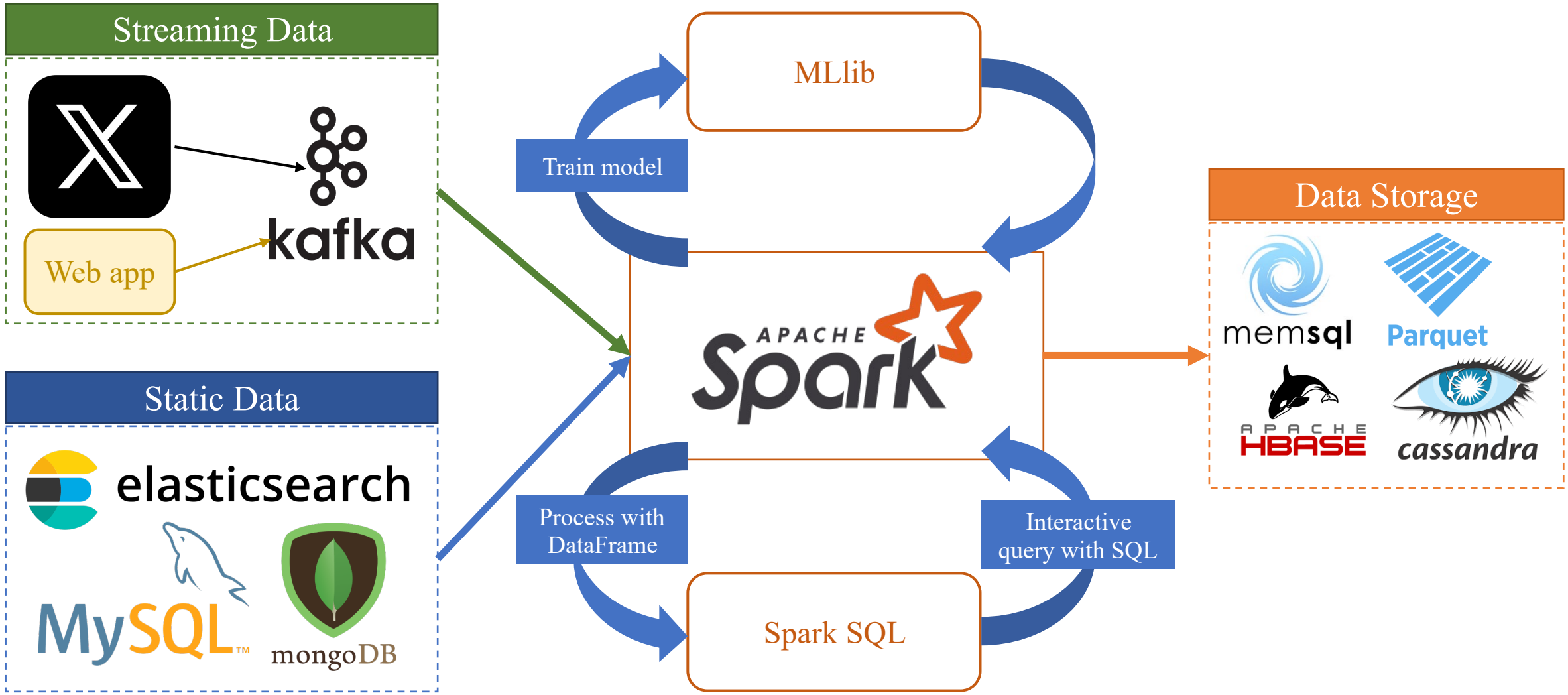
❖ Scaling



Apache Spark

Apache Spark

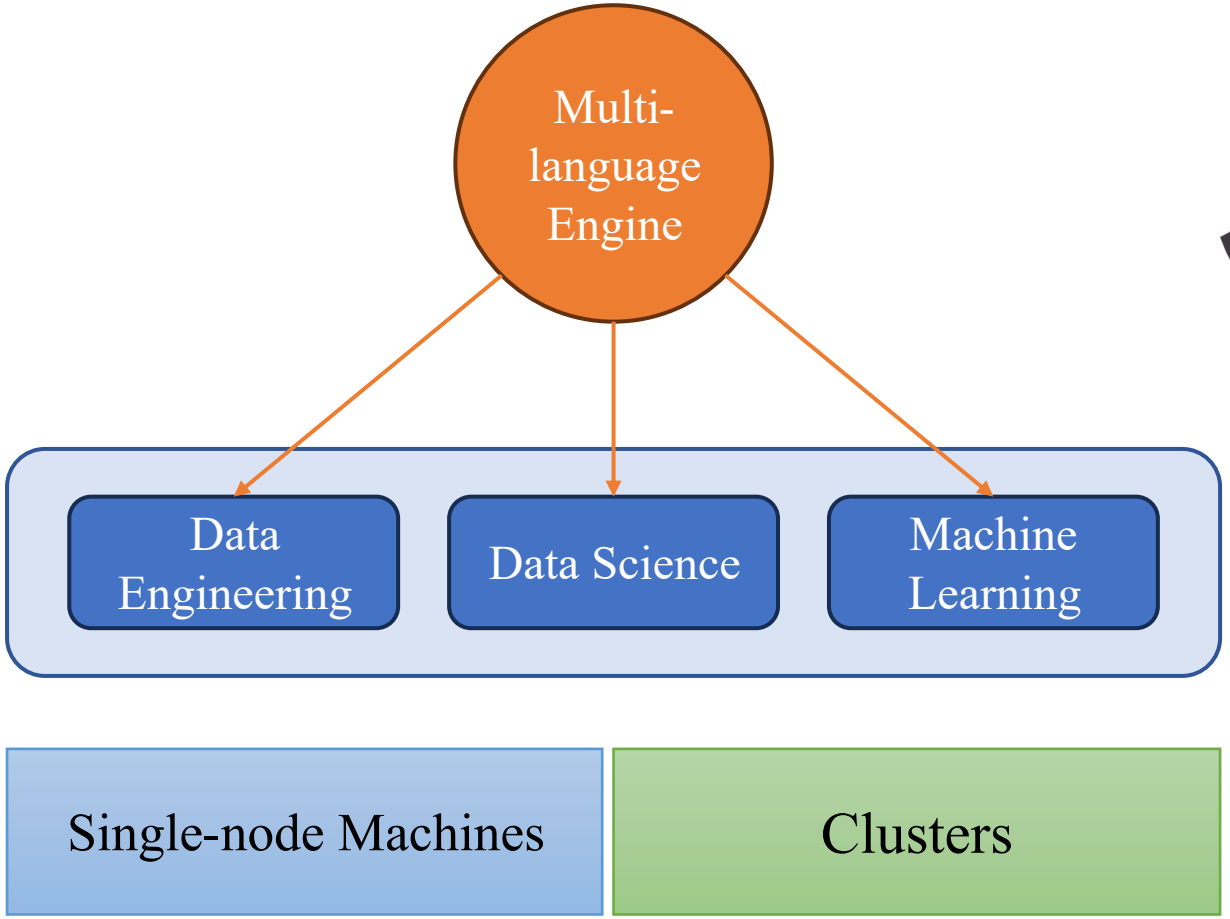
❖ Data pipeline with PySpark



Apache Spark

❖ Getting Started

- Spark is a **distributed processing engine,** unified analytics engine—capable of **parallel data processing** across **a cluster**.

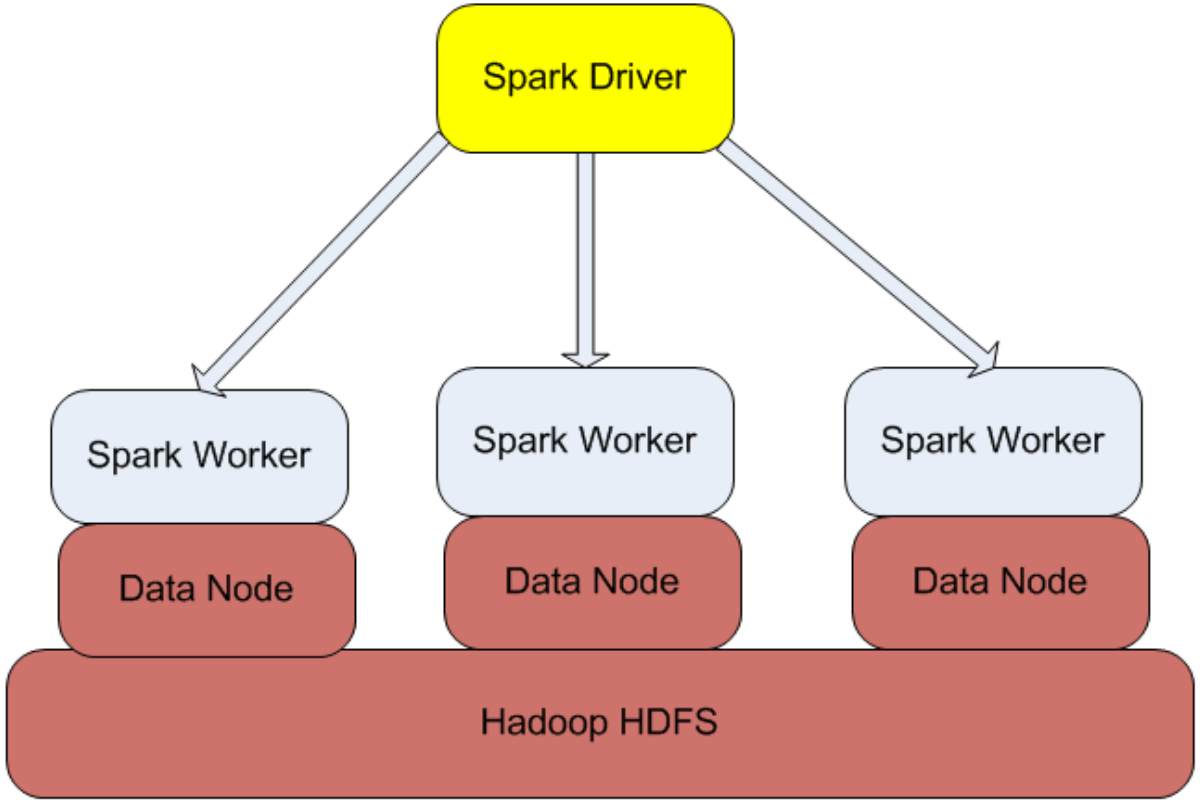


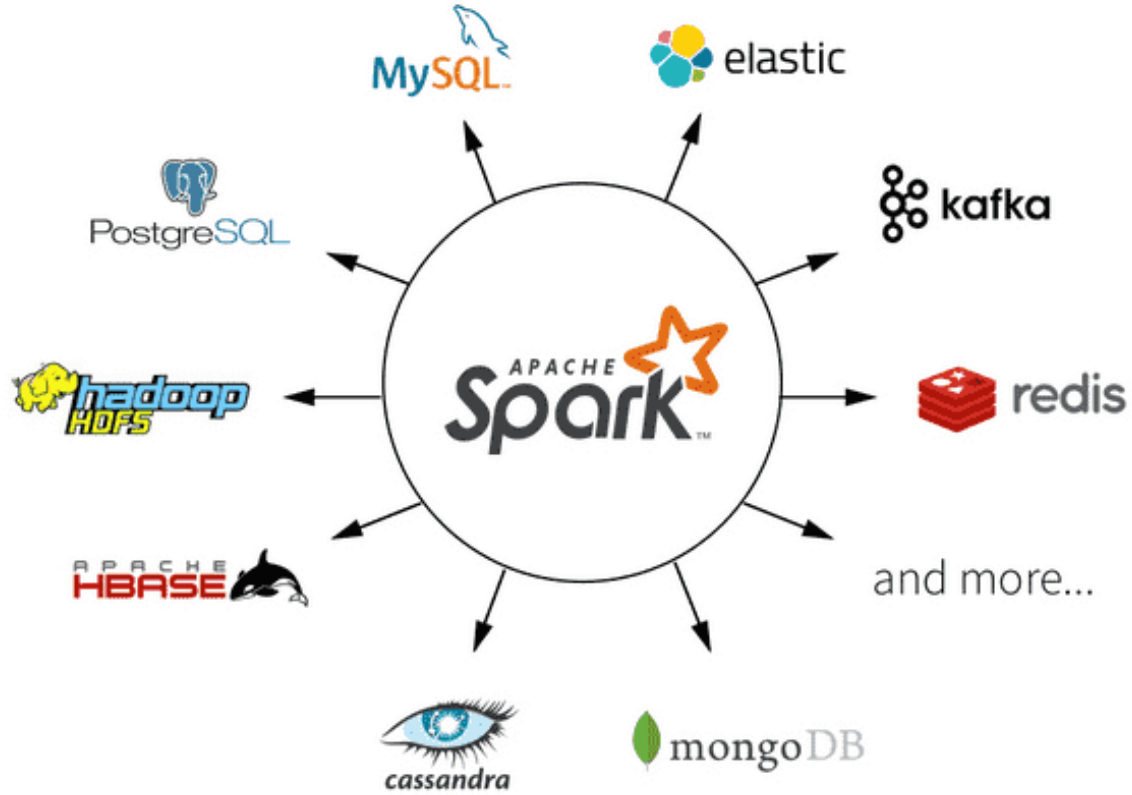
https://spark.apache.org/docs/latest/index.html

Apache Spark

❖ Getting Started

- Spark is a data processing tool that is faster and more flexible than Hadoop.
- It uses in-memory data processing, which significantly speeds up computations.





Apache Spark

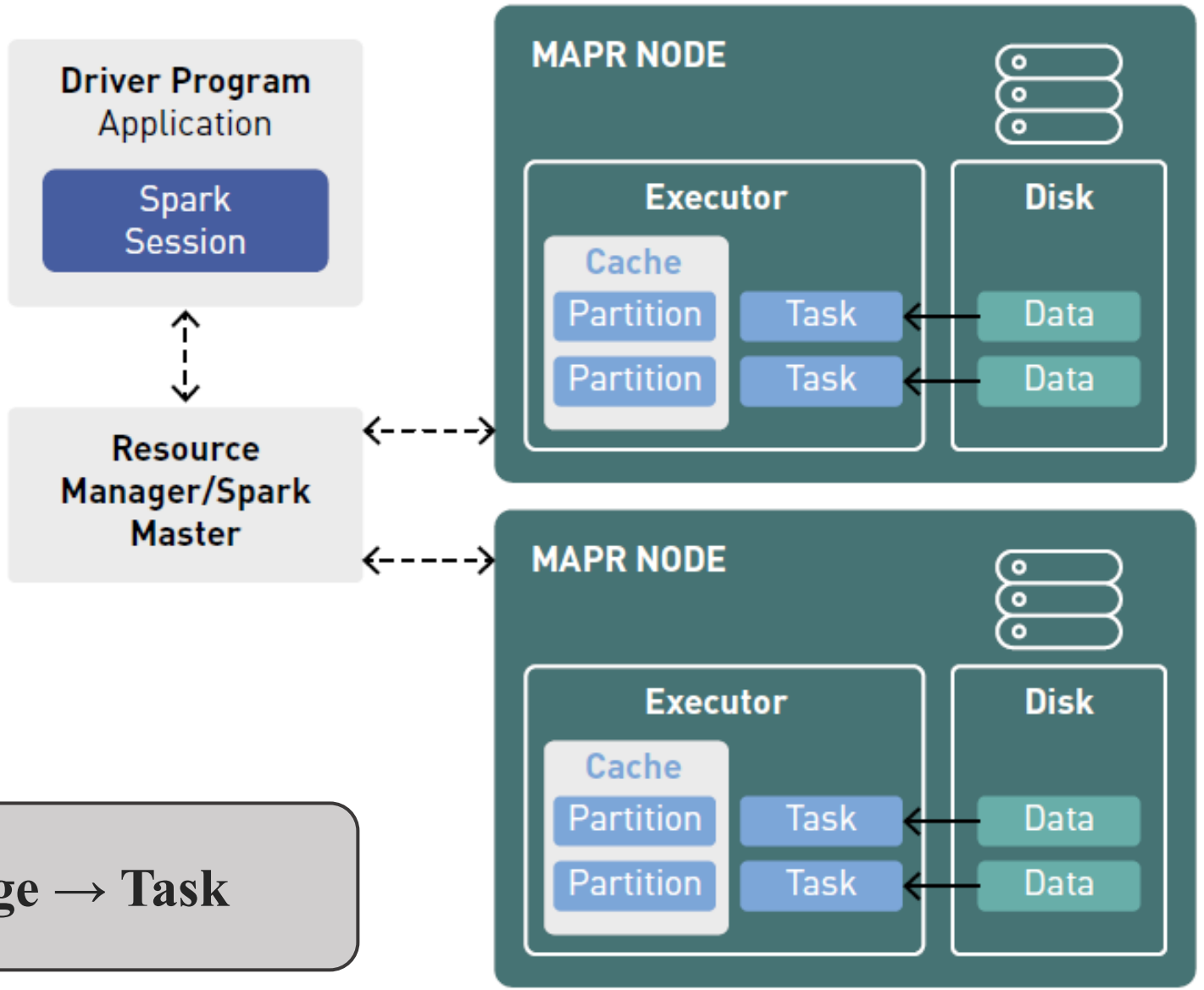
❖ Architecture

The Driver is the "brain" of the Spark application.

The Cluster Manager is responsible for allocating resources to Spark.

Executors are processes running on worker nodes to perform computations.

Partition là một phần nhỏ của distributed dataset.



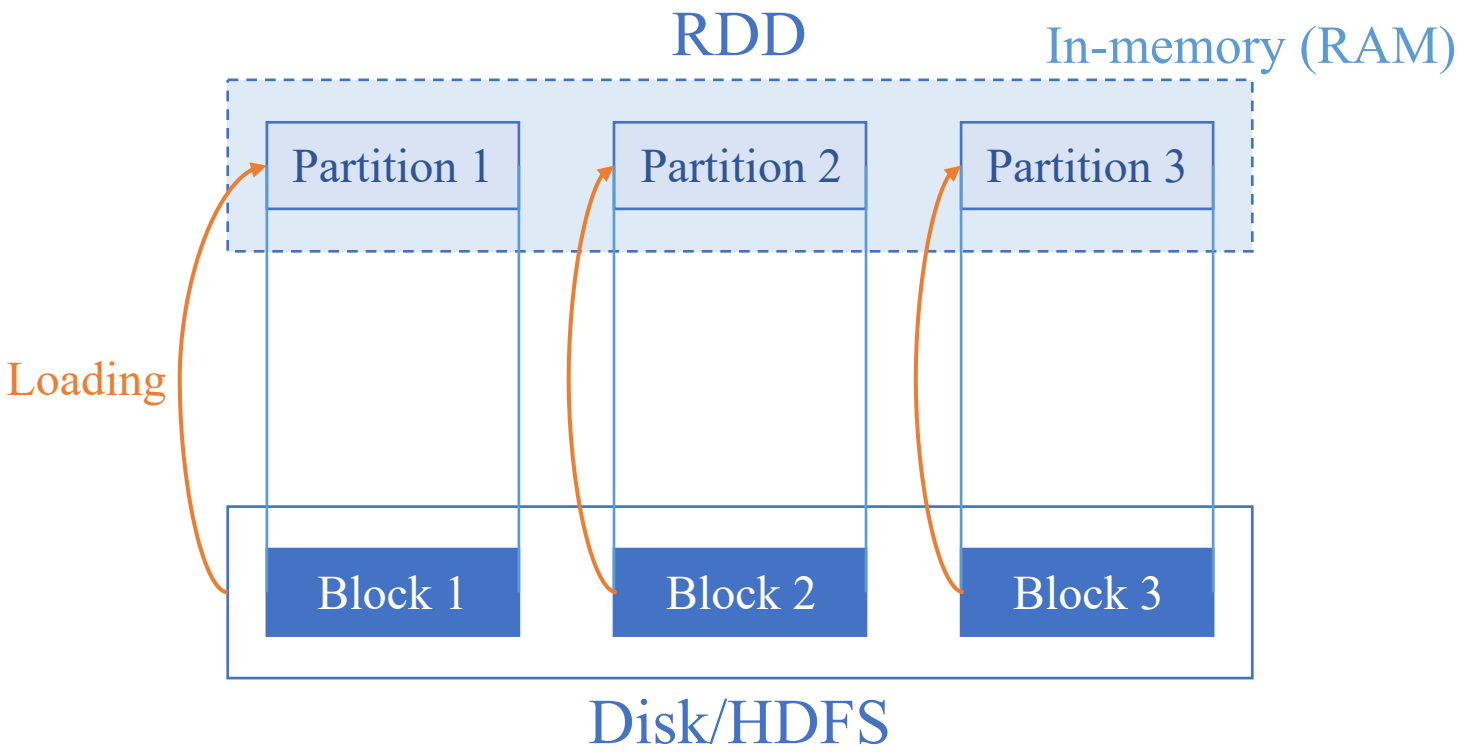
Job → Stage → Task

Apache Spark

❖ RDD (Resilient Distributed Dataset)

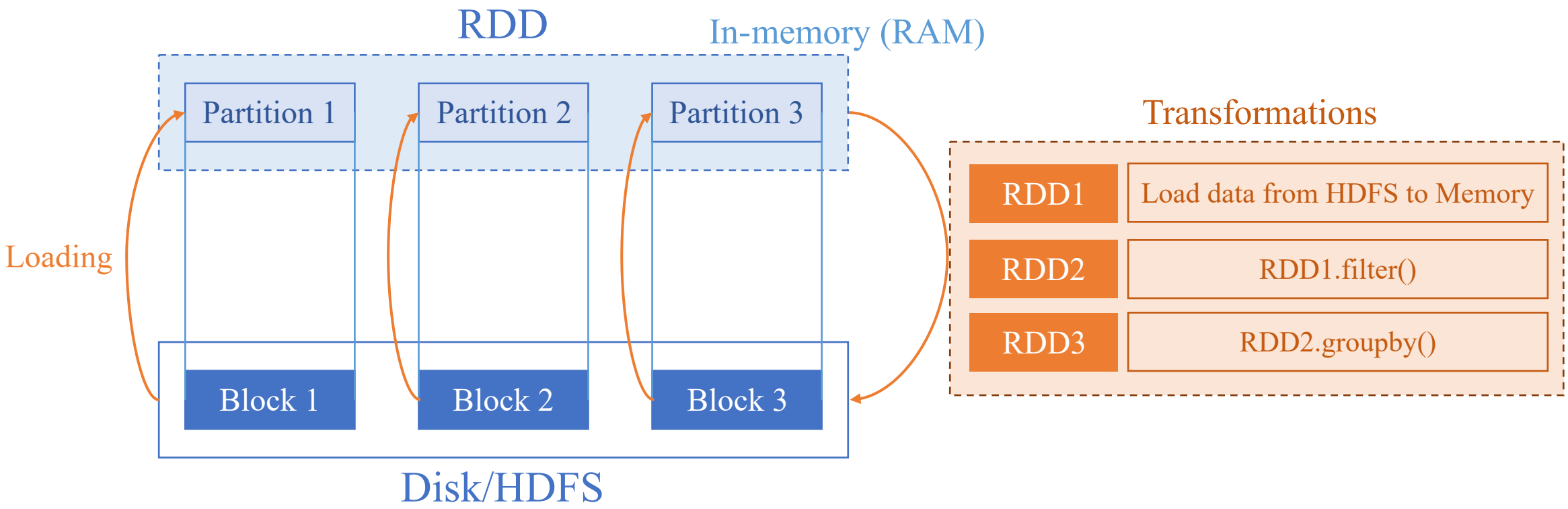
Resilient Distributed Datasets:

- Resilient: Ability to withstand failures
- Distributed: Spanning across multiple machines
- Datasets: Collection of partitioned data eg. Arrays, Tables, Tuples, etc.



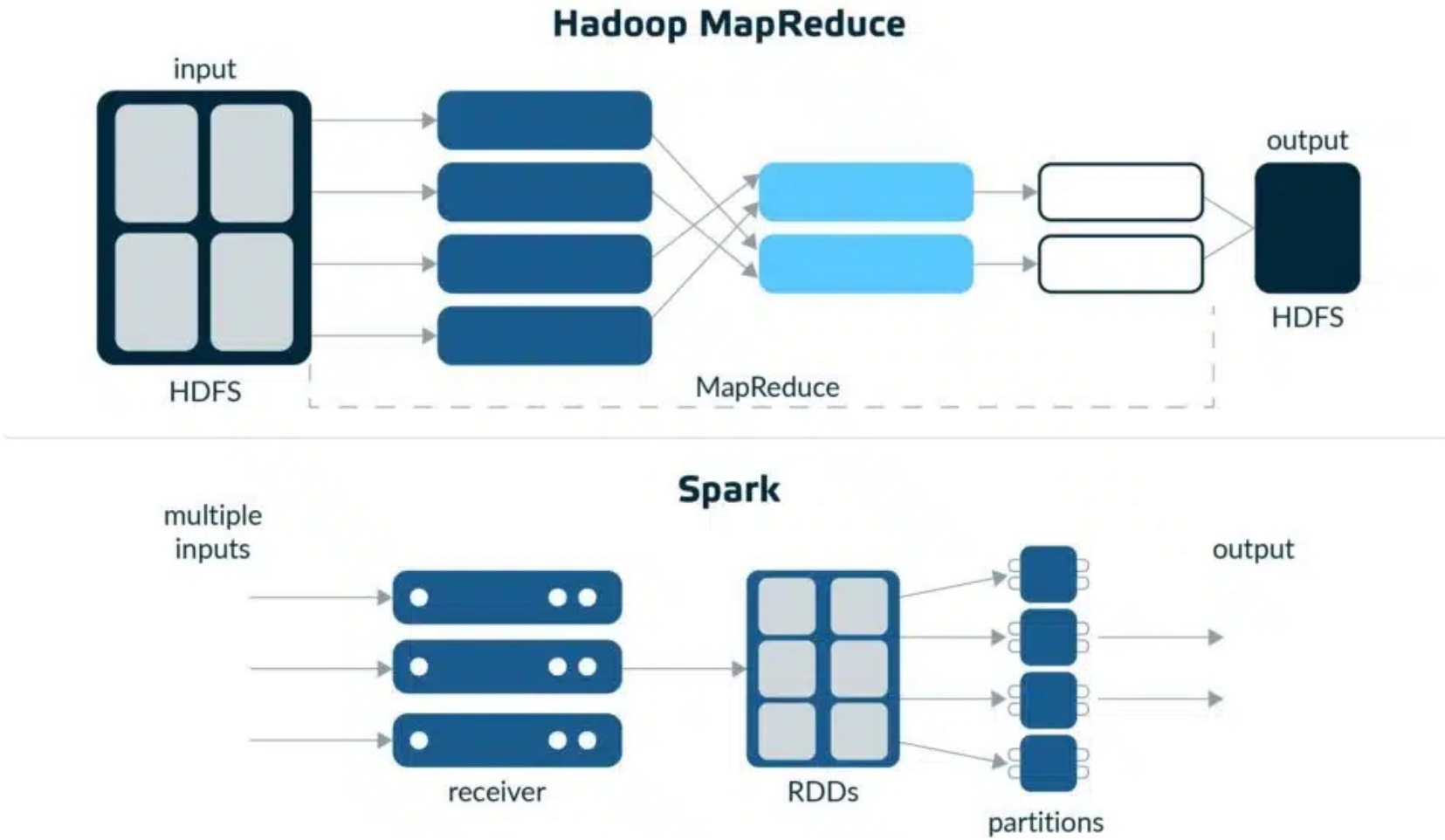
Apache Spark

❖ RDD (Resilient Distributed Dataset)



Apache Spark

❖ RDD (Spark) vs MapReduce (Hadoop)



Apache Spark

❖ Spark vs Hadoop: Comparison

Parameter	Hadoop	Spark
Intent	Data Processing	Data Analytics
Work Process	Analyses batches of data present in huge volumes	Analyses and processes real-time data
Processing Style	Batch	Real-time
Latency	High	Low
Cost	Less costly due to the MapReduce Model	Is more expensive (in-memory solution)
Ease of Use	Complex to use	Easier to use
Written	Java	Scala

Apache Spark

❖ Ecosystem

Data science and Machine learning















Apache Spark

❖ Ecosystem

SQL analytics and BI













Apache Spark

❖ Ecosystem

Storage and Infrastructure















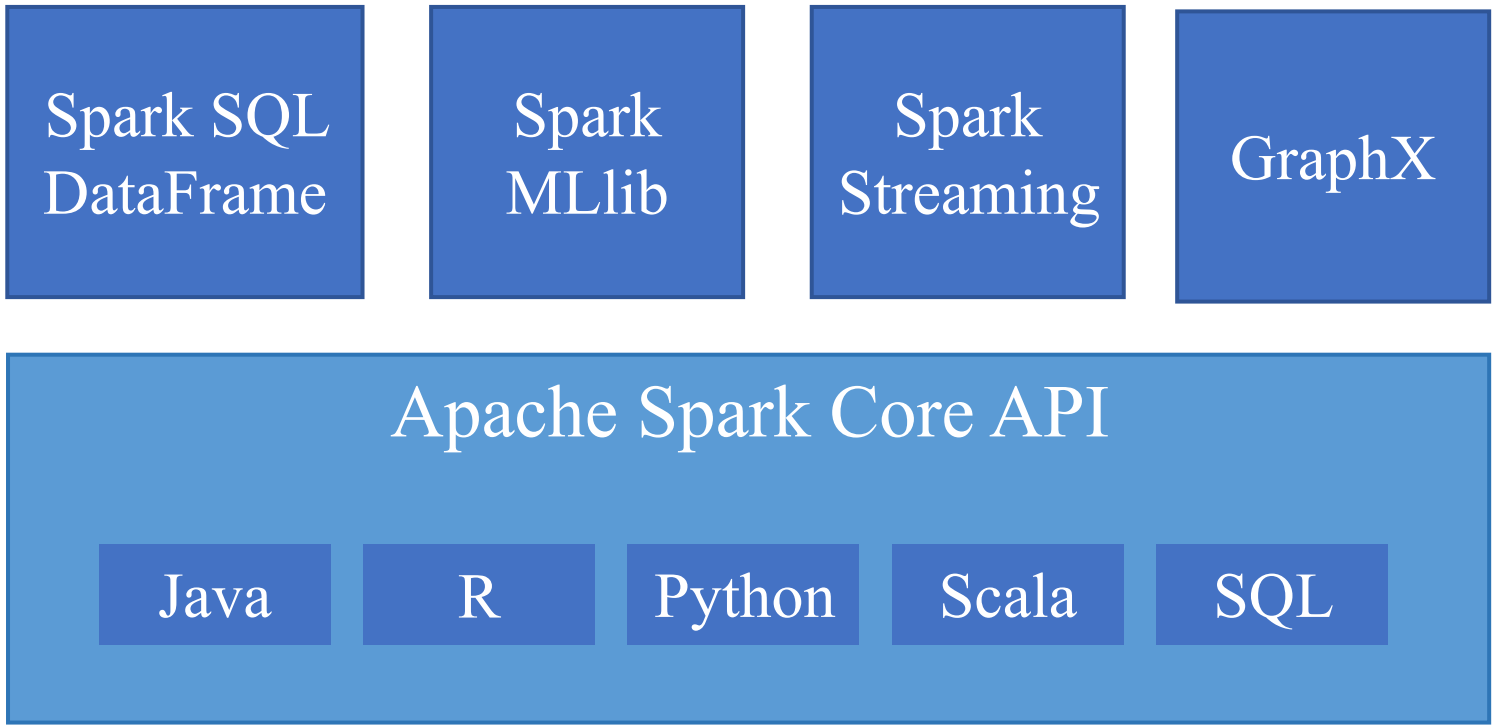






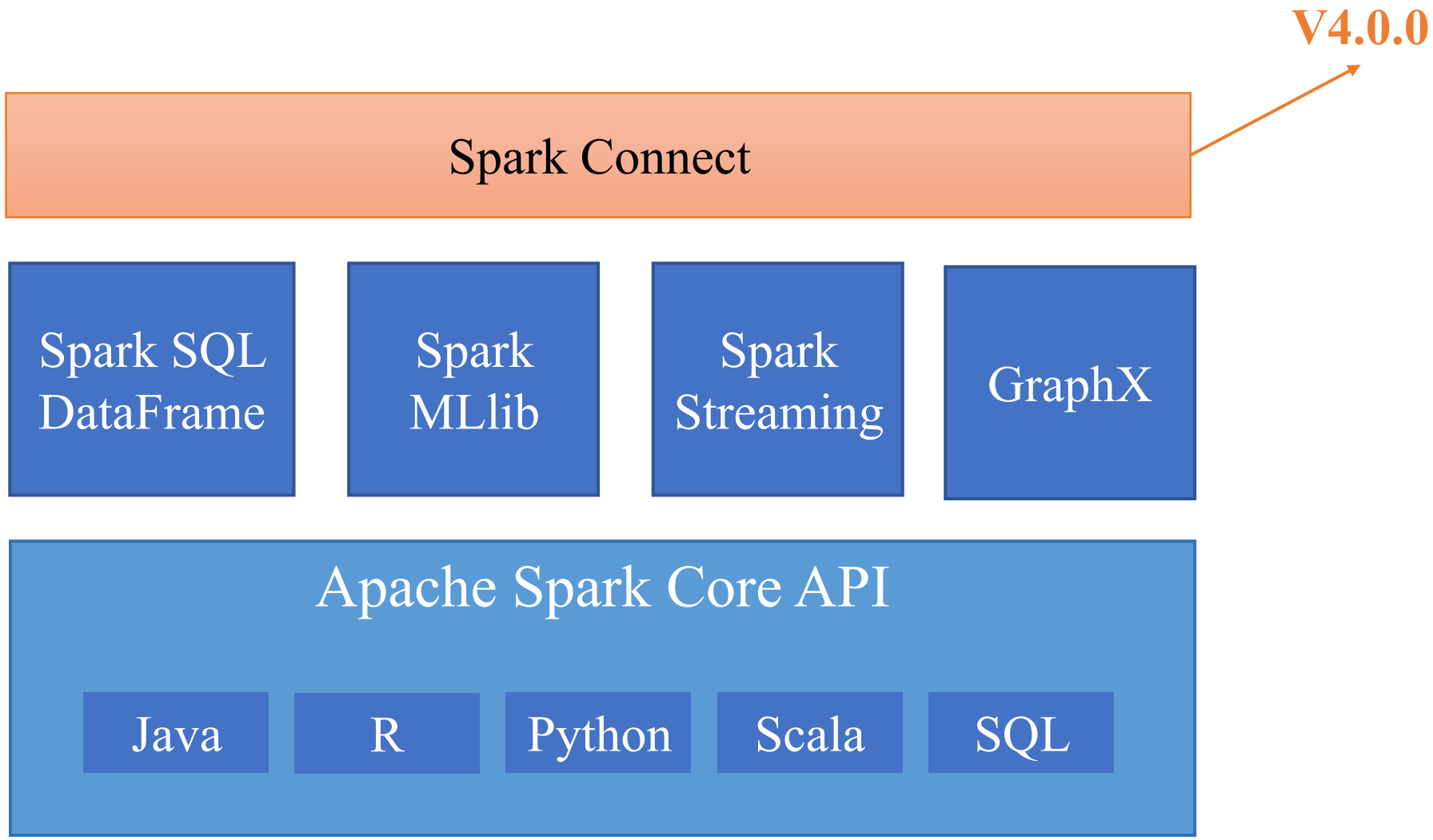
Apache Spark

❖ Apache Spark Components



Apache Spark

❖ Getting Started

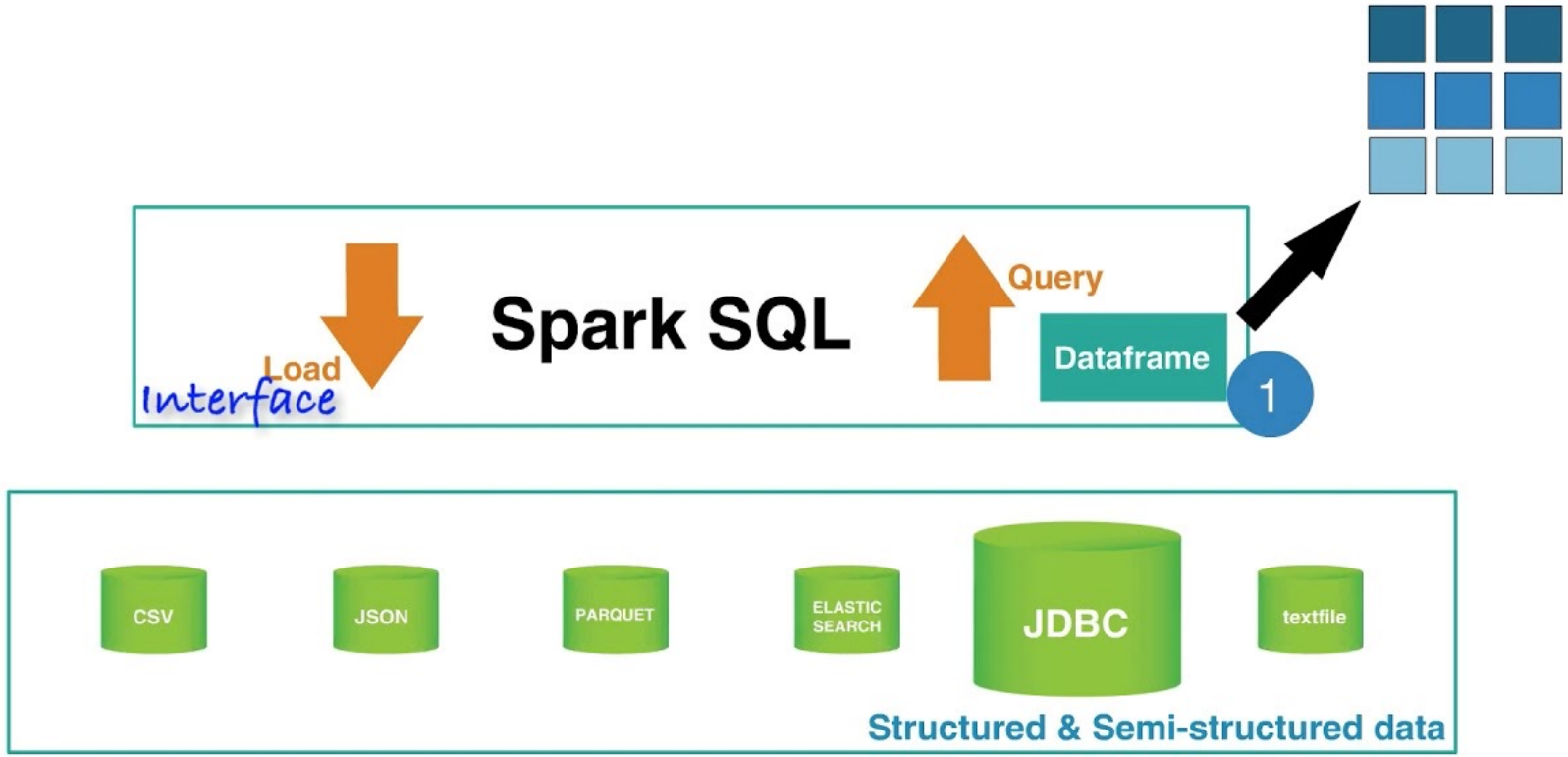


Apache Spark

❖ Spark SQL

Spark SQL integrates SQL queries with Spark's distributed data processing, combining SQL's expressiveness with Spark's speed and scalability.

- It can handle various **Spark SQL Spark SQL** data formats, including Parquet, JSON, and CSV.
- DataFrames provide a consistent way to work with all these different data sources.

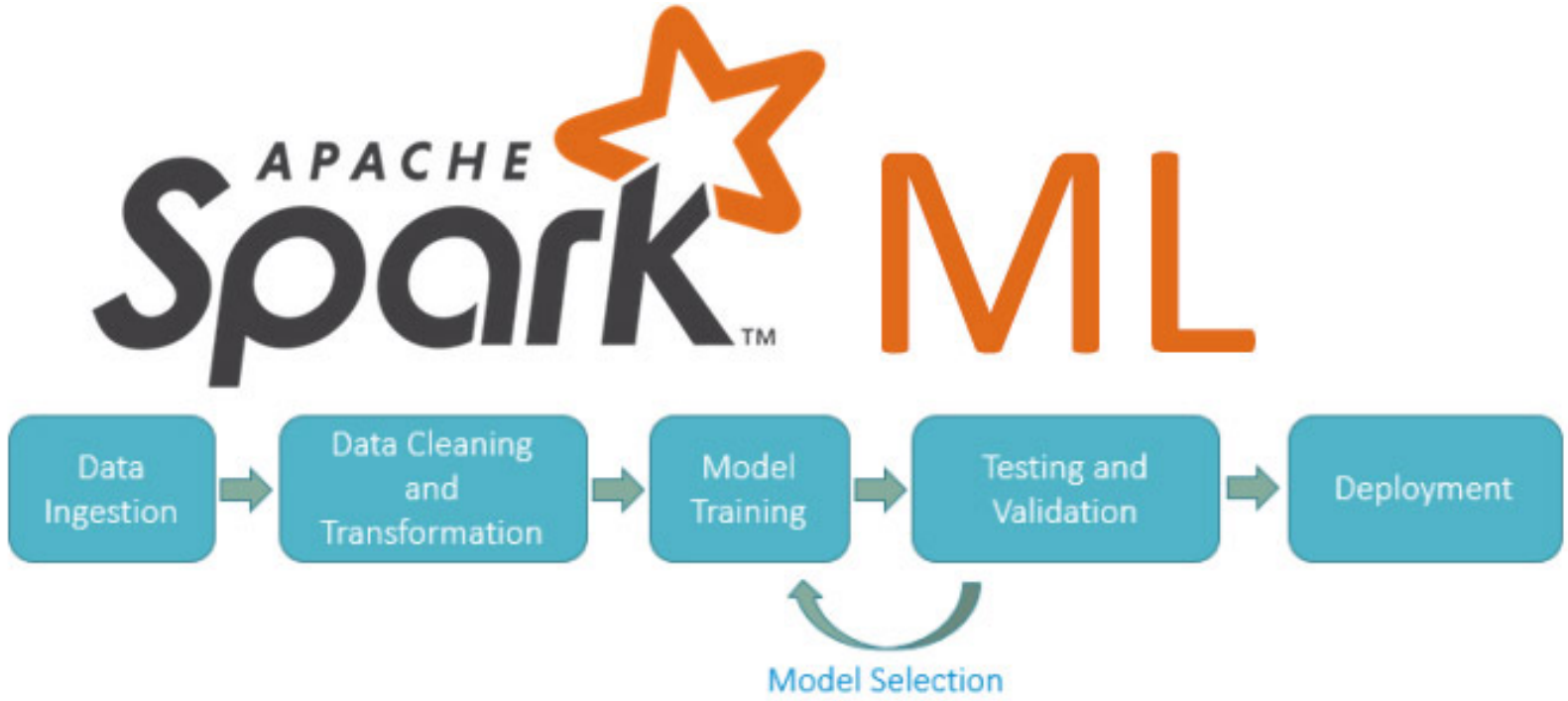


Apache Spark

❖ Spark MLlib (Machine Learning Library)

Various tools provided by MLlib include:

- ML Algorithms: collaborative filtering, classification, and clustering.
- Featurization: feature extraction, transformation, dimensionality reduction, and selection.
- Pipelines: tools for constructing, evaluating, and tuning ML Pipelines.



Apache Spark

❖ Spark Streaming

Spark Streaming is a powerful addition to Spark that allows for efficient processing of live data streams.

- It can connect to various data sources and perform different types of analysis on the data.
- The processed data can then be stored or visualized in real time.

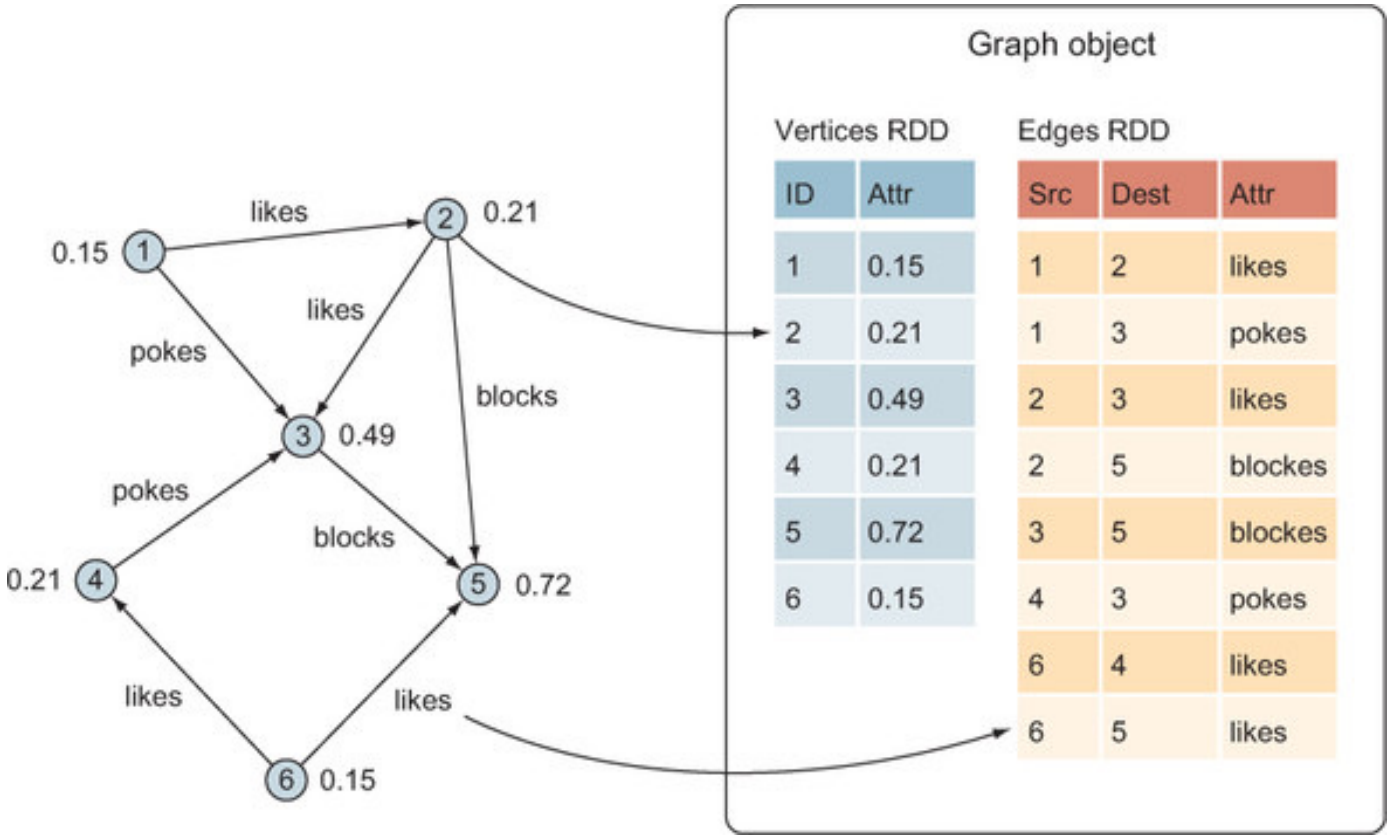


Apache Spark

❖ Spark GraphX

GraphX in Spark is API for graphs and graph parallel execution.

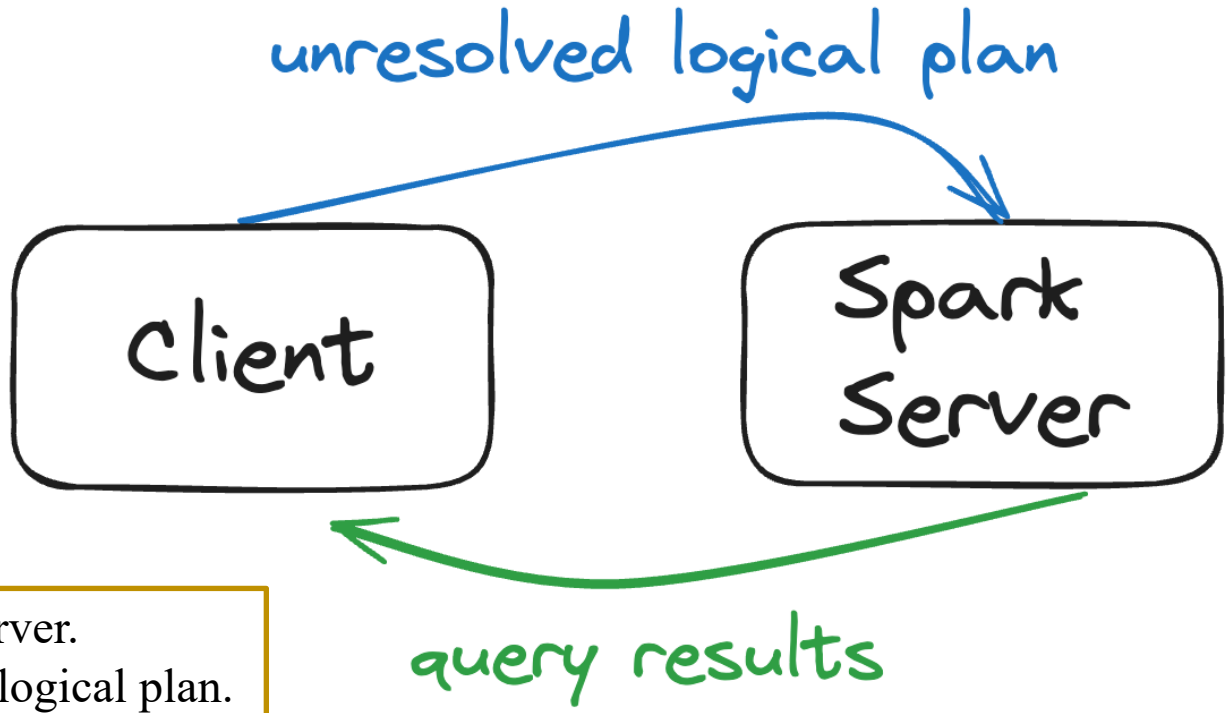
- It is network graph analytics engine and data store.
- Clustering, classification, traversal, searching, and pathfinding is also possible in graphs.



Apache Spark

❖ Spark Connect

Spark Connect is a protocol that defines how a client application communicates with a remote Spark server.



1. The client establishes a connection with the Spark server.
2. It translates the DataFrame query into an unresolved logical plan.
3. This logical plan is serialized and sent to the Spark server.
4. The server then analyzes, optimizes, and executes the query.
5. Finally, the Spark server returns the results to the client.

https://spark.apache.org/spark-connect/

Break time

❖ 5 Minutes



Quiz

❖ Choose one

Which PySpark code correctly reads a CSV file into a DataFrame?

A) df = spark.read.csv('data.csv')

B) df = pyspark.read('data.csv')

C) df = DataFrame.read('data.csv')

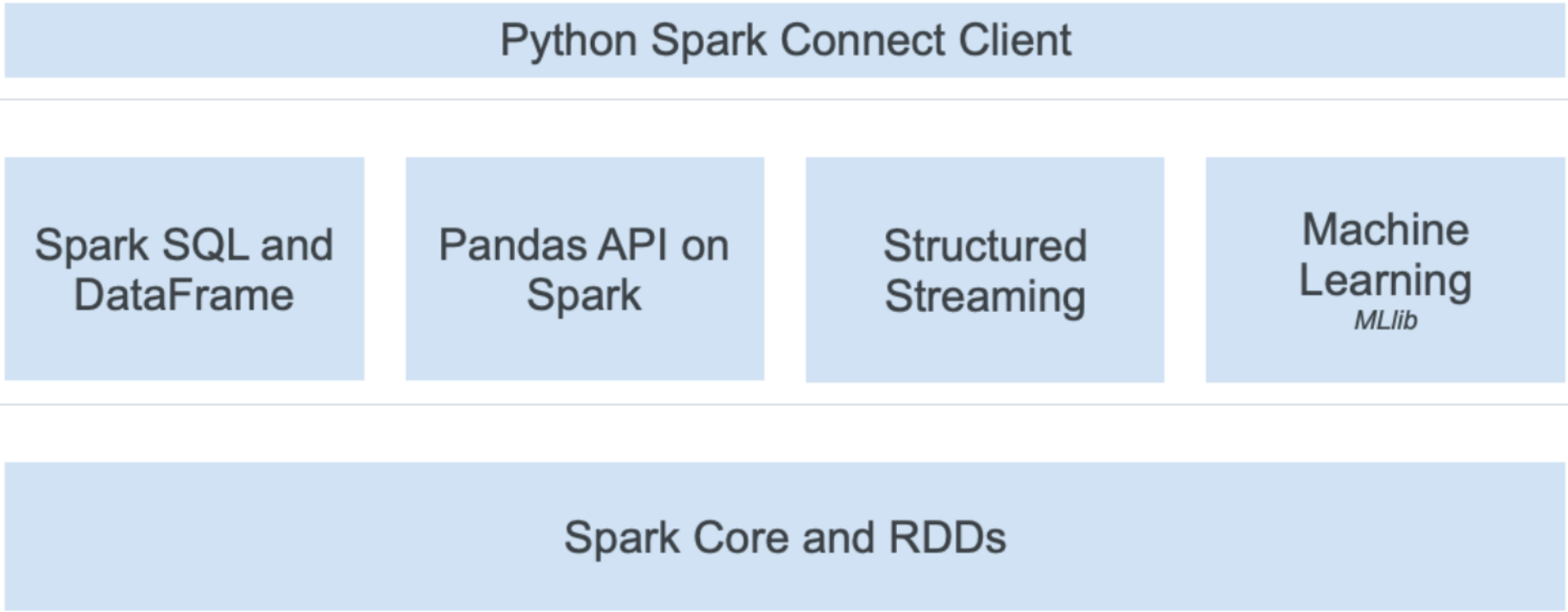
D) df = spark.sql.read('data.csv')

PySpark

PySpark

❖ Getting Started

v4.0.0



PySpark

❖ Getting Started

v4.0.0

```
---------------------------------------------------------------------------
PySparkRuntimeError                       Traceback (most recent call last)
/tmp/ipython-input-3760534044.py in <cell line: 0>()
      5     .config("spark.sql.adaptive.enabled", "true") \
      6     .config("spark.sql.adaptive.coalescePartitions.enabled", "true") \
----> 7     .getOrCreate()

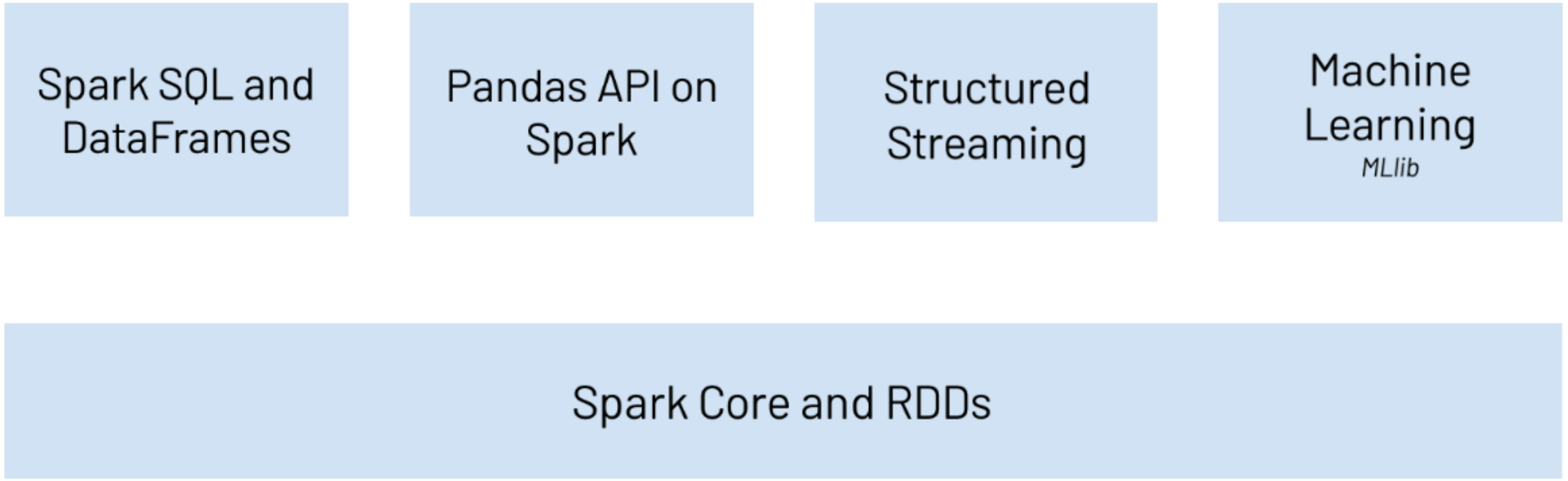
 4 frames
/usr/local/lib/python3.11/dist-packages/pyspark/java_gateway.py in launch_gateway(conf, popen_kwargs)
    109
    110             if not os.path.isfile(conn_info_file):
--> 111                 raise PySparkRuntimeError(
    112                     errorClass="JAVA_GATEWAY_EXITED",
    113                     messageParameters={},

PySparkRuntimeError: [JAVA_GATEWAY_EXITED] Java gateway process exited before sending its port number.
```

PySpark

❖ Getting Started

v3.5.x



https://archive.apache.org/dist/spark/docs/3.5.1/api/python/index.html

PySpark

❖ Getting Started

v3.5.x

```
[1]  1  !pip install pyspark==3.5.1
```

```
Requirement already satisfied: pyspark==3.5.1 in /usr/local/lib/python3.11
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.11
```

```
1  from pyspark.sql import SparkSession
2
3  spark = SparkSession.builder.getOrCreate()
4
5  spark
```

SparkSession - in-memory

SparkContext

Spark UI

Version
v3.5.1
Master
local[*]
AppName
pyspark-shell

PySpark

❖ PySpark Layers

Easier with DataFrame

Layer 2	**High Level APIs** Spart SQL, DF Streaming Mllib GraphX	
Layer 1	**Spark Core APIs** Scala Python Java R	

Working with RDD level

	Data file	Metadata
RDD	RAM	No
DataFrame	RAM	Yes
Spark SQL	Disk	Yes

```
person_id,name,sex,date of birth
100,Penelope Lewis,female,1990-08-31
105,David Simmons,male,1999-12-30
110,Freddie Claridge,male,2002-05-07
115,Robert Mcreynolds,male,1973-12-27
```

PySpark

❖ Example

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Intro") \
    .getOrCreate()

df = spark.read.csv("./data/people.csv", header=True, inferSchema=True)
df.show()

```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, u
25/08/07 18:21:57 WARN NativeCodeLoader: Unable to load native-hado
+-----+---------+----------------+------+-------------------+
|index|person_id|            name|   sex|      date of birth|
+-----+---------+----------------+------+-------------------+
|    0|      100|   Penelope Lewis|female|1990-08-31 00:00:00|
|    1|      101|    David Anthony|  male|1971-10-14 00:00:00|
|    2|      102|        Ida Shipp|female|1962-05-24 00:00:00|
|    3|      103|     Joanna Moore|female|2017-03-10 00:00:00|
|    4|      104|   Lisandra Ortiz|female|2020-08-05 00:00:00|
|    5|      105|    David Simmons|  male|1999-12-30 00:00:00|
|    6|      106|    Edward Hudson|  male|1983-05-09 00:00:00|
|    7|      107|     Albert Jones|  male|1990-09-13 00:00:00|
|    8|      108| Leonard Cavender|  male|1958-08-08 00:00:00|
|    9|      109|   Everett Vadala|  male|2005-05-24 00:00:00|
|   10|      110| Freddie Claridge|  male|2002-05-07 00:00:00|
|   11|      111|Annabelle Rosseau|female|1989-07-13 00:00:00|
|   12|      112|    Eulah Emanuel|female|1976-01-19 00:00:00|
|   13|      113|       Shaun Love|  male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|  male|1980-12-22 00:00:00|
|   15|      115|Robert Mcreynolds|  male|1973-12-27 00:00:00|
|   16|      116|   Carla Spickard|female|1985-06-13 00:00:00|
|   17|      117|Florence Eberhart|female|2024-06-01 00:00:00|
|   18|      118|     Tina Gaskins|female|1966-12-05 00:00:00|
|   19|      119| Florence Mulhern|female|1959-05-31 00:00:00|
+-----+---------+----------------+------+-------------------+
only showing top 20 rows
```

Spark DataFrame

Spark DataFrame

❖ Read csv file

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Intro2DF") \
    .getOrCreate()
```

```python
df = spark.read.csv("./data/large_dataset.csv", header=True, inferSchema=True)
df.show()
```

```
[Stage 1:>                                                          (0 + 12) / 12]
+--------------------+--------------------+--------------+---+----------+---+--------------------+
|                  id|                name|      username|sex|       dob|age|               email|
+--------------------+--------------------+--------------+---+----------+---+--------------------+
|40f38732-af30-4f9...|        Tiffany Lutz|williamsnatalie|  F|1947-11-16| 78|  mark53@hotmail.com|
|e5b492da-d0d4-4fe...|          Kyle Curry|  smithcourtney|  M|1918-11-18|107|torrescynthia@gma...|
|e5cd0077-44ca-46d...|        Robert Davis|      fanderson|  M|1978-05-14| 47| heather40@gmail.com|
|7e07dda8-d1c9-45f...|           Amy Duffy|        ccarson|  F|1992-04-09| 33|parrishamanda@yah...|
|de86af06-1b26-432...|    Catherine Davila|      russell46|  F|1991-01-31| 34|barbaraconley@hot...|
|91aa5e53-c57b-464...|     Nichole Ramirez|     jonesoscar|  F|1986-07-14| 39|christensenerica@...|
|43a1e938-a7ba-4f7...|        Angela Moore|     uhernandez|  F|1992-01-19| 33|michellerussell@y...|
|79261d97-c74a-41f...|          John White| joannamartinez|  M|1988-11-12| 37|donald91@hotmail.com|
|16645f34-33f3-408...|  Alexander Crawford|       karen37|  M|2022-04-04|  3|karenbrown@hotmai...|
|12603e3d-976c-45f...|        Jose Johnson|     danielle97|  M|1914-04-22|111|martinisabella@gm...|
|682c41de-0dfc-499...|        Amanda Green|   raymondclark|  F|1929-06-18| 96| chadsmith@gmail.com|
|fd1c2257-f6ce-49e...|Mr. Jonathan Solomon|  churchanthony|  M|1930-09-05| 95|  ahickman@yahoo.com|
|71f5cc40-abf7-43f...|          Gina Brown|  markalexander|  F|1971-07-22| 54|caroline71@gmail.com|
|1a1d3c9c-cb8e-462...|     Rachel Thompson|        wjarvis|  F|1910-05-24|115|crosskevin@yahoo.com|
|310e425e-9f2a-41d...|         Samuel Ward|       joshua90|  M|2002-12-25| 23|brownwillie@yahoo...|
|1ed55ec1-2ccb-48d...|   Christopher Baker|hannahwhitehead|  M|1910-05-08|115|     amy75@gmail.com|
|b48116d1-071b-412...|     Michael Aguirre|   willissteven|  M|2016-07-08|  9|montgomerysavanna...|
|c84e3949-f393-4db...|      Anthony Conrad|      zachary21|  M|2012-04-01| 13|michael91@hotmail...|
|cdfdcdbc-bcc6-4ac...|     Brandon Vazquez|    lewisarthur|  M|1946-06-17| 79|amandabuckley@hot...|
|0a7642a4-252f-4b8...|       Arthur Powell|      qthompson|  M|1931-12-05| 94|   vrogers@gmail.com|
+--------------------+--------------------+--------------+---+----------+---+--------------------+
only showing top 20 rows
```

Spark DataFrame

❖ Describe

```
# Describe the DataFrame

df.describe().show()
```

```
tringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
==========================>        (10 + 2) / 12]
+------------+--------+-------+-----------------+-----------------+-------------------+-------------------+----------+-------+----------+------------+----------+-----------------+
|        name|username|    sex|              age|            email|       phone_number|            address|      city|  state|   country|     company|occupation|           salary|
+------------+--------+-------+-----------------+-----------------+-------------------+-------------------+----------+-------+----------+------------+----------+-----------------+
|     5000000| 5000000|5000000|          5000000|          5000000|            5000000|            5000000|   5000000|5000000|   5000000|     5000000|   5000000|          5000000|
|        NULL|    NULL|   NULL|        57.888388|             NULL|6.00480884955549965E9|               NULL|      NULL|   NULL|      NULL|        NULL|      NULL|    89968.5991896|
|        NULL|    NULL|   NULL|33.49435612063372|             NULL| 2.309860964221136E9|               NULL|      NULL|   NULL|      NULL|        NULL|      NULL|34642.29185806566|
|Aaron Abbott| aabbott|      F|                0|aabbott@gmail.com|  (200)201-0423x2580|000 Aaron Club, L...|  Aaronberg|Alabama|Afghanistan| Abbott Group|    Artist|            30000|
|  Zoe Zamora| zzuniga|      M|              116|zzuniga@yahoo.com|         9999972126|Unit 9999 Box 884...|Zunigaville|Wyoming|  Zimbabwe|Zuniga-Zamora|   Teacher|           150000|
+------------+--------+-------+-----------------+-----------------+-------------------+-------------------+----------+-------+----------+------------+----------+-----------------+
```

Spark DataFrame

❖ Select

```
1  # Selecting specific columns
2
3  df.select("name", "age").show()
```

```
...  +--------------------+---+
     |                name|age|
     +--------------------+---+
     |        Tiffany Lutz| 78|
     |          Kyle Curry|107|
     |        Robert Davis| 47|
     |           Amy Duffy| 33|
     |    Catherine Davila| 34|
     |     Nichole Ramirez| 39|
     |        Angela Moore| 33|
     |          John White| 37|
     |  Alexander Crawford|  3|
     |        Jose Johnson|111|
     |        Amanda Green| 96|
     |Mr. Jonathan Solomon| 95|
     |          Gina Brown| 54|
     |     Rachel Thompson|115|
     |         Samuel Ward| 23|
     |   Christopher Baker|115|
     |     Michael Aguirre|  9|
     |      Anthony Conrad| 13|
     |     Brandon Vazquez| 79|
     |       Arthur Powell| 94|
     +--------------------+---+
     only showing top 20 rows
```

Spark DataFrame

❖ Filter

```python
from pyspark.sql.functions import col

# Filtering rows based on a condition
df.filter(col("age") > 30).show()
```

```
+--------------------+--------------+---+----------+---+
|                name|      username|sex|       dob|age|
+--------------------+--------------+---+----------+---+
|        Tiffany Lutz|williamsnatalie|  F|1947-11-16| 78|
|          Kyle Curry|  smithcourtney|  M|1918-11-18|107|
|        Robert Davis|      fanderson|  M|1978-05-14| 47|
|           Amy Duffy|        ccarson|  F|1992-04-09| 33|
|    Catherine Davila|      russell46|  F|1991-01-31| 34|
|     Nichole Ramirez|     jonesoscar|  F|1986-07-14| 39|
|        Angela Moore|     uhernandez|  F|1992-01-19| 33|
|          John White| joannamartinez|  M|1988-11-12| 37|
|        Jose Johnson|     danielle97|  M|1914-04-22|111|
|        Amanda Green|   raymondclark|  F|1929-06-18| 96|
|Mr. Jonathan Solomon|  churchanthony|  M|1930-09-05| 95|
|          Gina Brown|  markalexander|  F|1971-07-22| 54|
|     Rachel Thompson|        wjarvis|  F|1910-05-24|115|
|   Christopher Baker|hannahwhitehead|  M|1910-05-08|115|
|     Brandon Vazquez|    lewisarthur|  M|1946-06-17| 79|
|       Arthur Powell|      qthompson|  M|1931-12-05| 94|
|       Kelly Simmons|   justinobrien|  F|1943-10-22| 82|
|     Christian Green|   perkinsjamie|  M|1994-06-26| 31|
|      Thomas Douglas|         eric29|  M|1918-01-06|107|
|          Kerry Howe|waltonchristine|  M|1985-02-23| 40|
+--------------------+--------------+---+----------+---+
```

Spark DataFrame

❖ Add column

```
# Adding a new column with a calculation

df.withColumn("salary_plus_tax", col("salary") * 1.1).show()
```

```
+----------+------+------------------+
|occupation|salary|   salary_plus_tax|
+----------+------+------------------+
|    Artist| 56582|62240.200000000004|
|  Engineer|144167|          158583.7|
|   Teacher| 88417| 97258.70000000001|
|  Engineer| 69203|           76123.3|
|    Doctor| 80345|           88379.5|
|  Engineer| 91566|          100722.6|
|    Artist|131917|          145108.7|
|  Engineer| 88805| 97685.50000000001|
|    Artist|115572|127129.20000000001|
|  Engineer|105501|          116051.1|
|   Teacher| 90162| 99178.20000000001|
| Scientist| 40838|           44921.8|
|  Engineer|110308|          121338.8|
|    Doctor| 47995| 52794.50000000001|
|    Doctor|107558|          118313.8|
|    Artist|123635|          135998.5|
|    Doctor| 42994|           47293.4|
|   Teacher|101874|112061.40000000001|
|  Engineer|136750|          150425.0|
|   Teacher| 55686|61254.600000000006|
+----------+------+------------------+
```

Spark DataFrame

❖ Grouping

```python
from pyspark.sql.functions import avg

# Grouping by a column and calculating an aggregate
df.groupBy("occupation").agg(avg("salary")).show()
```

```
[Stage 6:>
+----------+------------------+
|occupation|       avg(salary)|
+----------+------------------+
| Scientist|89942.17145877547|
|   Teacher|90022.85484275773|
|    Artist| 89962.5090149945|
|    Doctor|89916.63644985134|
|  Engineer|89998.75605945347|
+----------+------------------+
```

Spark DataFrame

❖ UDF

```python
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Defining a UDF to categorize age
def categorize_age(age):
    if age < 20:
        return "teenager"
    elif 20 ⩽ age < 60:
        return "adult"
    else:
        return "senior"

categorize_age_udf = udf(categorize_age, StringType())

# Applying the UDF to create a new column
df.withColumn("age_category", categorize_age_udf(col("age"))) \
    .select("age", "age_category") \
    .show()
```

```
+---+------------+
|age|age_category|
+---+------------+
| 78|      senior|
|107|      senior|
| 47|       adult|
| 33|       adult|
| 34|       adult|
| 39|       adult|
| 33|       adult|
| 37|       adult|
|  3|    teenager|
|111|      senior|
| 96|      senior|
| 95|      senior|
| 54|       adult|
|115|      senior|
| 23|       adult|
|115|      senior|
|  9|    teenager|
| 13|    teenager|
| 79|      senior|
| 94|      senior|
+---+------------+
only showing top 20 rows
```

Spark SQL

Spark SQL

❖ PySpark Read Data: CSV

```python
df_csv = spark.read.csv("./people.csv" ,
                        header=True,
                        inferSchema=True)
```

```python
df_csv.printSchema()
```

[5] ✓ 0.0s

```
root
 |-- _c0: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)
```

people.csv

PySpark > people.csv > data

```
,person_id,name,sex,date of birth
0,100,Penelope Lewis,female,1990-08-31
1,101,David Anthony,male,1971-10-14
2,102,Ida Shipp,female,1962-05-24
3,103,Joanna Moore,female,2017-03-10
4,104,Lisandra Ortiz,female,2020-08-05
5,105,David Simmons,male,1999-12-30
6,106,Edward Hudson,male,1983-05-09
7,107,Albert Jones,male,1990-09-13
8,108,Leonard Cavender,male,1958-08-08
9,109,Everett Vadala,male,2005-05-24
10,110,Freddie Claridge,male,2002-05-07
```

Spark SQL

❖ PySpark Read Data: JSON

```
df_json = spark.read.json("./people.json")
```

```
df_json.printSchema()
```

[24] ✓ 0.0s

```
root
 |-- date of birth: string (nullable = true)
 |-- name: string (nullable = true)
 |-- person_id: long (nullable = true)
 |-- sex: string (nullable = true)
```


```
{"person_id": 100, "name": "Penelope Lewis", "sex": "female", "date of birth": "1990-08-31"}
{"person_id": 101, "name": "David Anthony", "sex": "male", "date of birth": "1971-10-14"}
{"person_id": 102, "name": "Ida Shipp", "sex": "female", "date of birth": "1962-05-24"}
{"person_id": 103, "name": "Joanna Moore", "sex": "female", "date of birth": "2017-03-10"}
{"person_id": 104, "name": "Lisandra Ortiz", "sex": "female", "date of birth": "2020-08-05"}
{"person_id": 105, "name": "David Simmons", "sex": "male", "date of birth": "1999-12-30"}
{"person_id": 106, "name": "Edward Hudson", "sex": "male", "date of birth": "1983-05-09"}
{"person_id": 107, "name": "Albert Jones", "sex": "male", "date of birth": "1990-09-13"}
{"person_id": 108, "name": "Leonard Cavender", "sex": "male", "date of birth": "1958-08-08"}
{"person_id": 109, "name": "Everett Vadala", "sex": "male", "date of birth": "2005-05-24"}
{"person_id": 110, "name": "Freddie Claridge", "sex": "male", "date of birth": "2002-05-07"}
```

Spark SQL

❖ PySpark Read Data: TXT

```
1  df_text = spark.read.text("./text_data.txt")
```

```
[42]  df_text.printSchema()
      ✓ 0.0s
...   root
       |-- value: string (nullable = true)
```

```
[54]  df_text.show()
      ✓ 0.1s
...   +--------------------+
      |               value|
      +--------------------+
      | Dữ liệu có cấu trúc|
      |Dữ liệu có cấu tr...|
      |để quản lý và tìm...|
      |lưu trữ và xử lý ...|
      |Các thành phần củ...|
      |cho phép các nhà ...|
      |đơn giản để tìm k...|
      |Dữ liệu phi cấu trúc|
      |Dữ liệu phi cấu t...|
```

```
1  Dữ liệu có cấu trúc
2  Dữ liệu có cấu trúc được xem là dữ liệu đơn giản nhất
3  để quản lý và tìm kiếm. Nó là những dữ liệu có thể truy cập,
4  lưu trữ và xử lý ở định dạng cố định.
5  Các thành phần của dữ liệu có cấu trúc được phân loại dễ dàng,
6  cho phép các nhà thiết kế và quản trị viên cơ sở dữ liệu xác định các thuật toán
7  đơn giản để tìm kiếm và phân tích.
8  Dữ liệu phi cấu trúc
9  Dữ liệu phi cấu trúc là bất kỳ tập hợp dữ liệu nào không được tổ chức
10 hoặc xác định rõ ràng. Loại dữ liệu này hỗn loạn, khó xử lý, khó hiểu và đánh giá.
11 Nó không có cấu trúc cố định và có thể thay đổi vào những thời điểm khác nhau.
12 Dữ liệu phi cấu trúc bao gồm các nhận xét,
13 tweet, lượt chia sẻ, bài đăng trên mạng xã hội,
```

Spark SQL

❖ PySpark DataFrame: select()

```
df_csv.printSchema()
df_csv.show(truncate=True)
```

[3] ✓ 0.2s

```
root
 |-- index: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)

+-----+---------+----------------+------+-------------------+
|index|person_id|            name|   sex|      date of birth|
+-----+---------+----------------+------+-------------------+
|    0|      100|  Penelope Lewis|female|1990-08-31 00:00:00|
|    1|      101|   David Anthony|  male|1971-10-14 00:00:00|
|    2|      102|       Ida Shipp|female|1962-05-24 00:00:00|
|    3|      103|    Joanna Moore|female|2017-03-10 00:00:00|
|    4|      104|  Lisandra Ortiz|female|2020-08-05 00:00:00|
|    5|      105|   David Simmons|  male|1999-12-30 00:00:00|
|    6|      106|   Edward Hudson|  male|1983-05-09 00:00:00|
|    7|      107|    Albert Jones|  male|1990-09-13 00:00:00|
|    8|      108|Leonard Cavender|  male|1958-08-08 00:00:00|
|    9|      109|  Everett Vadala|  male|2005-05-24 00:00:00|
|   10|      110|Freddie Claridge|  male|2002-05-07 00:00:00|
|   11|      111|Annabelle Rosseau|female|1989-07-13 00:00:00|
|   12|      112|   Eulah Emanuel|female|1976-01-19 00:00:00|
|   13|      113|      Shaun Love|  male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|  male|1980-12-22 00:00:00|
...
|   19|      119|Florence Mulhern|female|1959-05-31 00:00:00|
+-----+---------+----------------+------+-------------------+
only showing top 20 rows
```

```
df_csv.select("name").show(truncate=True)
```

[4] ✓ 0.1s

```
+----------------+
|            name|
+----------------+
|  Penelope Lewis|
|   David Anthony|
|       Ida Shipp|
|    Joanna Moore|
|  Lisandra Ortiz|
|   David Simmons|
|   Edward Hudson|
|    Albert Jones|
|Leonard Cavender|
|  Everett Vadala|
|Freddie Claridge|
|Annabelle Rosseau|
|   Eulah Emanuel|
|      Shaun Love|
|Alejandro Brennan|
|Robert Mcreynolds|
|  Carla Spickard|
|Florence Eberhart|
|    Tina Gaskins|
|Florence Mulhern|
+----------------+
only showing top 20 rows
```

Spark SQL

❖ PySpark DataFrame: filter()

```
df_csv.printSchema()
df_csv.show(truncate=True)
```

[3] ✓ 0.2s

```
root
 |-- index: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)

+-----+---------+----------------+------+-------------------+
|index|person_id|            name|   sex|      date of birth|
+-----+---------+----------------+------+-------------------+
|    0|      100|  Penelope Lewis|female|1990-08-31 00:00:00|
|    1|      101|   David Anthony|  male|1971-10-14 00:00:00|
|    2|      102|       Ida Shipp|female|1962-05-24 00:00:00|
|    3|      103|    Joanna Moore|female|2017-03-10 00:00:00|
|    4|      104|  Lisandra Ortiz|female|2020-08-05 00:00:00|
|    5|      105|   David Simmons|  male|1999-12-30 00:00:00|
|    6|      106|   Edward Hudson|  male|1983-05-09 00:00:00|
|    7|      107|    Albert Jones|  male|1990-09-13 00:00:00|
|    8|      108|Leonard Cavender|  male|1958-08-08 00:00:00|
|    9|      109|  Everett Vadala|  male|2005-05-24 00:00:00|
|   10|      110|Freddie Claridge|  male|2002-05-07 00:00:00|
|   11|      111|Annabelle Rosseau|female|1989-07-13 00:00:00|
|   12|      112|   Eulah Emanuel|female|1976-01-19 00:00:00|
|   13|      113|      Shaun Love|  male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|  male|1980-12-22 00:00:00|
...
|   19|      119|Florence Mulhern|female|1959-05-31 00:00:00|
+-----+---------+----------------+------+-------------------+
only showing top 20 rows
```

```
df_csv.filter(df_csv.sex == "male").show(truncate=True)
```

[6] ✓ 0.2s

```
+-----+---------+------------------+----+-------------------+
|index|person_id|              name| sex|      date of birth|
+-----+---------+------------------+----+-------------------+
|    1|      101|     David Anthony|male|1971-10-14 00:00:00|
|    5|      105|     David Simmons|male|1999-12-30 00:00:00|
|    6|      106|     Edward Hudson|male|1983-05-09 00:00:00|
|    7|      107|      Albert Jones|male|1990-09-13 00:00:00|
|    8|      108|  Leonard Cavender|male|1958-08-08 00:00:00|
|    9|      109|    Everett Vadala|male|2005-05-24 00:00:00|
|   10|      110|  Freddie Claridge|male|2002-05-07 00:00:00|
|   13|      113|       Shaun Love|male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|male|1980-12-22 00:00:00|
|   15|      115|Robert Mcreynolds|male|1973-12-27 00:00:00|
|   20|      120|       Joel Smith|male|1996-08-13 00:00:00|
|   23|      123|      Angel Moher|male|2017-12-22 00:00:00|
|   24|      124|   Charles Leonard|male|1972-03-09 00:00:00|
|   25|      125|      Mark Miller|male|1976-05-11 00:00:00|
|   29|      129|      David Bishop|male|1960-10-18 00:00:00|
|   31|      131|     Joseph Windus|male|2029-05-27 00:00:00|
|   32|      132|Christopher Gilbert|male|2010-09-21 00:00:00|
|   33|      133|  Robert Salisbury|male|1965-06-22 00:00:00|
|   36|      136|     Wilbert Glass|male|1976-10-10 00:00:00|
|   40|      140|         Juan Dunn|male|1969-10-06 00:00:00|
+-----+---------+------------------+----+-------------------+
only showing top 20 rows
```

Spark SQL

❖ PySpark DataFrame: groupby()

```
df_csv.printSchema()
df_csv.show(truncate=True)
```

[3] ✓ 0.2s

```
root
 |-- index: integer (nullable = true)
 |-- person_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- sex: string (nullable = true)
 |-- date of birth: timestamp (nullable = true)

+-----+---------+----------------+------+-------------------+
|index|person_id|            name|   sex|      date of birth|
+-----+---------+----------------+------+-------------------+
|    0|      100|   Penelope Lewis|female|1990-08-31 00:00:00|
|    1|      101|    David Anthony|  male|1971-10-14 00:00:00|
|    2|      102|        Ida Shipp|female|1962-05-24 00:00:00|
|    3|      103|     Joanna Moore|female|2017-03-10 00:00:00|
|    4|      104|   Lisandra Ortiz|female|2020-08-05 00:00:00|
|    5|      105|    David Simmons|  male|1999-12-30 00:00:00|
|    6|      106|    Edward Hudson|  male|1983-05-09 00:00:00|
|    7|      107|     Albert Jones|  male|1990-09-13 00:00:00|
|    8|      108| Leonard Cavender|  male|1958-08-08 00:00:00|
|    9|      109|   Everett Vadala|  male|2005-05-24 00:00:00|
|   10|      110| Freddie Claridge|  male|2002-05-07 00:00:00|
|   11|      111|Annabelle Rosseau|female|1989-07-13 00:00:00|
|   12|      112|    Eulah Emanuel|female|1976-01-19 00:00:00|
|   13|      113|       Shaun Love|  male|1970-05-26 00:00:00|
|   14|      114|Alejandro Brennan|  male|1980-12-22 00:00:00|
...
|   19|      119| Florence Mulhern|female|1959-05-31 00:00:00|
+-----+---------+----------------+------+-------------------+
only showing top 20 rows
```

```
df_groupby_sex = df_csv.groupby('sex')
print(type(df_groupby_sex))
```

[7] ✓ 0.0s

```
<class 'pyspark.sql.group.GroupedData'>
```

```
df_groupby_sex.count().show()
```

[8] ✓ 0.9s

```
+------+-----+
|sex   |count|
+------+-----+
|NULL  |1920 |
|female|49014|
|male  |49066|
+------+-----+
```

Spark SQL

❖ PySpark SQL: SQL query

```
1  df_csv.createOrReplaceTempView("people")
```

```
1  query = '''SELECT name, sex
2  FROM people'''
3
4  df2 = spark.sql(query)
```

```
[8]  df2.show(truncate=True)

+----------------+------+
|            name|   sex|
+----------------+------+
|   Penelope Lewis|female|
|    David Anthony|  male|
|        Ida Shipp|female|
|     Joanna Moore|female|
|   Lisandra Ortiz|female|
|    David Simmons|  male|
|    Edward Hudson|  male|
|     Albert Jones|  male|
| Leonard Cavender|  male|
|   Everett Vadala|  male|
| Freddie Claridge|  male|
|Annabelle Rosseau|female|
|    Eulah Emanuel|female|
|       Shaun Love|  male|
|Alejandro Brennan|  male|
|Robert Mcreynolds|  male|
|   Carla Spickard|female|
|Florence Eberhart|female|
|     Tina Gaskins|female|
| Florence Mulhern|female|
+----------------+------+
only showing top 20 rows
```

Spark SQL

❖ PySpark SQL: grouping

```
1  df_csv.createOrReplaceTempView("people")
```

```
1  query = '''SELECT sex, COUNT(person_id) FROM people GROUP BY sex'''
2
3  df2 = spark.sql(query)
```

```
df2.show()
[23]  ✓ 0.3s
...  +------+----------------+
     |   sex|count(person_id)|
     +------+----------------+
     |  NULL|            1920|
     |female|           49014|
     |  male|           49066|
     +------+----------------+
```

Spark SQL

❖ PySpark SQL: filter

```
1  df_csv.createOrReplaceTempView("people")
```

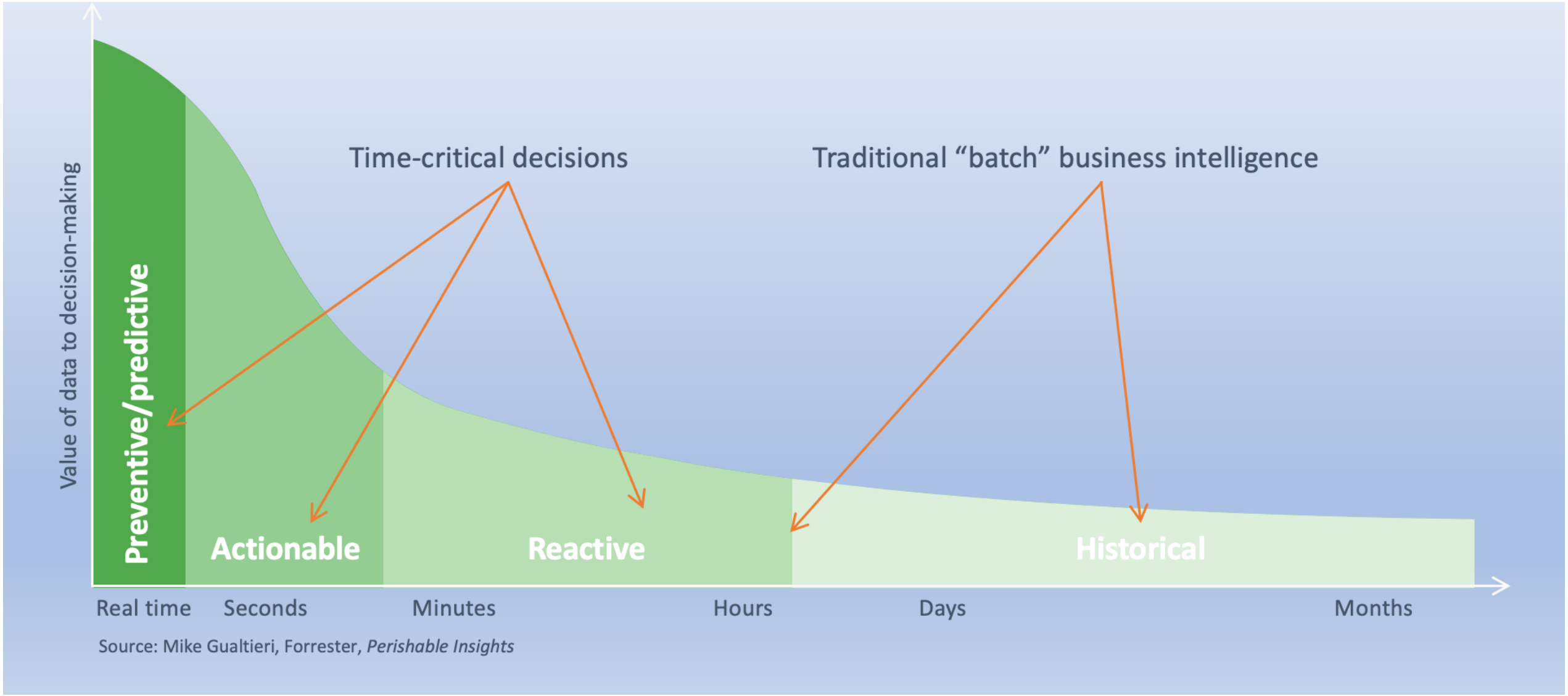
```
1  query = '''SELECT name, sex
2  FROM people
3  WHERE sex = "female"'''
4
5  df2 = spark.sql(query)
```

```
df2.show(truncate=True)
[18]  ✓ 0.2s
...
+----------------+------+
|            name|   sex|
+----------------+------+
|   Penelope Lewis|female|
|        Ida Shipp|female|
|     Joanna Moore|female|
|   Lisandra Ortiz|female|
|Annabelle Rosseau|female|
|    Eulah Emanuel|female|
|   Carla Spickard|female|
|Florence Eberhart|female|
|     Tina Gaskins|female|
| Florence Mulhern|female|
|    Evelyn Kriner|female|
|     Heather Luce|female|
|      Marion Baca|female|
|       Devona Kay|female|
|   Betty Endicott|female|
|        Jane Ross|female|
|   Pauline Steele|female|
|     Anne Novotny|female|
|      Carol Noble|female|
| Constance Fulmer|female|
+----------------+------+
only showing top 20 rows
```

Spark Streaming

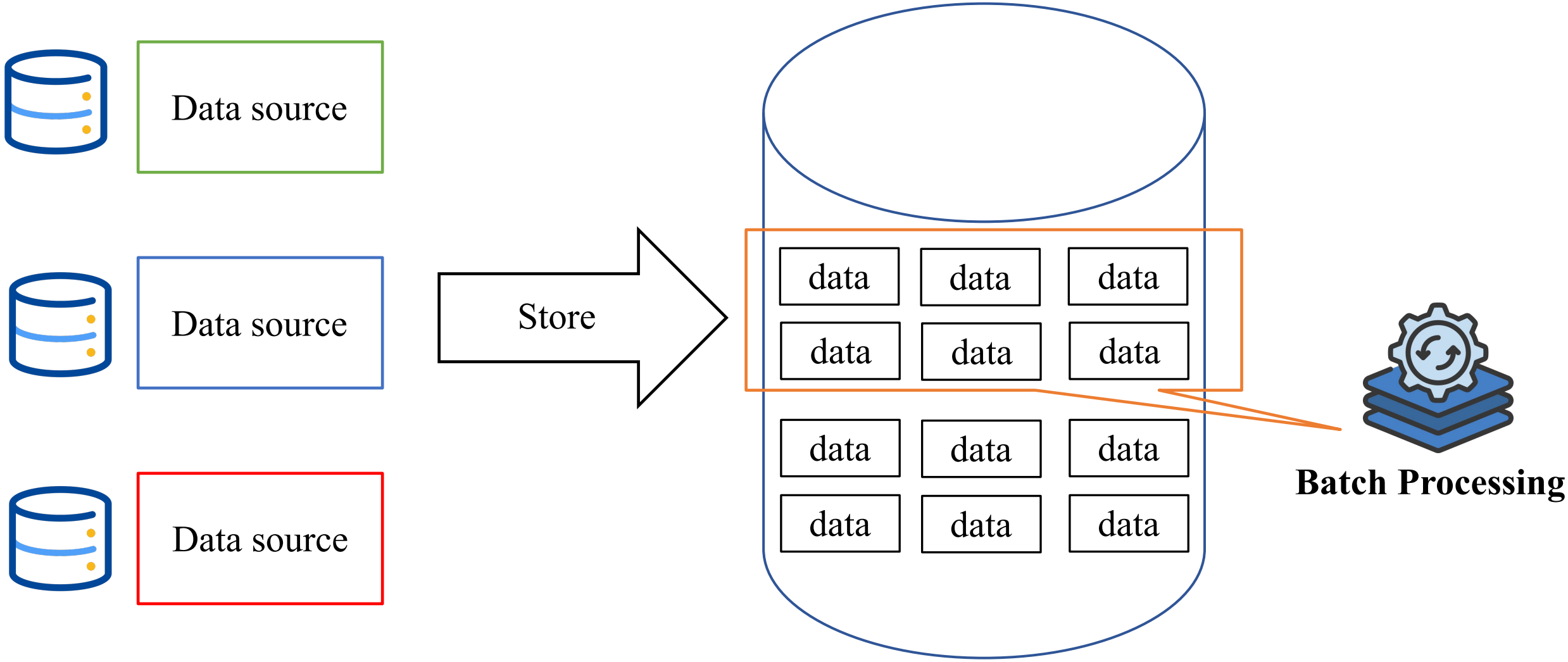
Spark Streaming

❖ The diminishing Value of Data



Spark Streaming

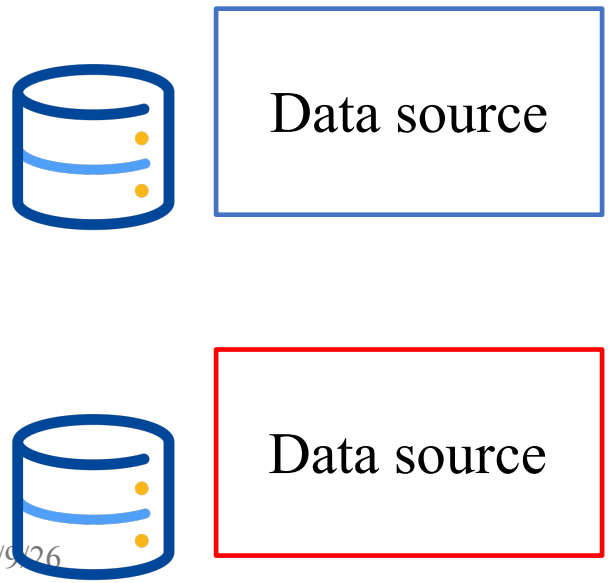
❖ Batch Processing vs Stream Processing



Spark Streaming

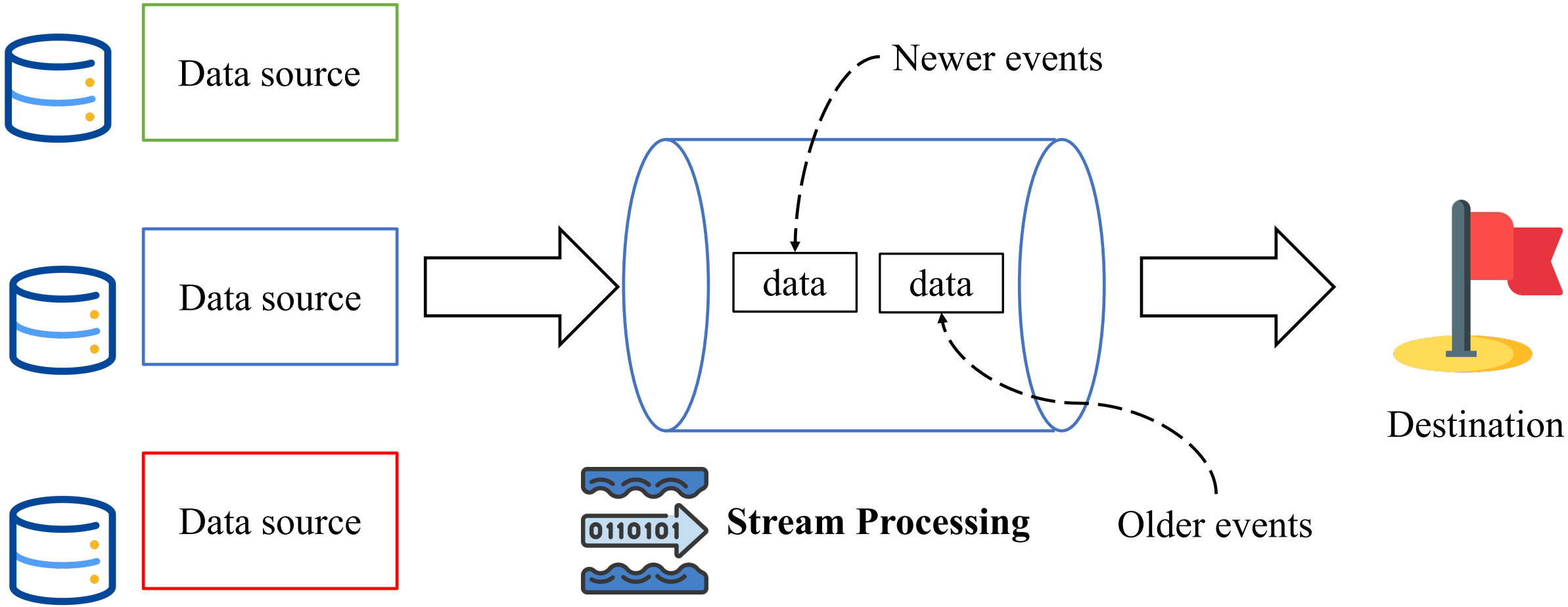
❖ Batch Processing vs Stream Processing

- Typically used for processing **large amounts of data**.
- Ideal for tasks that do not require real-time processing



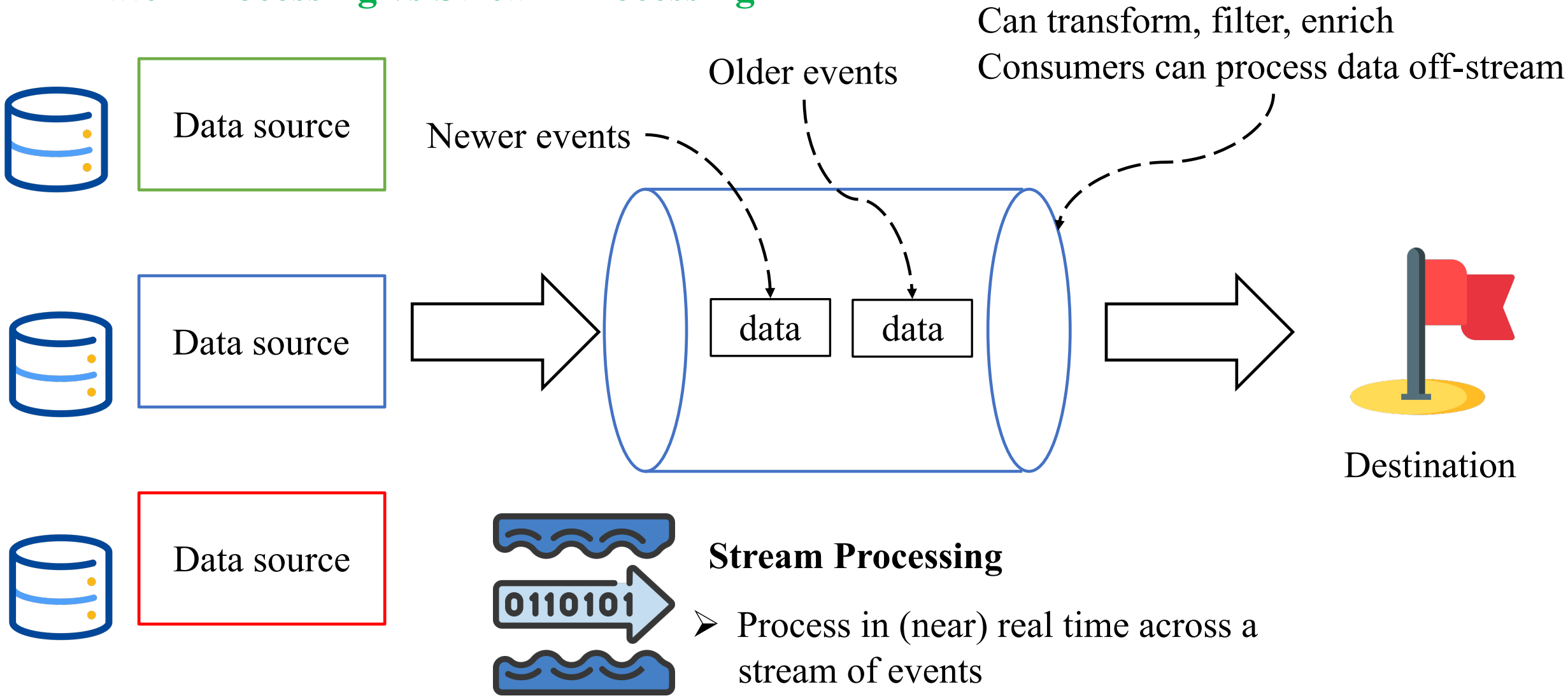
Spark Streaming

❖ Batch Processing vs Stream Processing



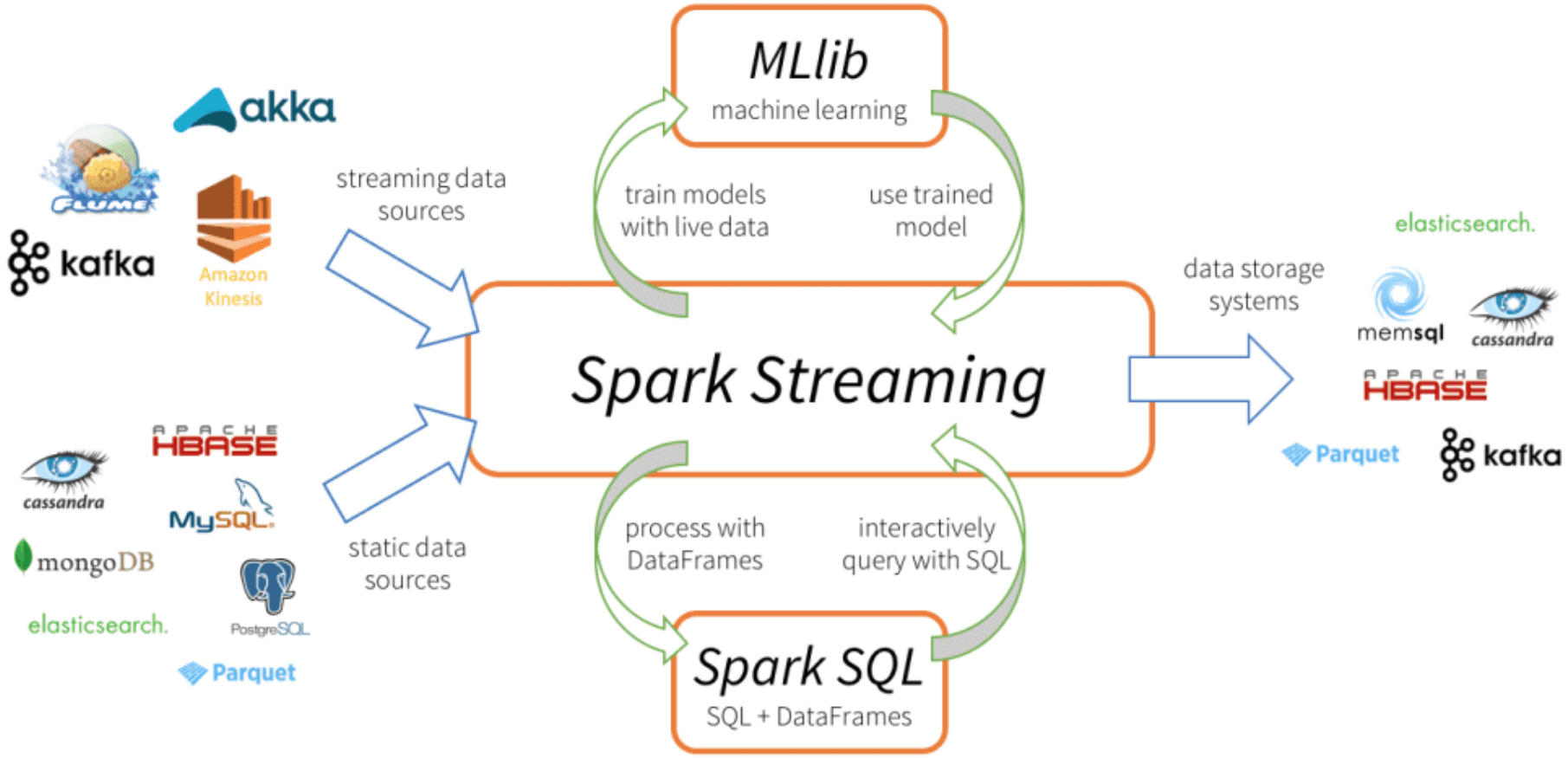
Spark Streaming

❖ Batch Processing vs Stream Processing



Spark Streaming

❖ Overview



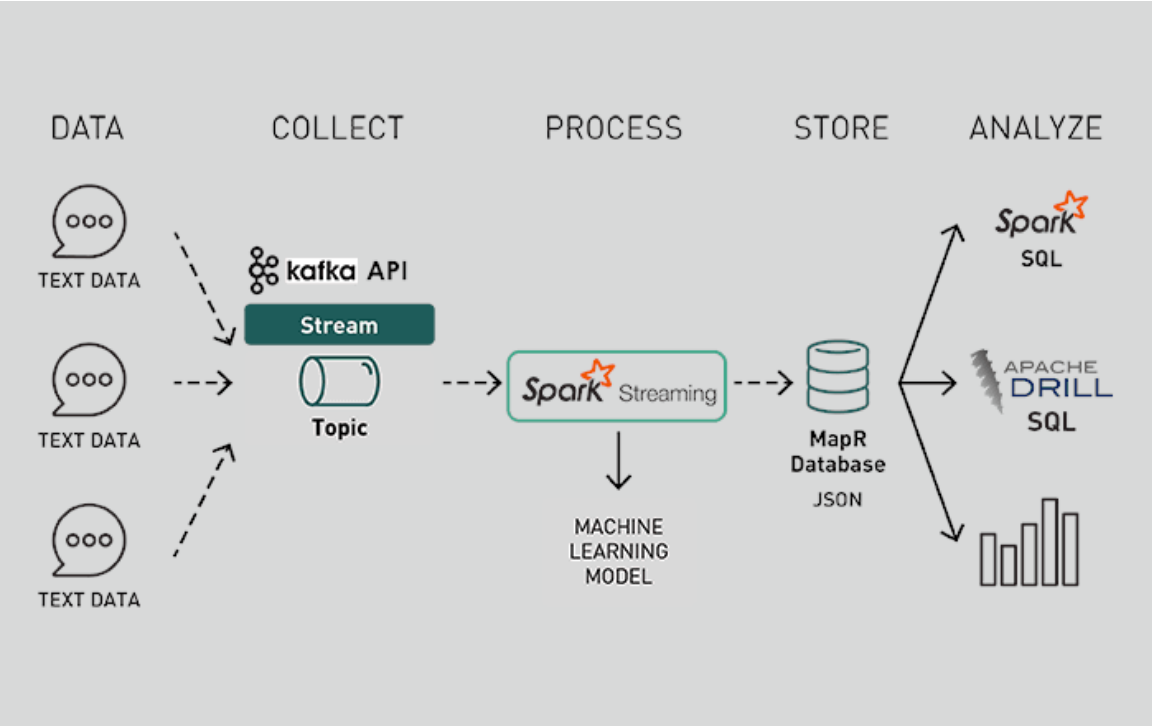
- **Spark Streaming** was launched in 2013 to enable data engineers and data scientists to **process real-time data** from SQL databases, Flume, Amazon Kinesis, etc.
- **Spark Streaming** to seamlessly **integrate** with any **other Spark components** like MLlib and Spark SQL

https://www.projectpro.io/article/spark-streaming-example/540

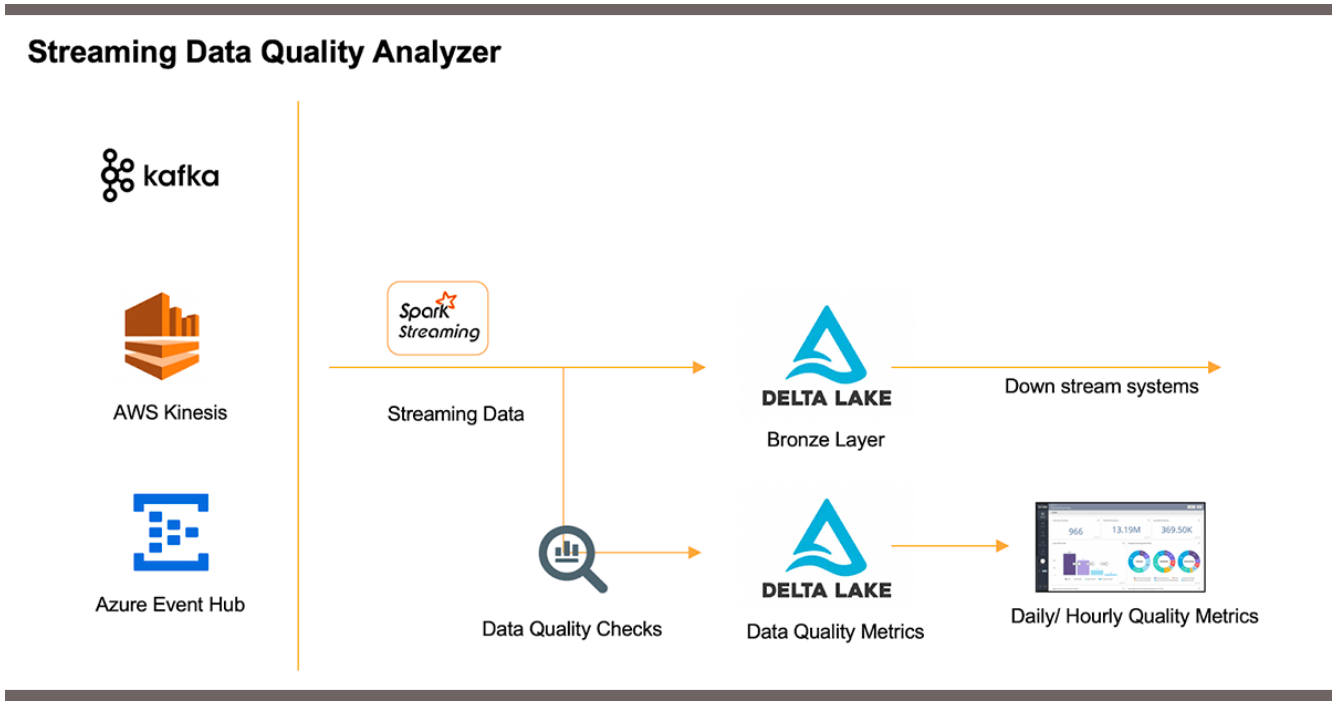
Spark Streaming

❖ Example simple architectures

➢ Apache Spark Streaming acts as a core processing engine seamlessly integrated across diverse data architectures.



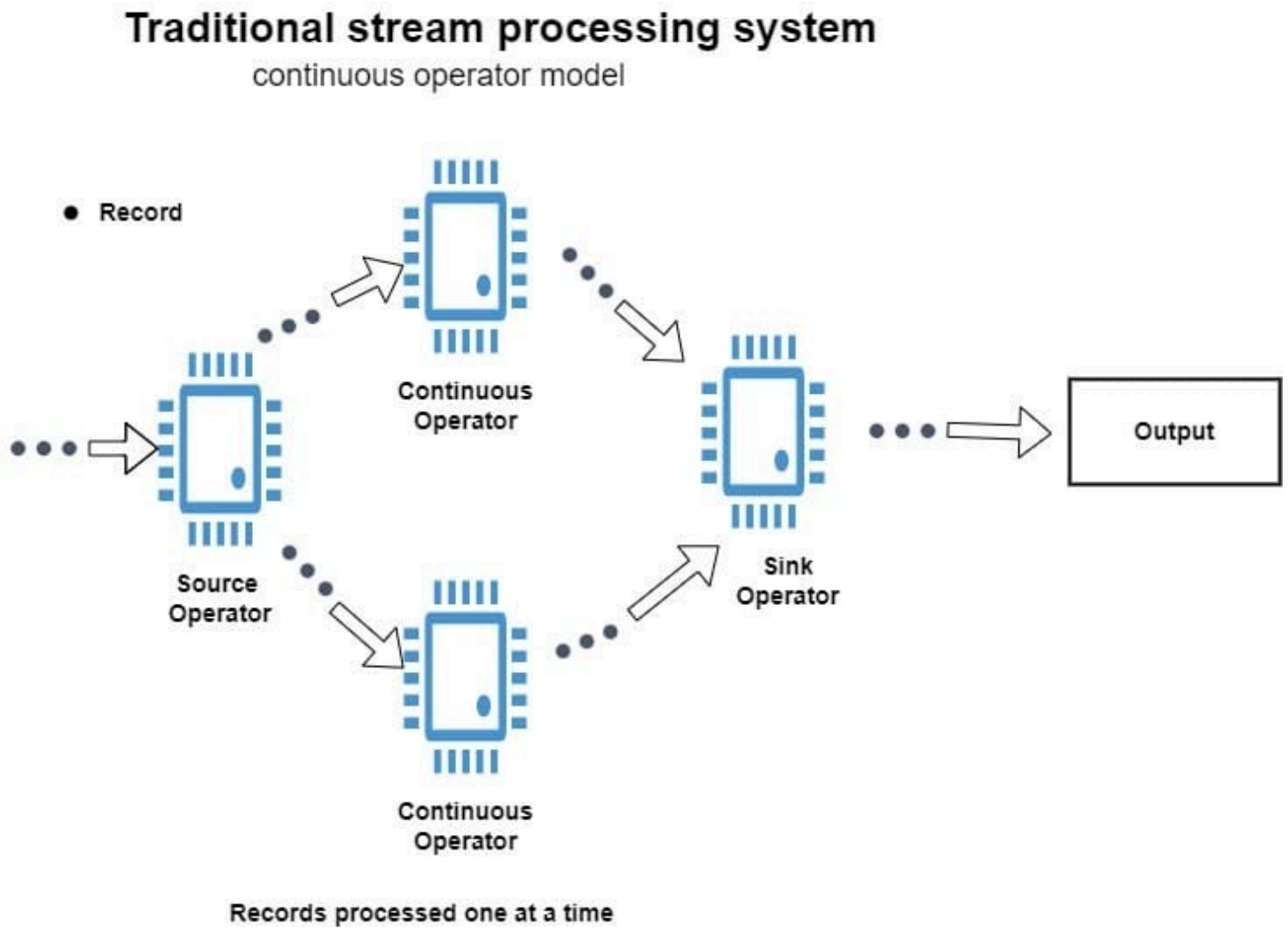
https://developer.hpe.com/blog/streaming-ml-pipeline-for-sentiment-analysis-using-apache-apis-kafka-spark-and-drill-part-2/



https://www.databricks.com/blog/2020/03/04/how-to-monitor-data-stream-quality-using-spark-streaming-and-delta-lake.html

Spark Streaming

❖ Traditional stream processing system

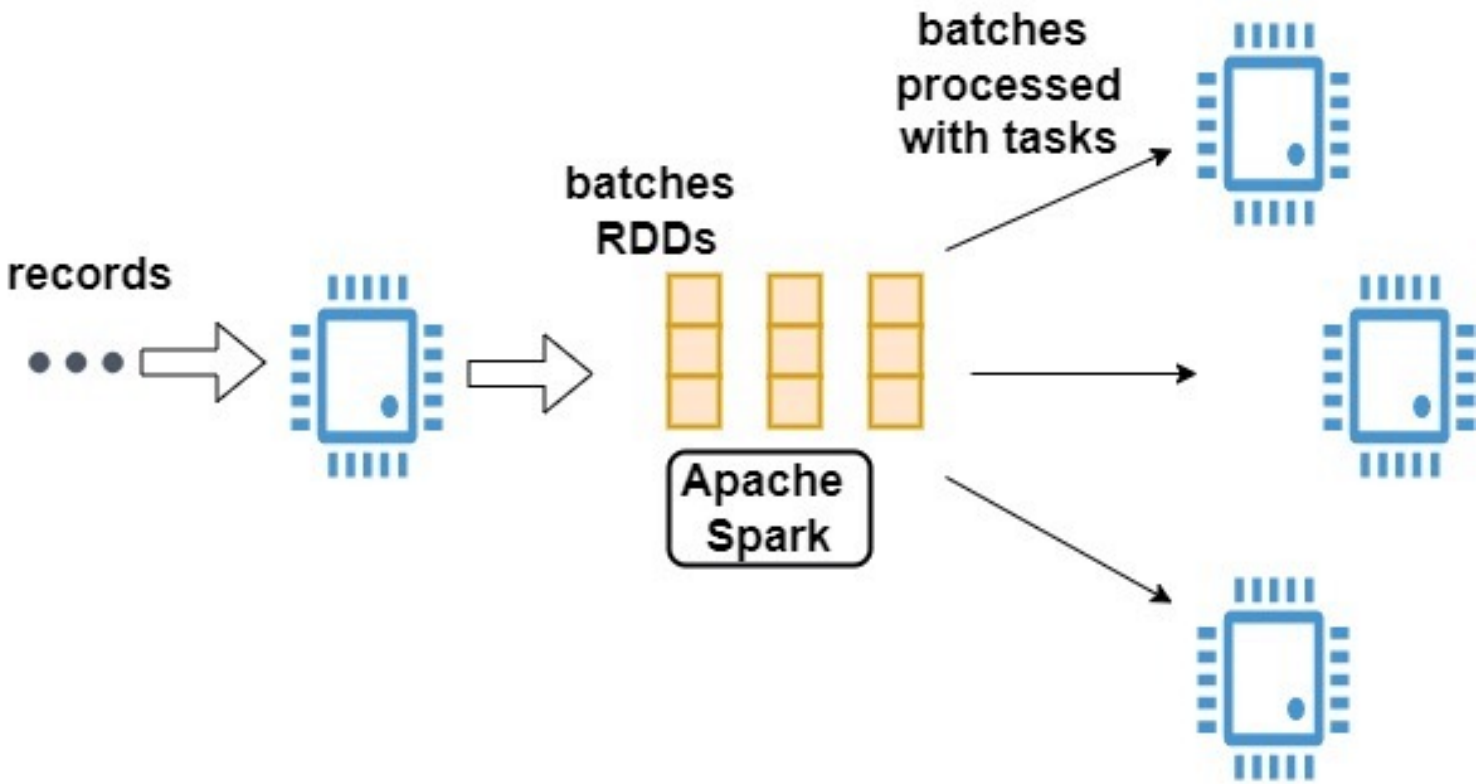


Work as follows:

- There is a set of **worker nodes**, each of which run one or more **continuous operators**.
- Each continuous operator processes the **streaming data one record** at a time and forwards the records to other **operators** in the **pipeline**.
- There are **"source" operators** for **receiving data** from **ingestion systems**, and **"sink" operators** that **output** to **downstream systems**.

Spark Streaming

❖ Spark Streaming Architecture: Discretized Streams



Records processed in batches with short tasks
Each batch is an RDD (partitioned dataset)

Work as follows:

- Spark Streaming **discretizes** continuous streaming data into tiny, sub-second **micro-batches** rather than processing one record at a time.
- Receivers accept data concurrently and **buffer** it in the memory of Spark worker nodes.
- **Dynamically assigns tasks** to workers based on data locality and available resources.
- Processes batches in milliseconds and outputs results to downstream systems.

Spark Streaming

❖ Benefits of Discretized Stream Processing

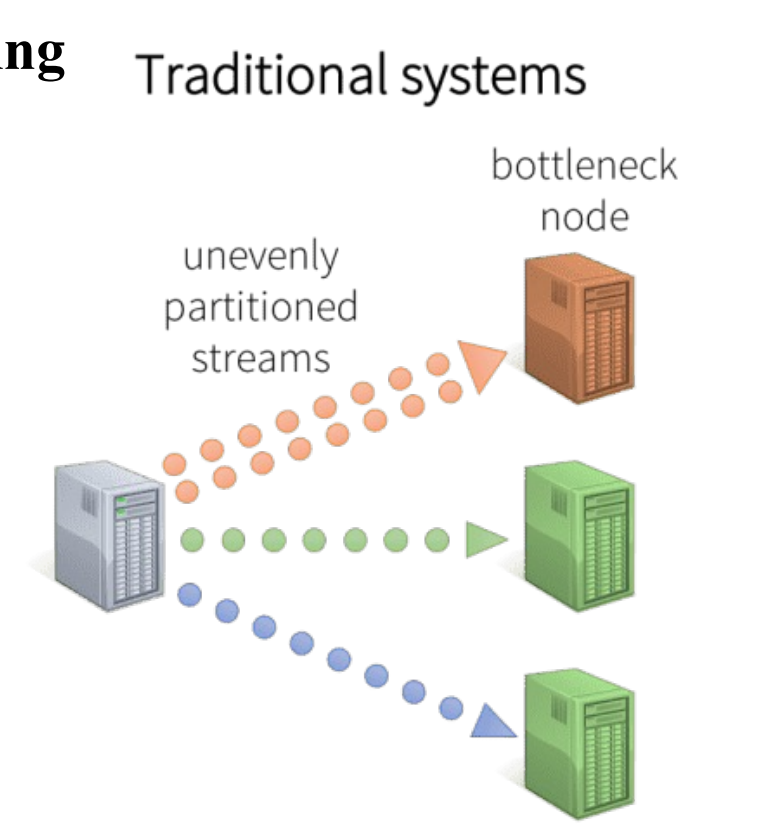
1 Dynamic load balancing

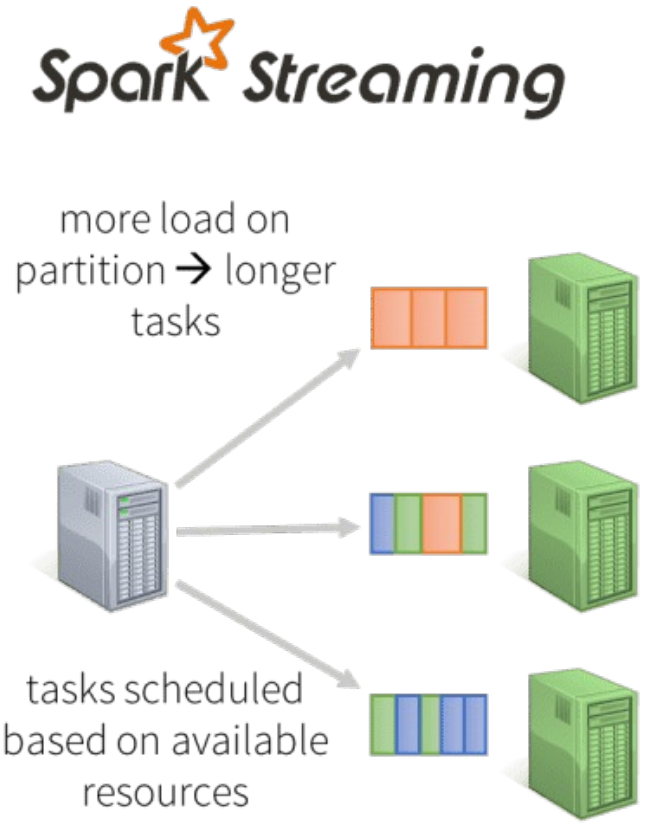
Traditional systems

Assigning nodes statically to data partitions can lead to intensive partitions creating severe **bottlenecks and slowing down.**

Spark Streaming

Discretizes data into micro-batches, **allowing** for **fine-grained computation allocation**.

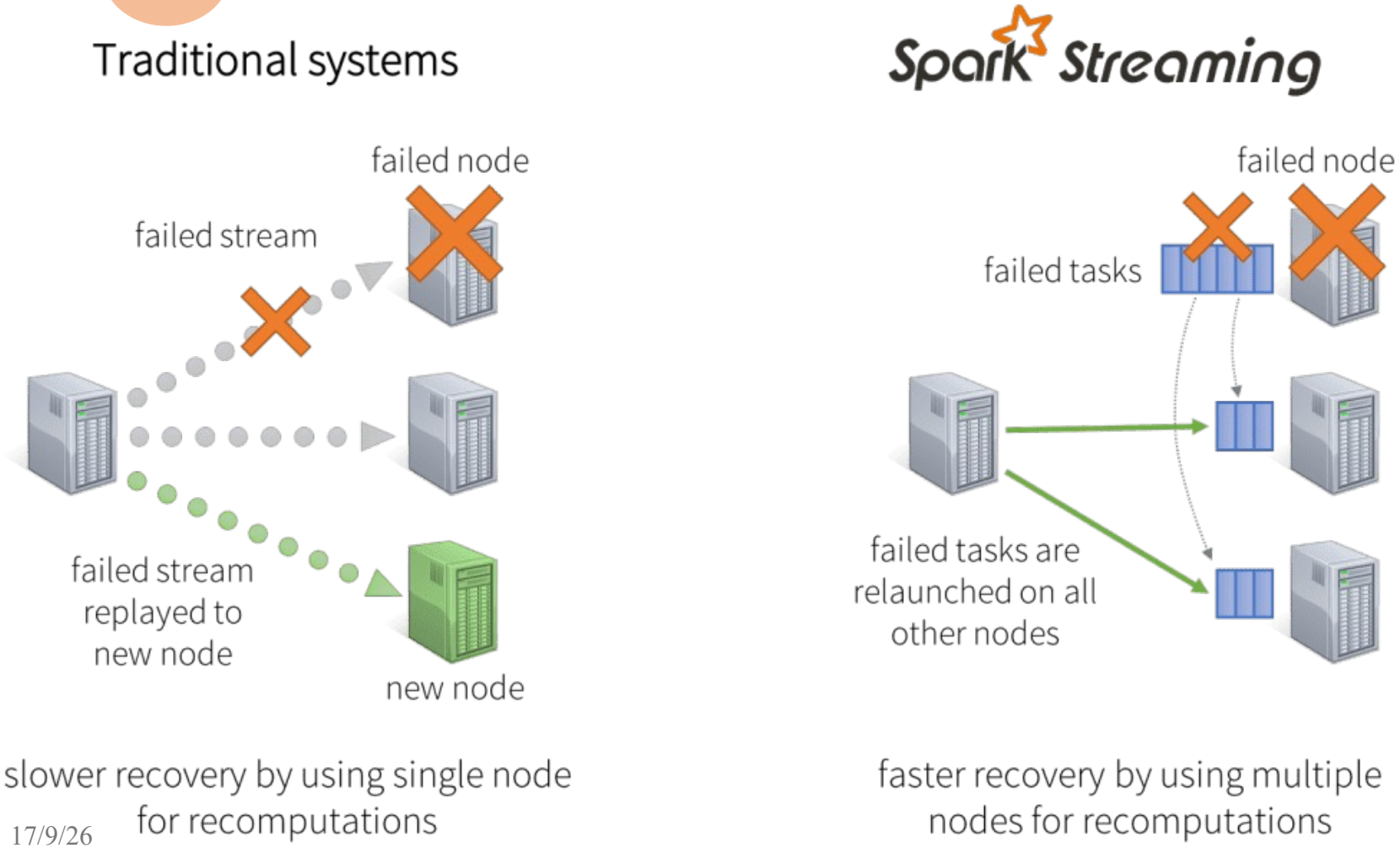




Spark Streaming

❖ Benefits of Discretized Stream Processing

2 **Fast failure and staggler recovery**



Traditional systems

Restart failed continuous operators on a single replacement node.

Slower recovery due to single-node bottlenecks

Spark Streaming

Relaunch failed tasks in parallel across all other healthy nodes in the cluster.

Faster recovery by distributing the recomputation.

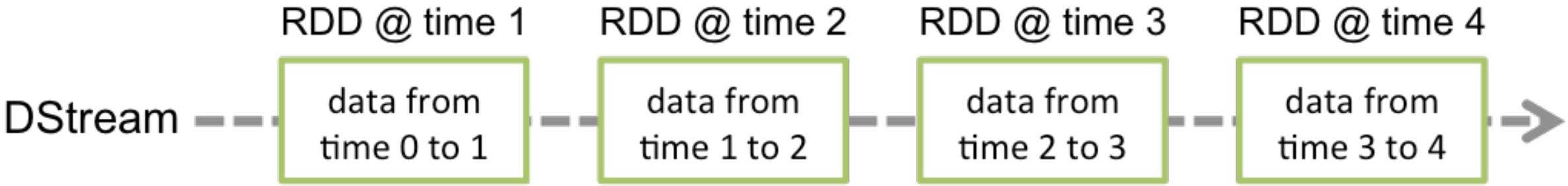
Spark Streaming

❖ Benefits of Discretized Stream Processing

3 **Unification of batch, streaming, and interactive analytics**



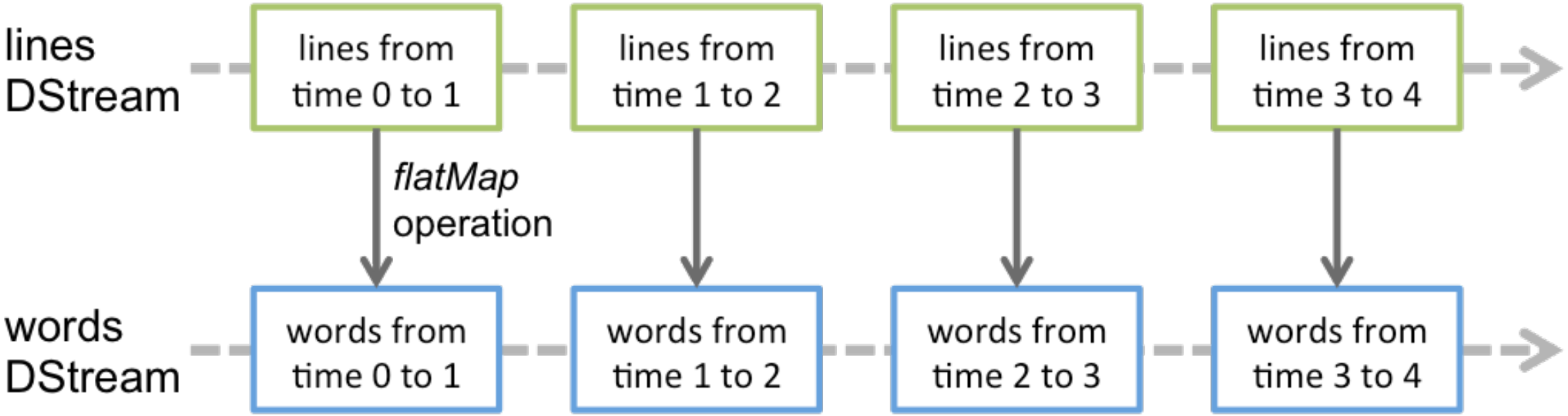
- The **primary programming abstraction** is the **DStream** (distributed stream), which functions **as a continuous sequence of RDDs**.



Spark Streaming

❖ Example

Let's say we want to count the number of words in text data received from a data server listening on a TCP socket



Spark Streaming

❖ Implementation

Let's say we want to count the number of words in text data received from a data server listening on a TCP socket

```python
def main():
    hostname = sys.argv[1]
    port = int(sys.argv[2])

    # Create SparkContext with 2 threads
    sc = SparkContext("local[2]", "NetworkWordCount")
    sc.setLogLevel("WARN")

    # Create StreamingContext with 1 second batch interval
    ssc = StreamingContext(sc, 1)

    # Create DStream connecting to TCP socket
    lines = ssc.socketTextStream(hostname, port)

    # Split each line into words
    words = lines.flatMap(lambda line: line.split(" "))

    # Count each word in each batch
    pairs = words.map(lambda word: (word, 1))
    wordCounts = pairs.reduceByKey(lambda x, y: x + y)

    # Print results
    wordCounts.pprint()

    # Start processing

    ssc.start()
    ssc.awaitTermination()


if __name__ == "__main__":
    main()
```

Running

```
python counting_example.py localhost 9999
```

```
================================================
TCP Server started on localhost:9999
Waiting for Spark Streaming to connect...
================================================

Type text and press Enter to send to Spark:
(Press Ctrl+C to stop)

> [+] Client connected: ('127.0.0.1', 64331)
Hello AI Vietnam
  [Sent to 1 client(s)]: Hello AI Vietnam
> Im a student
  [Sent to 1 client(s)]: Im a student
> Helllo World
  [Sent to 1 client(s)]: Helllo World
>
```

From the TCP Server, send messages

Result:

```
-------------------------------------------
Time: 2026-09-07 15:58:44
-------------------------------------------
('AI', 1)
('Hello', 1)
('Vietnam', 1)

-------------------------------------------
Time: 2026-09-07 15:59:03
-------------------------------------------
('Im', 1)
('a', 1)
('student', 1)


-------------------------------------------
Time: 2026-09-07 15:59:17
-------------------------------------------
('Helllo', 1)
('World', 1)

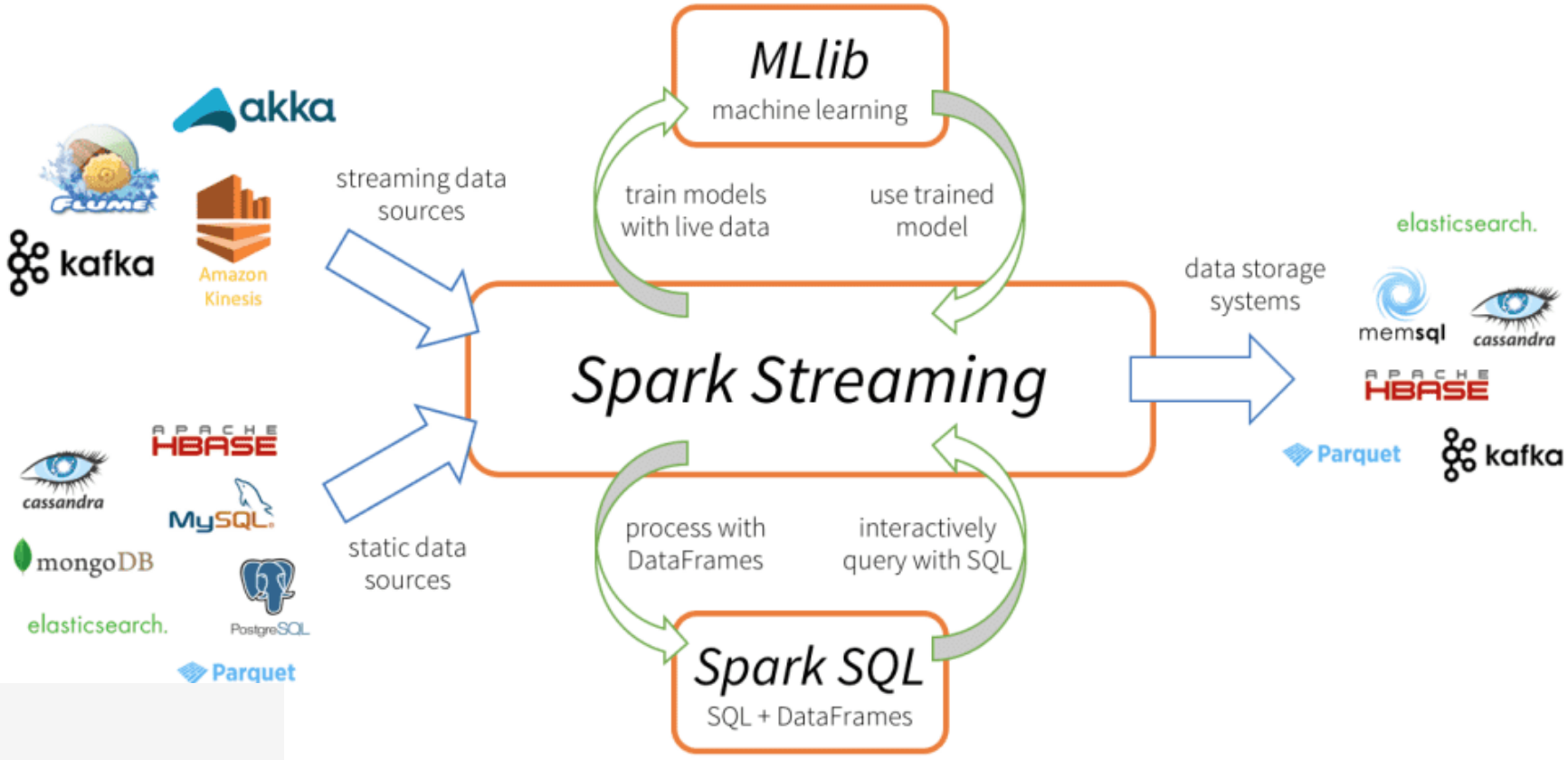
-------------------------------------------
Time: 2026-09-07 15:59:18
-------------------------------------------
```

Spark Streaming

❖ Benefits of Discretized Stream Processing

4 **Advanced analytics like machine learning and interactive SQL**

- Spark interoperability extends to rich libraries like MLlib (machine learning), SQL, DataFrames, and GraphX.
- Example pseudo code: Streaming + MLib




```
// Learn model offline
val model = KMeans.train(dataset, ...)
// Apply model online on stream
val kafkaStream = KafkaUtils.createDStream(...)
kafkaStream.map { event => model.predict(featurize(event)) }
```

Spark Streaming

❖ Example

It **modifies** the **earlier word count example** to generate word counts using DataFrames and SQL. Each RDD is converted to a DataFrame, registered as a temporary table, and then queried using SQL.

```
def getSparkSessionInstance(sparkConf: SparkConf) -> SparkSession:
    if ('sparkSessionSingletonInstance' not in globals()):
        globals()['sparkSessionSingletonInstance'] = SparkSession\
            .builder\
            .config(conf=sparkConf)\
            .getOrCreate()
    return globals()['sparkSessionSingletonInstance']
```

```
if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Usage: sql_network_wordcount.py <hostname> <port> ", file=sys.stderr)
        sys.exit(-1)
    host, port = sys.argv[1:]
    sc = SparkContext(appName="PythonSqlNetworkWordCount")
    ssc = StreamingContext(sc, 1)

    # Create a socket stream on target ip:port and count the
    # words in input stream of \n delimited text (e.g. generated by 'nc')
    lines = ssc.socketTextStream(host, int(port))
    words = lines.flatMap(lambda line: line.split(" "))
```

```
# Convert RDDs of the words DStream to DataFrame and run SQL query
def process(time: datetime.datetime, rdd: RDD[str]) -> None:
    print("========= %s =========" % str(time))

    try:
        # Get the singleton instance of SparkSession
        spark = getSparkSessionInstance(rdd.context.getConf())

        # Convert RDD[String] to RDD[Row] to DataFrame
        rowRdd = rdd.map(lambda w: Row(word=w))
        wordsDataFrame = spark.createDataFrame(rowRdd)

        # Creates a temporary view using the DataFrame.
        wordsDataFrame.createOrReplaceTempView("words")

        # Do word count on table using SQL and print it
        wordCountsDataFrame = \
            spark.sql("select word, count(*) as total from words group by word")
        wordCountsDataFrame.show()
    except BaseException:
        pass

words.foreachRDD(process)
ssc.start()
ssc.awaitTermination()
```

Spark Streaming

❖ Getting Started

```
# Convert RDDs of the words DStream to DataFrame and run SQL query
def process(time: datetime.datetime, rdd: RDD[str]) -> None:
    print("========= %s =========" % str(time))

    try:
        # Get the singleton instance of SparkSession
        spark = getSparkSessionInstance(rdd.context.getConf())

        # Convert RDD[String] to RDD[Row] to DataFrame
        rowRdd = rdd.map(lambda w: Row(word=w))
        wordsDataFrame = spark.createDataFrame(rowRdd)

        # Creates a temporary view using the DataFrame.
        wordsDataFrame.createOrReplaceTempView("words")

        # Do word count on table using SQL and print it
        wordCountsDataFrame = \
            spark.sql("select word, count(*) as total from words group by word")
        wordCountsDataFrame.show()
    except BaseException:
        pass

words.foreachRDD(process)
ssc.start()
ssc.awaitTermination()
```

From the TCP Server, send messages

Running

```
python counting_sql.py localhost 9999
```

```
> [+] Client connected: ('127.0.0.1', 52796)
hello spark streaming
  [Sent to 1 client(s)]: hello spark streaming
> hello aivn
  [Sent to 1 client(s)]: hello aivn
> Hi spark
  [Sent to 1 client(s)]: Hi spark
>
```

Result

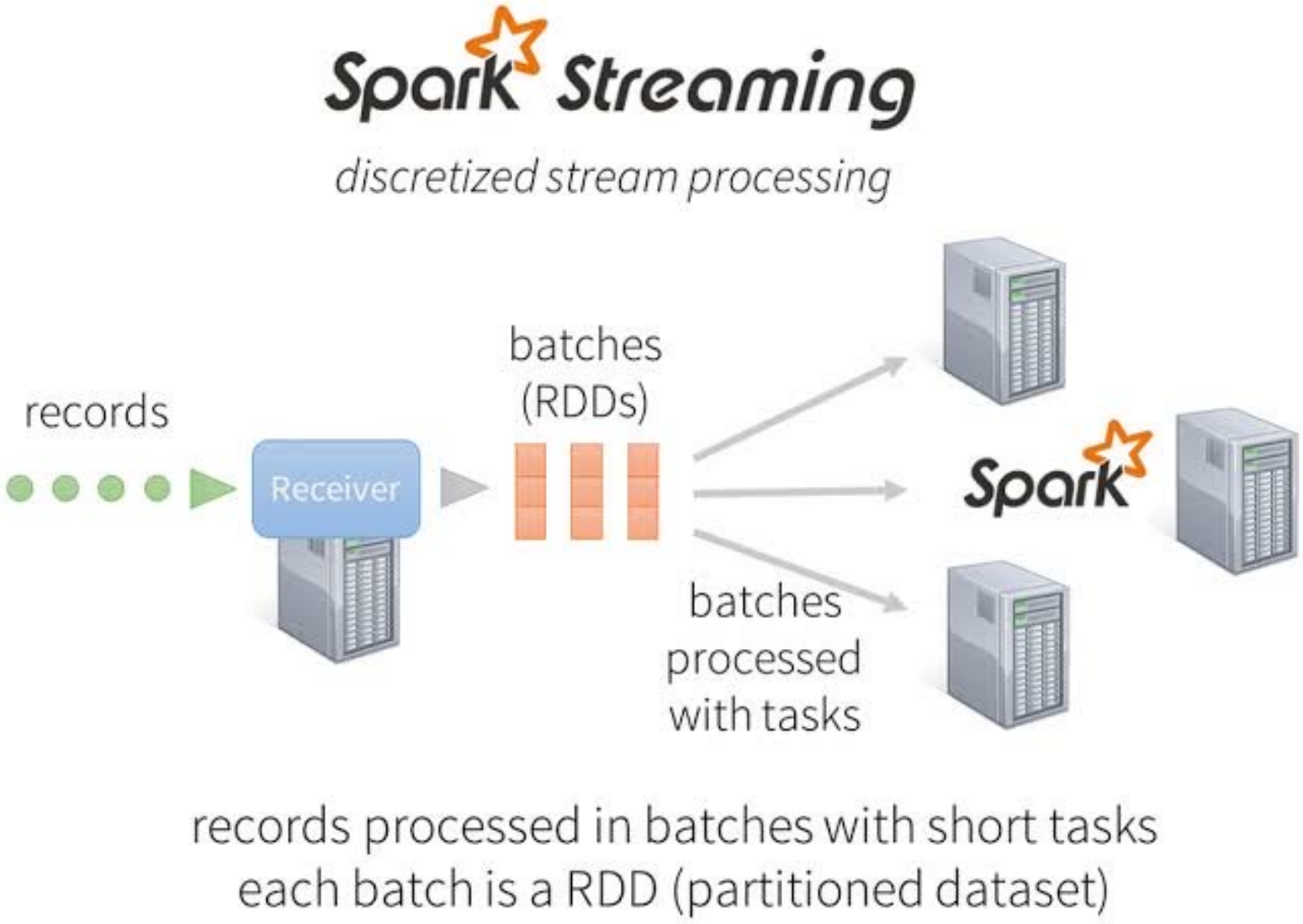
```
========= 2026-09-07 16:56:19 =========
+---------+-----+
|     word|total|
+---------+-----+
|    hello|    1|
|streaming|    1|
|    spark|    1|
+---------+-----+

========= 2026-09-07 16:56:28 =========
+-----+-----+
| word|total|
+-----+-----+
|hello|    1|
| aivn|    1|
+-----+-----+

========= 2026-09-07 16:56:39 =========
+-----+-----+
| word|total|
+-----+-----+
|   Hi|    1|
|spark|    1|
+-----+-----+
```

Spark Streaming

❖ Input DStreams and Receivers

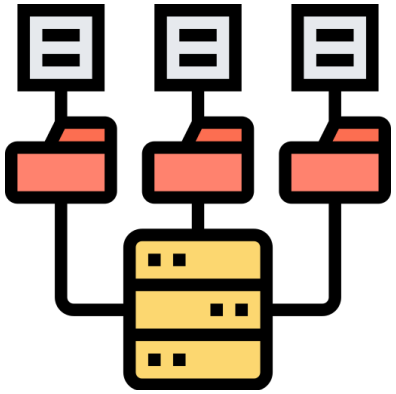


- Input **DStreams** are DStreams **representing** the **stream of input data received** from streaming sources.
- A **Receiver** is an object **associated** with **input DStreams** (except file streams) that **receives data** from a **source** and **stores it in Spark's memory** for processing.
- There are **two categories** of built-in **streaming sources**

Spark Streaming

❖ Input DStreams and Receivers

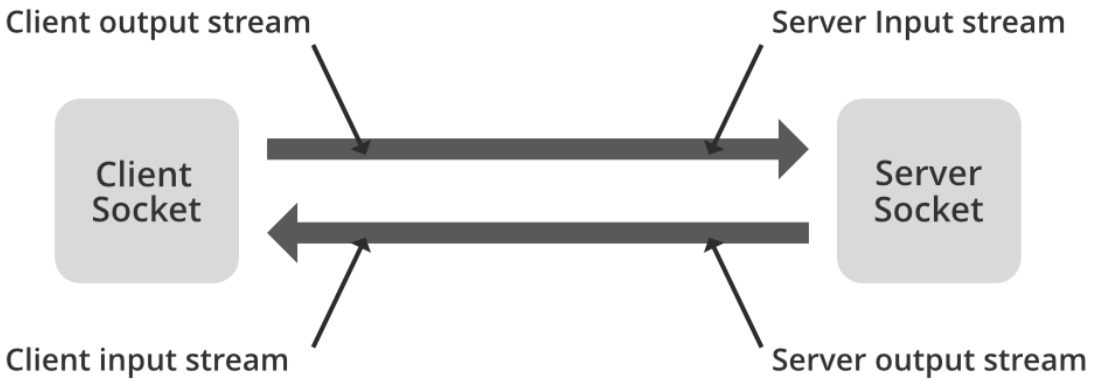
Basic Sources	Advantage Sources

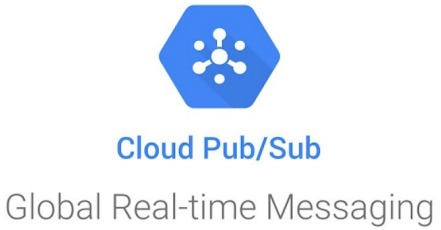














Spark Streaming

❖ Input DStreams and Receivers

Basic Sources

Socket stream connection:

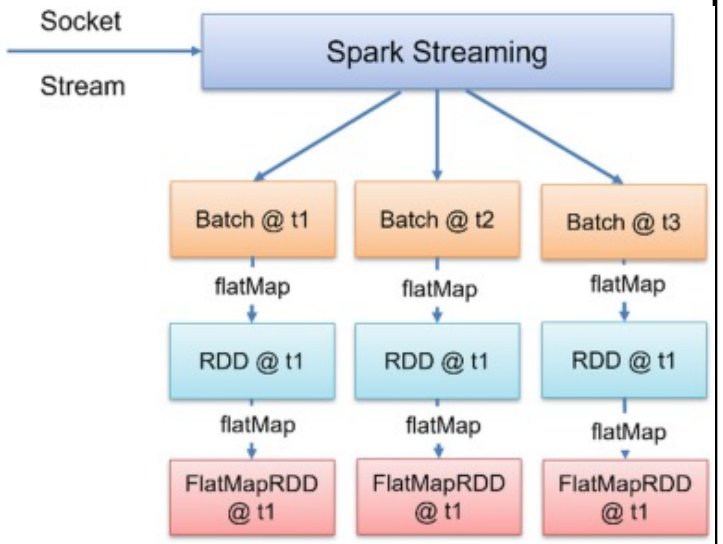
```
DStream Transformation
(word stream)

val ssc = new StreamingContext(conf,
Seconds(5))

// Create a DStream that will connect to
hostname:port, like localhost:50050
val lines =
ssc.socketTextStream("localhost", 50050)

// Split each line into words
val words = lines.flatMap(_.split(" "))

// Count each word in each batch
val pairs = words.map(word => (word, 1))
val wordCounts = pairs.reduceByKey(_ + _)
```



Advantage Sources

```
# 1. Initialize Spark Session with Kafka dependency package
spark = SparkSession.builder \
    .appName("KafkaSparkStreaming") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

# 2. Define the schema matching the Producer JSON
schema = StructType([
    StructField("username", StringType(), True),
    StructField("action", StringType(), True),
    StructField("timestamp", IntegerType(), True)
])

# 3. Read stream from Kafka
kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "test-topic") \
    .option("startingOffsets", "latest") \
    .load()

# Kafka provides data as binary (key, value). Cast the value to String.
kafka_string_df = kafka_df.selectExpr("CAST(value AS STRING)")

# 4. Parse the JSON string stream using the schema
parsed_df = kafka_string_df \
    .withColumn("data", from_json(col("value"), schema)) \
    .select("data.*")

# 5. Output the stream to the Console window
query = parsed_df.writeStream \
    .outputMode("append") \
    .format("console") \
    .start()

query.awaitTermination()
```

Spark Streaming

❖ Transformations on DStreams

Basic Operations

- **Similar to RDDs**, transformations allow the data in the input DStream to be modified.
- DStreams **support many of the transformations** available on normal Spark RDD's

```
// Read text file from the provided HDFS path
val lines = spark.sparkContext.textFile(filePath)

// Filter lines that start with "ERROR"
val errors = lines.filter(_.startsWith("ERROR"))

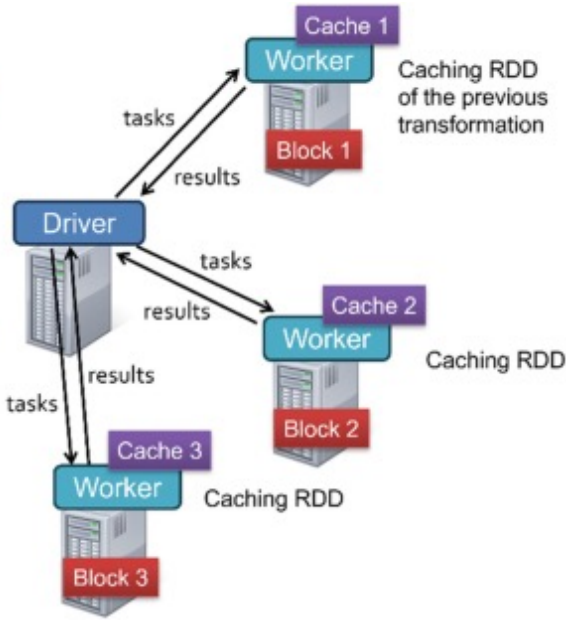
// Extract messages assuming tab-delimiters
val messages =
errors.map(_.split('\t').lift(2).getOrElse(""))

// Cache the extracted messages
val cachedMsgs = messages.cache()

// Count occurrences of "foo" and "bar"
val fooCount =
cachedMsgs.filter(_.contains("foo")).count()
val barCount =
cachedMsgs.filter(_.contains("bar")).count()
```

Base RDD

Transformed RDD



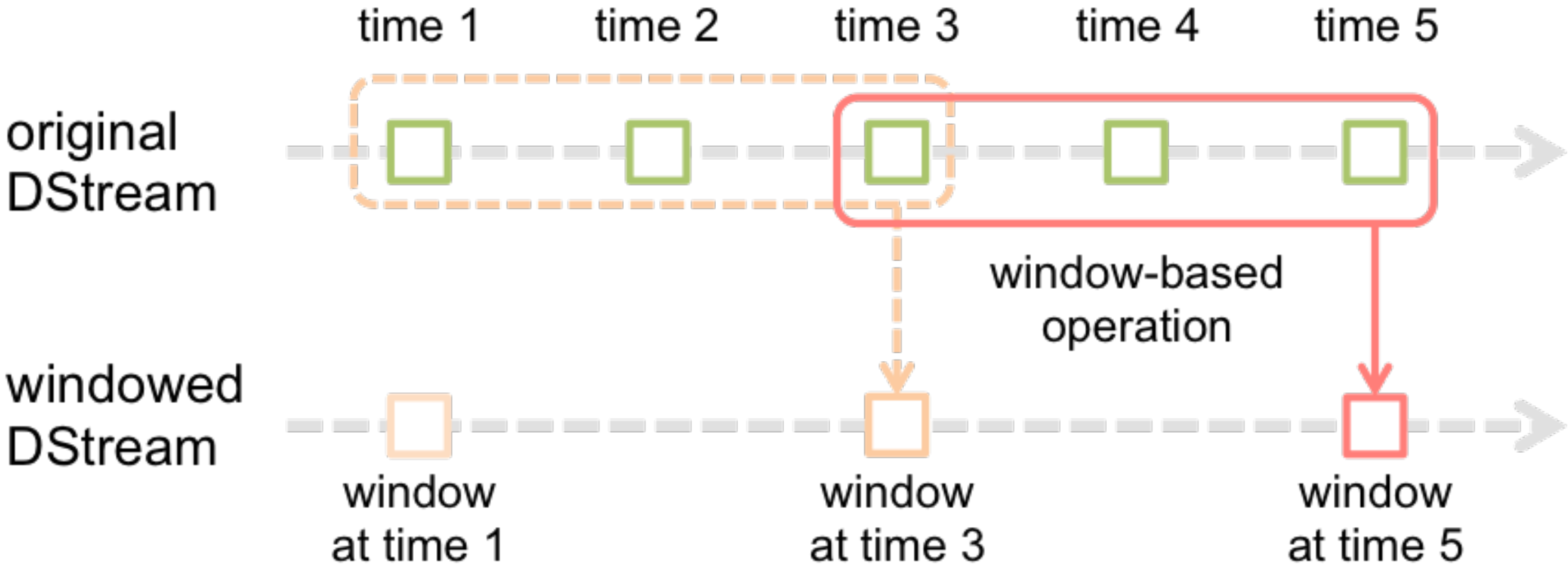
Transformation	Meaning
map(*func*)	Return a new DStream by passing each element of the source DStream through a function *func*.
flatMap(*func*)	Similar to map, but each input item can be mapped to 0 or more output items.
filter(*func*)	Return a new DStream by selecting only the records of the source DStream on which *func* returns true.
repartition(*numPartitions*)	Changes the level of parallelism in this DStream by creating more or fewer partitions.
union(*otherStream*)	Return a new DStream that contains the union of the elements in the source DStream and *otherDStream*.
count()	Return a new DStream of single-element RDDs by counting the number of elements in each RDD of the source DStream.
reduce(*func*)	Return a new DStream of single-element RDDs by aggregating the elements in each RDD of the source DStream using a function *func* (which takes two arguments and returns one). The function should be associative and commutative so that it can be computed in parallel.
countByValue()	When called on a DStream of elements of type K, return a new DStream of (K, Long) pairs where the value of each key is its frequency in each RDD of the source DStream.
reduceByKey(*func*, [*numTasks*])	When called on a DStream of (K, V) pairs, return a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function. **Note:** By default, this uses Spark's default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property `spark.default.parallelism`) to do the grouping. You can pass an optional `numTasks` argument to set a different number of tasks.
join(*otherStream*, [*numTasks*])	When called on two DStreams of (K, V) and (K, W) pairs, return a new DStream of (K, (V, W)) pairs with all pairs of elements for each key.

Spark Streaming

❖ Transformations on DStreams

Window Operations

- Spark Streaming also provides ***windowed computations***, which allow you to apply transformations over a sliding window of data.

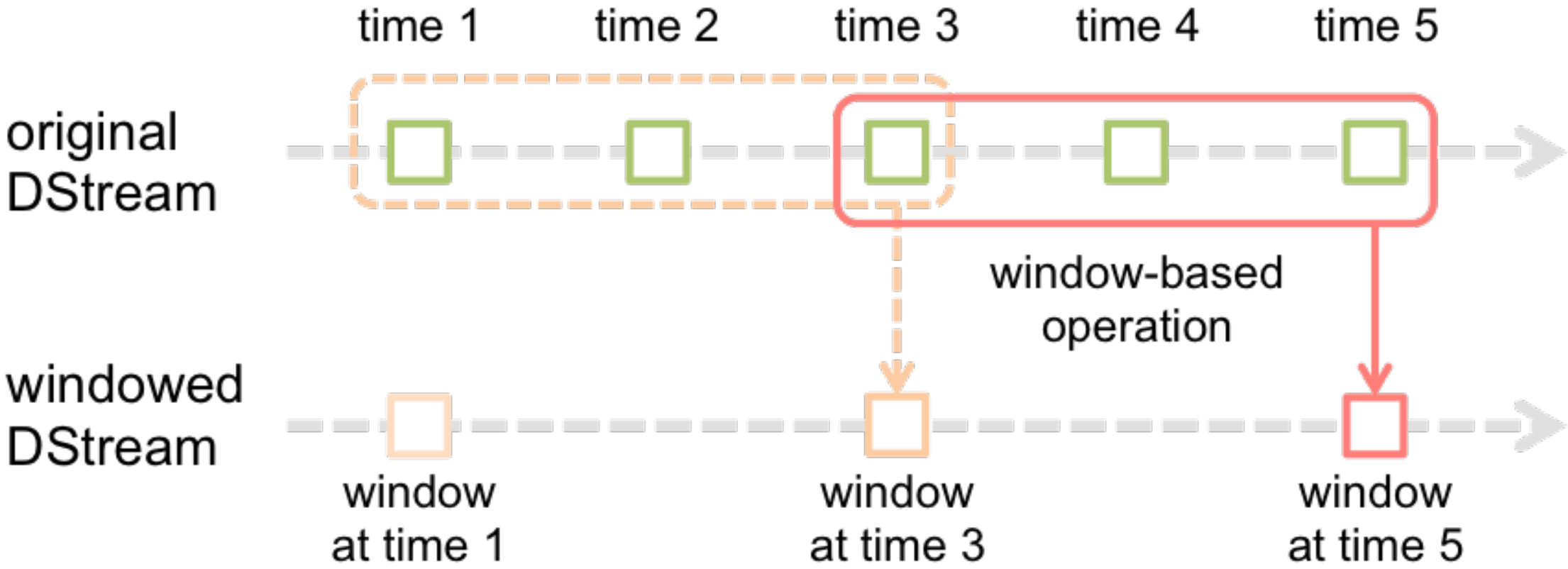


Spark Streaming

❖ Transformations on DStreams

Window Operations

- Spark Streaming also provides ***windowed computations***, which allow you to apply transformations over a sliding window of data.

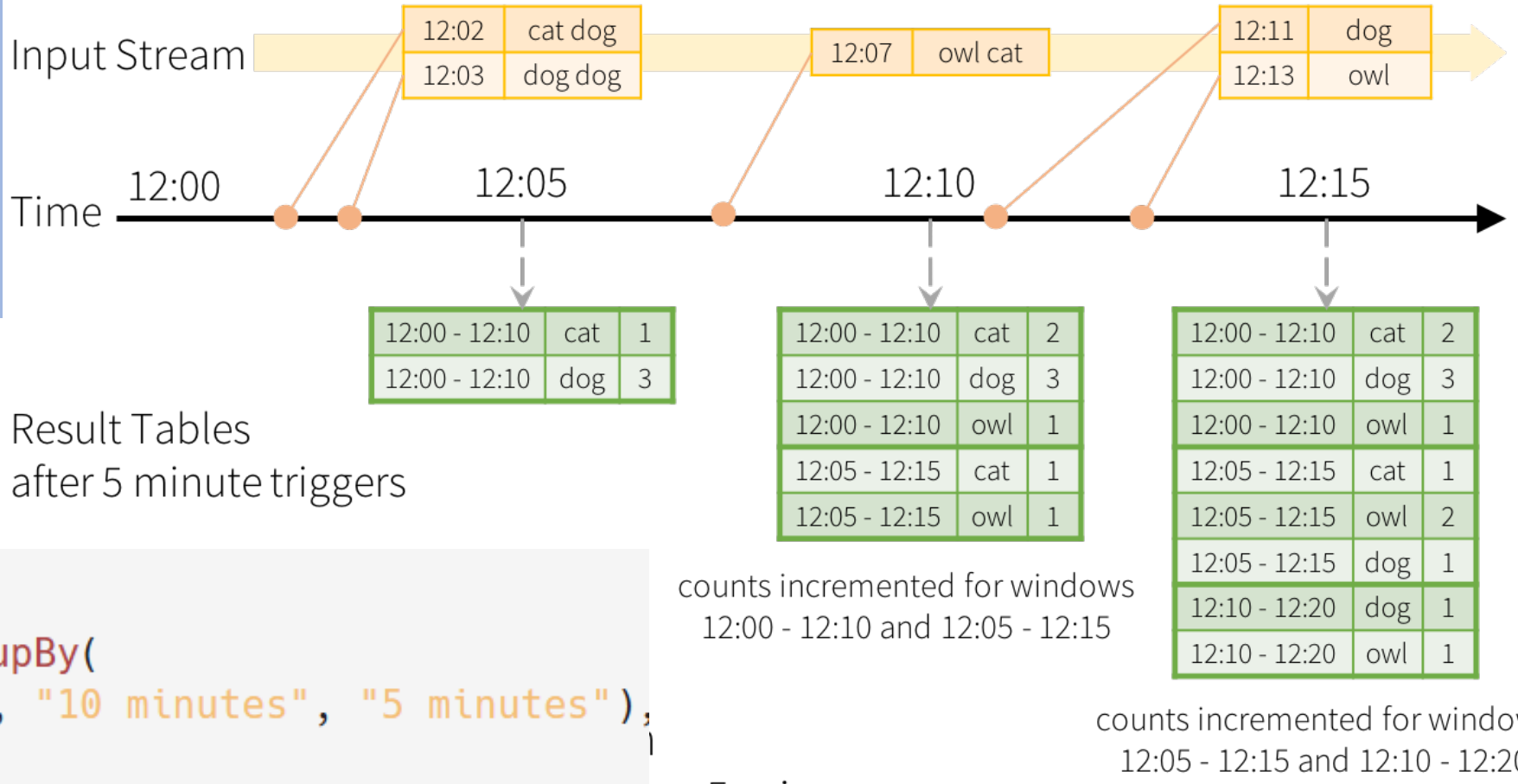


Spark Streaming

❖ Transformations on DStreams

Window Operations on Event Time

Example: Instead of running word counts, we want to *count words within 10-minute windows, updating every 5 minutes.*



Example code:

```
words = ...
windowedCounts = words.groupBy(
    window(words.timestamp, "10 minutes", "5 minutes"),
    words.word
).count()
```

ery 5 mins

Spark Streaming

❖ Transformations on DStreams

Transformation	Meaning
window(*windowLength*, *slideInterval*)	Returns a new **DStream** computed over **windowed batches** of the source.
reduceByWindow(*func*, *windowLength*, *slideInterval*)	**Aggregates elements** over a **sliding window** into a **single-element stream** using ***func***
countByWindow(*windowLength*, *slideInterval*)	Returns a sliding window **count of elements** in the stream.
reduceByKeyAndWindow(*func*, *invFunc*, *windowLength*, *slideInterval*, [*numTasks*])	Efficient incremental calculation: uses ***func*** for new data and ***invFunc*** to **"inverse reduce" old data**, leaving the window.
countByValueAndWindow(*windowLength*, *slideInterval*, [*numTasks*])	Returns **(K, Long) pairs** showing the **frequency of each key** within a sliding window.

Spark Streaming

❖ Transformations on DStreams

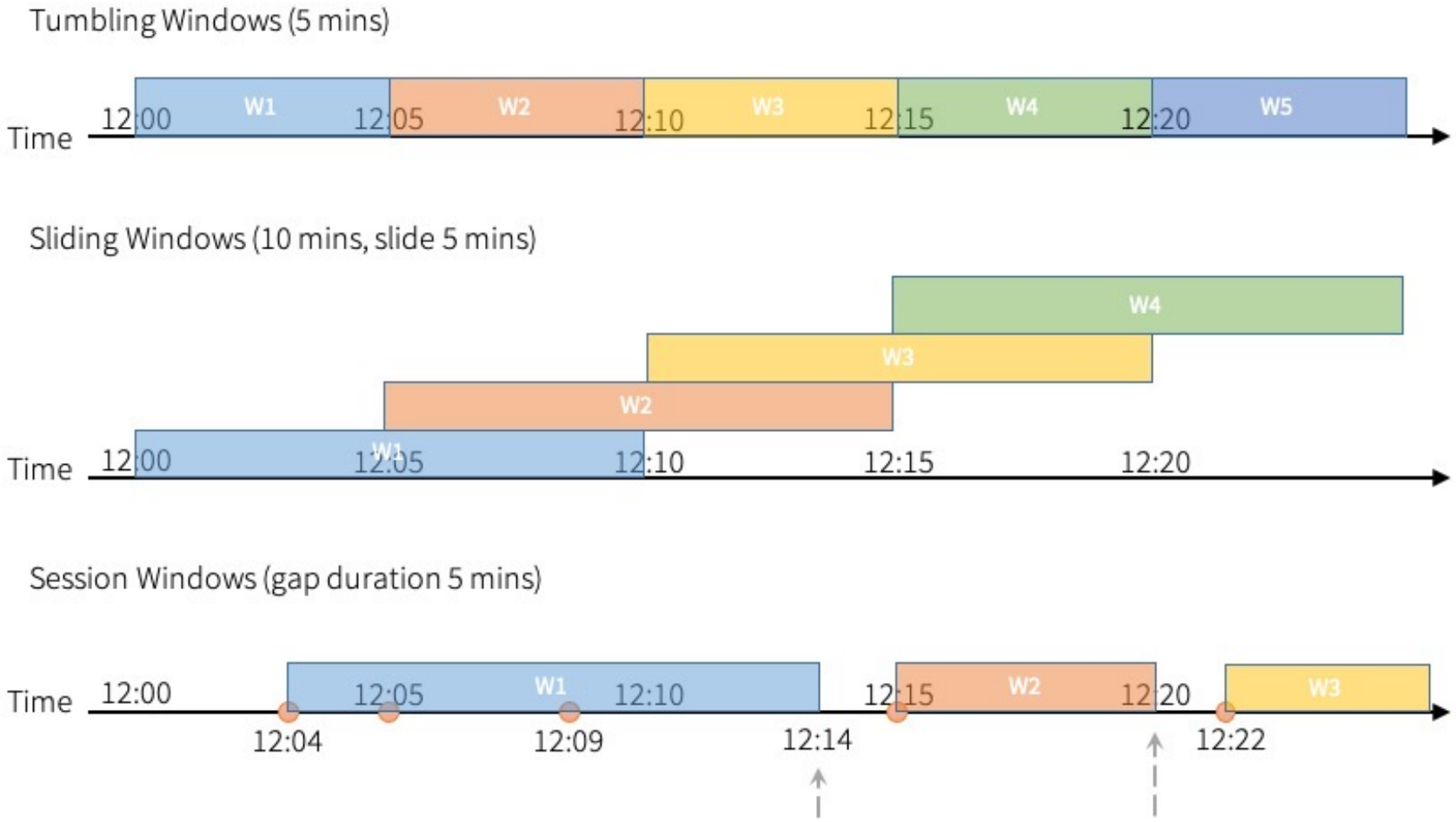
Types of time windows

- Spark **supports** three types of time windows: **tumbling** (fixed), **sliding**, and **session**.

```
window("eventTime", "5 minute")
```

```
window("eventTime", "10 minutes", "5 minutes")
```

```
session_window(events.timestamp, "5 minutes")
```



Spark Streaming

❖ Transformations on DStreams

UpdateStateByKey Operation

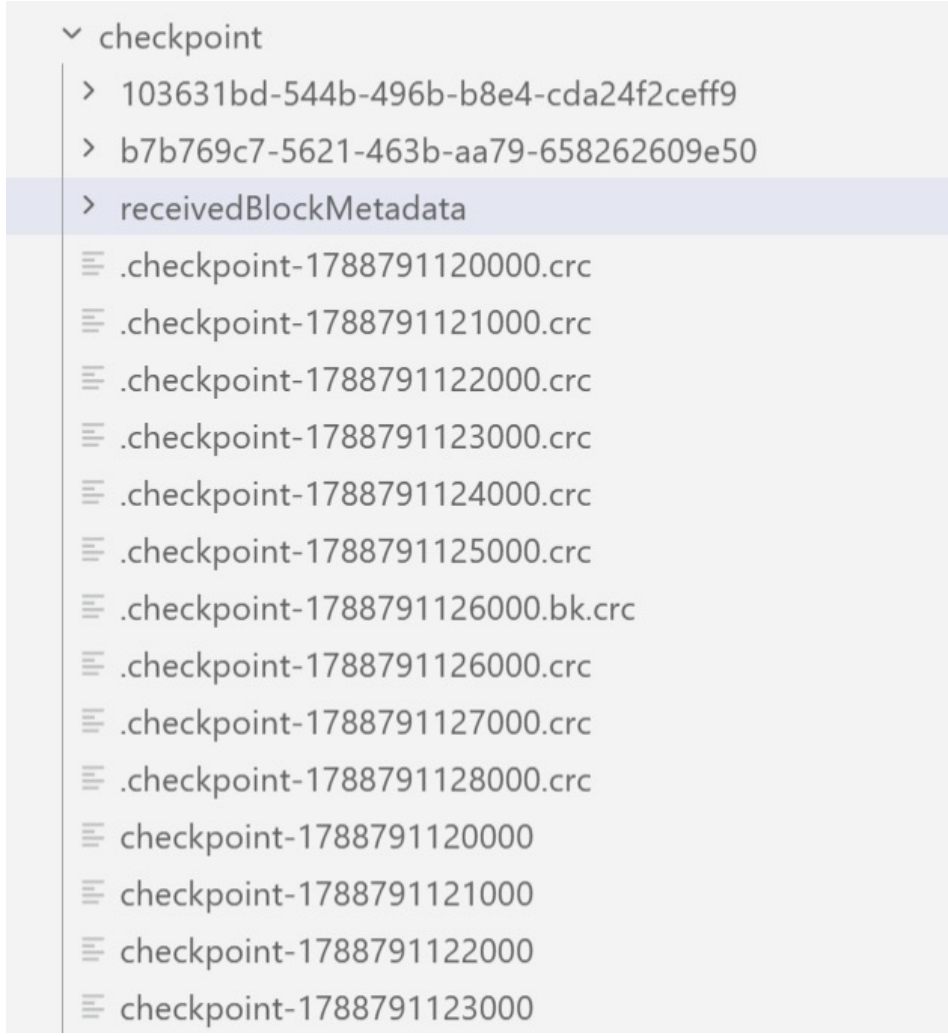
The **updateStateByKey** operation allows you to maintain arbitrary state **while continuously updating** it with **new information**.

Steps to do

1. Define the state - The state can be an arbitrary data type.
2. Define the state update function

Note

Note that using **updateStateByKey requires** the **checkpoint directory** to be configured.



Spark Streaming

❖ Transformations on DStreams

UpdateStateByKey Operation

Example:
We want to maintain a running count of each word seen in a **text data stream** (Develop from the previous example).

Input:

```
================================================
TCP Server started on localhost:9999
Waiting for Spark Streaming to connect...
================================================

Type text and press Enter to send to Spark:
(Press Ctrl+C to stop)

> [+] Client connected: ('127.0.0.1', 52293)
hello ai vietnam
  [Sent to 1 client(s)]: hello ai vietnam
> hello spark
  [Sent to 1 client(s)]: hello spark
> hello world
  [Sent to 1 client(s)]: hello world
> hello everyone
  [Sent to 1 client(s)]: hello everyone
>
```

Steps to do

1. Define the state - The state can be an arbitrary data type.
2. Define the state update function

```
def updateFunc(new_values: Iterable[int], last_sum: Optional[int]) -> int:
    return sum(new_values) + (last_sum or 0)
```

```
running_counts = (
    lines.flatMap(lambda line: line.split(" "))
         .filter(lambda w: w != "")
         .map(lambda word: (word, 1))
         .updateStateByKey(updateFunc, initialRDD=initialStateRDD)
)
```

Spark Streaming

❖ Transformations on DStreams

UpdateStateByKey Operation

```
def updateFunc(new_values: Iterable[int], last_sum: Optional[int]) -> int:
    return sum(new_values) + (last_sum or 0)
```

```
def main():
    hostname = sys.argv[1]
    port = int(sys.argv[2])

    # local[2] is required for Streaming (1 receiver + 1 processing)
    sc = SparkContext("local[2]", "StatefulNetworkWordCount")
    sc.setLogLevel("WARN")

    ssc = StreamingContext(sc, 1)

    # Use absolute checkpoint path (Windows-friendly)
    checkpoint_dir = str(Path(__file__).parent / "checkpoint")
    ssc.checkpoint(checkpoint_dir)

    # Initial state
    initialStateRDD = sc.parallelize([("hello", 1), ("world", 1)], 2)

    lines = ssc.socketTextStream(hostname, port)
    running_counts = (
        lines.flatMap(lambda line: line.split(" "))
             .filter(lambda w: w != "")
             .map(lambda word: (word, 1))
             .updateStateByKey(updateFunc, initialRDD=initialStateRDD)
    )

    running_counts.pprint()

    ssc.start()
    ssc.awaitTermination()
```

```
-------------------------------------------
Time: 2026-09-07 21:21:02
-------------------------------------------
('hello', 1)
('world', 1)

-------------------------------------------
Time: 2026-09-07 21:21:03
-------------------------------------------
('hello', 2)
('world', 1)
('ai', 1)
('vietnam', 1)

-------------------------------------------
Time: 2026-09-07 21:21:04
-------------------------------------------
('hello', 2)
('world', 1)
('ai', 1)
('vietnam', 1)

-------------------------------------------
Time: 2026-09-07 21:21:05
-------------------------------------------
('hello', 3)
('world', 1)
('ai', 1)
('vietnam', 1)
('spark', 1)
```

```
-------------------------------------------
Time: 2026-09-07 21:21:09
-------------------------------------------
('hello', 4)
('world', 2)
('ai', 1)
('vietnam', 1)
('spark', 1)

-------------------------------------------
Time: 2026-09-07 21:21:10
-------------------------------------------
('hello', 4)
('world', 2)
('ai', 1)
('vietnam', 1)

-------------------------------------------
Time: 2026-09-07 21:21:14
-------------------------------------------
('hello', 5)
('world', 2)
('ai', 1)
('vietnam', 1)
('everyone', 1)
('spark', 1)

-------------------------------------------
Time: 2026-09-07 21:21:15
-------------------------------------------
('hello', 5)
('world', 2)
('ai', 1)
('vietnam', 1)
('everyone', 1)
('spark', 1)
```

Spark Streaming

❖ Output Operations on DStreams

Output Operation	Meaning
print()	Prints the elements of each batch (useful for debugging).
saveAsTextFiles(*prefix*, [*suffix*])	Save the contents of this DStream as **text files**.
saveAsObjectFiles(prefix, [suffix])	Save this DStream's contents as SequenceFiles of **serialized Java objects**.
saveAsHadoopFiles(prefix, [suffix])	Save the contents of this DStream as **Hadoop files**.
foreachRDD(func)	Applies **func** to each RDD to **push data** to **external systems** (e.g., database).

https://spark.apache.org/docs/latest/streaming-programming-guide.html

Spark Streaming

❖ Example code

Receiver

Streaming Context

```
object StreamingApp extends App {

val sparkConf = new SparkConf().setMaster("local[*]").setAppName("StreamingApp")

val streamingContext = new StreamingContext(sparkConf, Seconds(5))

val lines: ReceiverInputDStream[String] = streamingContext.socketTextStream("localhost", 9000)

val words: DStream[String] = lines.flatMap(_.split(" "))

val filteredWords = words.filter(!_.trim.isEmpty)

val pairs: DStream[(String, Int)] = filteredWords.map(word => (word, 1))

val wordCounts: DStream[(String, Int)] = pairs.reduceByKey(_ + _)

wordCounts.print()

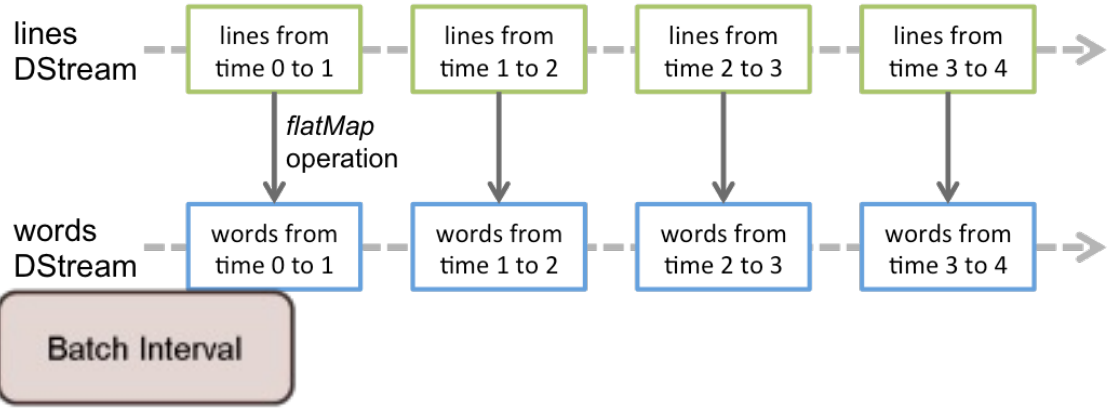
streamingContext.start()
streamingContext.awaitTermination()

}
```

Batch Interval

Transformations on DStreams

Output Operations on DStreams



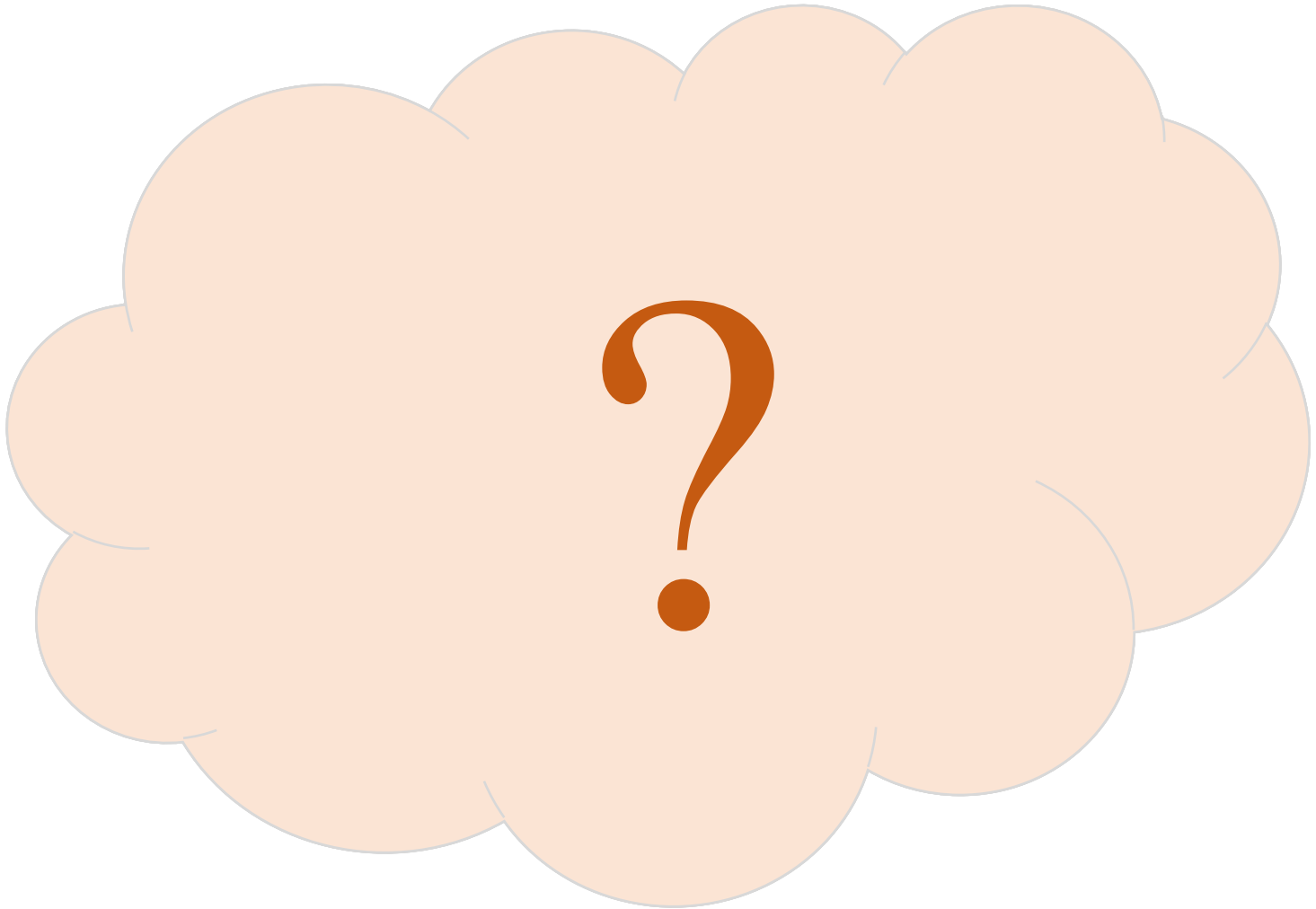
Spark Streaming Conceptual Overview

References

References

- https://www.projectpro.io/article/spark-streaming-example/540
- https://developer.hpe.com/blog/streaming-ml-pipeline-for-sentiment-analysis-using-apache-apis-kafka-spark-and-drill-part-2/
- https://www.databricks.com/blog/2020/03/04/how-to-monitor-data-stream-quality-using-spark-streaming-and-delta-lake.html
- https://spark.apache.org/docs/latest/streaming-programming-guide.html
- https://www.databricks.com/blog/2021/10/12/native-support-of-session-window-in-spark-structured-streaming.html
- https://link.springer.com/chapter/10.1007/978-3-031-69366-3_6
- https://spark.apache.org/docs/latest/streaming/performance-tips.html
- https://www.domo.com/learn/article/data-ingestion-vs-data-integration
- https://www.digitalsilk.com/digital-trends/how-much-data-is-generated-per-day
- https://www.mygreatlearning.com/blog/what-is-data-ingestion

Question




Thank you!